

REPRESENTATIVE LEARNING FOR DISTRIBUTED SUPPORT VECTOR
MACHINES

by

JOSEPH CASEY

KEREN LI, COMMITTEE CHAIR
BAOCHENG GENG
MUHAMMAD MOHEBUJJAMAN
ROMAN SHTERENBERG
BOXIANG WANG

A DISSERTATION

Submitted to the graduate faculty of The University of Alabama at Birmingham,
in partial fulfillment of the requirements for the degree of
Doctor of Philosophy

BIRMINGHAM, ALABAMA

2026

REPRESENTATIVE LEARNING FOR DISTRIBUTED SUPPORT VECTOR
MACHINES

JOSEPH CASEY

APPLIED MATHEMATICS

ABSTRACT

Distributed learning with nonsmooth objective functions is known to be particularly challenging when working under the constraints of limited sharing among clients. The poor behavior of nonsmooth functions diminishes the reliability and robustness of distributed algorithms aimed at solving minimization problems at a global level. Classical algorithms can diverge while more modern algorithms tend to require excessive amounts of data and computation to ensure a reasonable solution.

The foundation of this work is the lifted Empirical Risk Minimization (Lifted ERM) framework. Rather than optimizing a loss over deterministic observations, each observation is replaced by a local probability distribution representing small perturbations of the original data. Lifted ERM then averages the loss over all possible local perturbations of the training dataset, analogously to bagging.

To develop and evaluate this framework, we consider the problem of solving distributed Support Vector Machines (DSVMs), a classical learning problem using the nonsmooth hinge loss, based on Lifted ERM. Our choice to work at the risk level makes our problem extendable to other nonsmooth optimization problems while leaving us with a strong enough algorithm to solve real-world tasks. The lifted formulation transforms the classical hinge loss problem into a smooth optimization problem while preserving its underlying classification structure. More importantly, the framework provides a principled mechanism for constructing distributional representatives, allowing nodes to communicate local information through compact distributional summaries rather than raw data, gradients, or model parameters. This leads to communication-efficient distributed algorithms that naturally extend to kernel SVM and provide a foundation for applying Lifted ERM to other nonsmooth learning problems.

Finally, we study a separate representative-based Collaborative Transfer Learning (CTL) framework in which representatives serve as a shared synthetic dataset for knowledge distillation. Energy-based weights are used to identify informative collaborators, enabling personalized learning across nodes with heterogeneous feature distributions without requiring direct parameter sharing.

We demonstrate theoretically and experimentally the robustness of our algorithms, while remaining under the usual communication constraints of distributed machine learning.

Keywords: support vector machines, distributed learning, federated learning, nodes, representatives, kernel trick

TABLE OF CONTENTS

ABSTRACT	iii
LIST OF TABLES	viii
LIST OF FIGURES	xi
CHAPTER 1. INTRODUCTION	1
Motivation	1
Problem Statement	2
Contributions	3
Dissertation Organization	4
CHAPTER 2. BACKGROUND AND LITERATURE REVIEW	6
1. Distributed Learning and FL	6
1.1. Heterogeneity	7
1.2. Personalized Learning	8
2. Representative Learning	9
2.1. Notation	9
2.2. Representatives for GLMs	10
2.3. Representatives in DML	11
2.4. Coresets	12
2.5. Data Condensation	12
3. Support Vector Machine	12
3.1. Hard-Margin Formulation	13
3.2. Soft-margin Formulation	13
3.3. Dual Form of SVM	13
3.4. Kernels in SVM	15
3.5. Optimizing SVM	17
4. Empirical Risk Minimization	20
4.1. Risk	20
4.2. Empirical Risk	20
4.3. Empirical Risk Minimization	20
4.4. Regularized Distributed Learning Objective	21
5. Support Vector Machine	23
5.1. Support Vector Methods	23
5.2. Consensus-based SVM Using ADMM	24
CHAPTER 3. REPRESENTATIVE LEARNING FOR DSVM	25
1. Methodology	25
1.1. ERM Reformulation and Lifted Hinge Loss	25

1.2. Loss Properties	27
1.3. Lifted Loss Simulations	32
1.4. Representative Constructions	44
1.5. Distributed Algorithm	49
2. Experiments	51
2.1. Representatives for Single Node Training	51
2.2. Simulations on Synthetic Data	55
3. Theoretical Analysis	74
CHAPTER 4. KERNEL EXTENSION FOR DSVM	103
1. Motivation	103
2. Kernel Lifted SVM	105
2.1. Notation and Derivatives	107
3. Representatives for Kernel SVM	108
3.1. Representatives using Dictionary Span	113
3.2. Distributed Loss and Algorithm	115
4. Experiments	116
4.1. Experimental Design	116
4.2. Public Dictionary	117
4.3. Comparisons	117
5. Experiments with RFFs	124
5.1. Background	124
5.2. Experiments	125
CHAPTER 5. REPRESENTATIVE BASED COLLABORATIVE LEARNING	128
1. Motivation	128
2. Collaborative Framework	128
3. Energy Coefficients	131
4. Algorithm	132
5. Simulation Results	133
CHAPTER 6. CONCLUSION AND FUTURE WORK	136
1. Conclusion	136
2. Further Experiments	136
3. Representative Kernel Dictionary	137
4. SVM and Deep Learning	137
5. Representatives For Neural Networks	138
Appendix A. LINEAR AND KERNEL SVM EXPERIMENTAL DETAILS .	139
1. Linear SVM Simulation Details	140
1.1. Simulation Parameters	140
1.2. Synthetic Data Generation	140
1.3. Covariance Selection	141
1.4. Dirichlet Data Partitioning	142
1.5. Covariance Estimation	143
1.6. Initialization	143
2. Kernel SVM Experimental Details	144
2.1. Experimental Configuration	144

2.2. Datasets	144
2.3. Data Preprocessing and Public Dictionary	145
2.4. Distributed Data Partitioning	145
2.5. Kernel and Optimization Parameters	146
Appendix B. COLLABORATIVE TRANSFER LEARNING DETAILS . . .	147
1. CTL	148
REFERENCES	149

LIST OF TABLES

3.1	Hinge (constant) vs Lifted: iterations to tolerance and final accuracy . . .	44
3.2	Training accuracy over 100 simulations for low-overlap classes.	59
3.3	Test accuracy over 100 simulations for low-overlap classes.	60
3.4	Low-overlap linear SVM percentage of simulations with local model agreement (Medium local/outer iterations)	60
3.5	Training accuracy over 100 simulations for high-overlap classes (Medium local/outer iterations).	63
3.6	Test accuracy over 100 simulations for high-overlap classes (Medium local/outer iterations).	64
3.7	High-overlap linear SVM percentage of simulations with local model agreement (Medium local/outer iterations).	64
3.8	Training accuracy over 100 simulations for high-overlap classes (One outer iteration).	65
3.9	Test accuracy over 100 simulations for high-overlap classes (One outer iteration).	66
3.10	High-overlap linear SVM percentage of simulations with local model agreement (One outer iteration).	66
3.11	Training accuracy over 100 simulations for high-overlap classes (One local per outer).	67
3.12	Test accuracy over 100 simulations for high-overlap classes (One local per outer).	68

3.13 High-overlap linear SVM percentage of simulations with local model agreement (One local per outer).	68
3.14 Low overlap parameter distance ($d = 2$)	71
3.15 Low overlap parameter distance ($d = 3$)	71
3.16 Low overlap parameter distance ($d = 5$)	71
3.17 Low overlap parameter distance ($d = 7$)	72
3.18 High overlap parameter distance ($d = 2$)	72
3.19 High overlap parameter distance ($d = 3$)	72
3.20 High overlap parameter distance ($d = 5$)	72
3.21 High overlap parameter distance ($d = 7$)	73
4.1 Ionosphere dataset training accuracy over 100 simulations for varying rounds/local iteration settings.	118
4.2 Ionosphere dataset test accuracy over 100 simulations for varying rounds/local iteration settings.	118
4.3 Ionosphere kernel SVM percentage of local model agreement (tolerance 10^{-2}).	119
4.4 Australian dataset training accuracy over 100 simulations for varying rounds/local iteration settings.	119
4.5 Australian dataset test accuracy over 100 simulations for varying rounds/local iteration settings.	120
4.6 Australian kernel SVM percentage of local model agreement (tolerance 10^{-2}).	120
4.7 Ionosphere - RKHS model distance to centralized	121
4.8 Ionosphere - RKHS cosine similarity to centralized	122
4.9 Australian - RKHS model distance to centralized	122
4.10 Australian - RKHS cosine similarity to centralized	122

4.11 RFF Ionosphere train accuracy with $\alpha = 0.1$	126
4.12 RFF Ionosphere test accuracy with $\alpha = 0.1$	126
4.13 RFF Australian train accuracy with $\alpha = 0.1$	126
4.14 RFF Australian test accuracy with $\alpha = 0.1$	127

LIST OF FIGURES

3.1	Plentiful data, Compact, Separable	32
3.2	Plentiful data, Compact, Overlapping	33
3.3	Plentiful data, Dispersed, Separable	33
3.4	Plentiful data, Dispersed, Overlapping	34
3.5	Few data, Compact, Separable	35
3.6	Few data, Compact, Overlapping	35
3.7	Few data, Dispersed, Separable	36
3.8	Few data, Dispersed, Overlapping	36
3.9	Heavy imbalance, Compact, Separable	37
3.10	Heavy imbalance, Compact, Overlapping	37
3.11	Heavy imbalance, Dispersed, Separable	38
3.12	Heavy imbalance, Dispersed, Overlapping	38
3.13	Plentiful data, Compact, Separable	40
3.14	Plentiful data, Compact, Overlapping	40
3.15	Plentiful data, Dispersed, Separable	40
3.16	Plentiful data, Dispersed, Overlapping	41
3.17	Few data, Compact, Separable	41
3.18	Few data, Compact, Overlapping	41
3.19	Few data, Dispersed, Separable	42
3.20	Few data, Dispersed, Overlapping	42
3.21	Heavy imbalance, Compact, Separable	42

3.22 Heavy imbalance, Compact, Overlapping	43
3.23 Heavy imbalance, Dispersed, Separable	43
3.24 Heavy imbalance, Dispersed, Overlapping	43
3.25 Plentiful data, Compact, Separable	51
3.26 Plentiful data, Compact, Overlapping	51
3.27 Plentiful data, Dispersed, Separable	52
3.28 Plentiful data, Dispersed, Overlapping	52
3.29 Few data, Compact, Separable	52
3.30 Few data, Compact, Overlapping	53
3.31 Few data, Dispersed, Separable	53
3.32 Few data, Dispersed, Overlapping	53
3.33 Heavy Imbalance, Compact, Separable	54
3.34 Heavy Imbalance, Compact, Overlapping	54
3.35 Heavy Imbalance, Dispersed, Separable	54
3.36 Heavy Imbalance, Dispersed, Overlapping	55
5.1 Flowchart of Collaborative Transfer Learning framework.	132
5.2 Performance of guest-node selection strategies over 100 simulations. (a) Mean accuracy across all prediction tasks. (b) Mean accuracy for tasks from distributions different from local training data. (c) Mean accuracy for tasks aligned with the local training data distribution. (d) A zoomed-in view of early-stage accuracy on non-local tasks (first 25 iterations), highlighting the initial adaptation dynamics of different strategies.	134

CHAPTER 1

Introduction

Motivation

Throughout the last decade, Distributed Machine Learning (DML) and Federated Learning (FL) have become increasingly more popular [23], [13]. Often DML and FL can refer to the same thing, where the training data are partitioned across potentially many **nodes**. While in the early days DML was a tool for handling large datasets, now such datasets are usually spread across devices from the beginning. In addition, transferring raw data between nodes is prohibited due to privacy concerns. In some cases, large data may be infeasible to transfer between nodes due to communicational bottlenecks and inaccessibility of nodes for lengthy periods of time.

Nodes can be cellular devices, sensors, computers, . . . and therefore contain data that are often highly heterogeneous by nature. This requires distributed algorithms capable of learning appropriate models despite the potentially large differences in node data or objectives. Since IID assumptions across nodes will often fail, many simple (and thus popular) distributed learning algorithms can suffer from poor accuracy or lack of convergence. Algorithms involving parameter averaging require compatible models across every node and may require frequent communication to remain reasonable.

One of the biggest issues in DML/FL is overcoming the need for frequent communication. Often, nodes are unable to communicate or have slow communication speeds. Even with a fast internet, some data, such as very large model parameters, can take an unreasonable amount of time to communicate for every communication round. Since raw data can also not be shared, we can be greatly limited in our techniques for

training in such environments. This necessitates adaptable methods which transfer compact and robust summaries between nodes.

One powerful classification algorithm in machine learning is the Support Vector Machine (SVM). While learning neural networks in distributed settings is (likely) the most common choice of study, robust learning of distributed SVMs (or DSVMs) has been considered far less. When data are limited, simpler classifiers can be the optimal choice. SVMs seem like a natural choice for distributed learning due to the convexity of hinge loss and the usefulness of the so-called support vectors. However, raw data cannot be exchanged and robust SVMs are hard to guarantee without hefty amounts of communication. This motivates our use of **representatives**, or summaries of important information from nodes, to develop new distributed methods for SVMs in distributed learning.

Problem Statement

Existing methods for learning DSVMs commonly rely on sending support vectors or simply exchanging parameters. Additionally, methods aimed at handling heterogeneity or increasing generalizability of the predictors ([15], [42]) may require heavy computation and/or be heavily reliant on frequent transferring of model parameters. Methods like ADMM also require frequent communication of model parameters, and asynchronous adaptations require strong assumptions for similar convergence guarantees.

When local models differ greatly from the global one, frequent communication is required to keep them together. Methods that attempt to align local models to a current global approximation can be slow, and when model parameter sizes get very large, such model transferring can easily violate communication constraints in FL.

In this dissertation, we propose a method aimed at increasing robustness of the SVM classifier, while keeping computation and communication at a minimum.

We also do not require model parameters to be transferred, however, such methods mentioned above may be naturally fused with ours for an even better result.

Contributions

Our first contribution is in the form of using a unique method for smoothing out the hinge loss. We begin from the Empirical Risk Minimization (ERM) formulation using hinge loss and add robustness in the form of lifting data points to be Gaussian distributions. While this is similar to using kernel smoothing methods ([37]), our method allows different covariance for different points (or groups). While this is the main difference in the loss formula, our perspective is not trying to obtain a good density estimation for the data under Gaussian assumptions but rather trying to optimize a more robust loss for SVM. This allows for a richer form for representatives and a nicer optimization problem.

Secondly, utilizing the new ERM formulation mentioned earlier, we form our so-called lifted SVM. Using the loss expression and gradient expressions, we derive the form of multiple representatives which may be shared between nodes for efficient training under heterogeneous node settings. We provide theoretical analysis on the connection between our primary representative choice and the compactness of the data. The lifted loss and representative sharing are then combined for our algorithm for linear DSVM.

Utilizing our lifted SVM formulation, we derive a meaningful kernel extension to our loss and derive the form for our previous representative ideas but now lying in the Reproducing Kernel Hilbert Space (RKHS). We utilize a public dictionary idea to overcome incompatibility of local models and inability to transfer points in the RKHS. We also revisit linear representatives in conjunction with Random Fourier Features (RFFs) for approximating kernels.

To reinforce the strength of both our linear and kernel SVM algorithms, we perform both synthetic simulations involving heavy heterogeneity between nodes, as well as experiments on splitting up and training on usual baseline datasets.

Lastly, we work in a setting involving linear logistic regression models across nodes which utilize energy distance to perform Collaborative Transfer Learning (CTL) and aid appropriate nodes in their node-specific prediction tasks. This allows for overcoming feature heterogeneity between the nodes. We utilize Knowledge Distillation (KD) to avoid the necessity of transferring model parameters and overcome the necessity of a public dataset by constructing ours with representative data points. This use of representatives differs from the previous: facilitating the distributed algorithm with a public dataset rather than directly being used to handle heterogeneity.

Dissertation Organization

In Chapter 2, we give a thorough background of distributed machine learning and the corresponding techniques to learn in both standard settings, as well as heterogeneous settings. We discuss techniques like KD in DML settings as well as previously established methods in aligning local models with the global one. We also review the idea of representatives in DML and some useful choices. Lastly, we discuss SVMs and their popular history in DML.

In Chapter 3, we define our linear lifted SVM loss function resulting from our modification of the usual empirical distribution in ERM. We derive the form of various representative choices to be constructed by the nodes during training and establish the full distributed learning algorithm. We provide basic properties of this loss function as well as theoretical properties of our Score-Matching Representative (SMR) in approximating gradients. Lastly, we show the results of our algorithm on simulated mixture Gaussian data with highly imbalanced class setups.

In Chapter 4, we extend our lifted loss to the kernel form. We derive the corresponding representatives in the RKHS with the use of a public dictionary. We

offer two different distributed algorithms utilizing representatives and provide the corresponding results on two different datasets under multiple class imbalance settings for each algorithm.

In Chapter 5, we describe our work on Collaborative Transfer Learning (CTL). We use representatives to overcome the lack of a shared public dataset between nodes in order to utilize KD. Feature heterogeneity between nodes is understood through energy distance, which motivates our choice of adaptive weights. These weights dictate the amount of collaboration between nodes for handling their specific prediction task goals.

In Chapter 6, we conclude with our thoughts on the current work and many ideas for future work involving SVMs, representatives, and neural networks. Namely, SVMs have been shown to play an important role in training neural networks along with transformers. Additionally, representatives matching data and gradients (like those in this dissertation) may have important extensions to deep neural networks that have not yet been explored.

CHAPTER 2

Background and Literature Review

Here we will discuss some of the seminal work in the areas of distributed and Federated Learning (FL), SVM, and representative learning. We will also discuss some methods that motivated our work such as the use of Knowledge Distillation (KD) to reduce communication.

We will conclude the literature review with work on DSVMs, motivating our primary work in this dissertation. The research in each of these areas is rather extensive, so we aim to provide references that give a meaningful introduction to the different methods.

1. Distributed Learning and FL

Distributed learning refers to machine learning on datasets that are spread out among multiple nodes. In [23] FL was introduced as a distributed learning setting where we potentially have a large number of nodes with datasets that are naturally split among the devices (as opposed to being split intentionally for computational purposes). Initially, this was proposed with a central server involved that handles communication with privacy preserving algorithms. The most common objective in FL is

$$\min_w \sum_k F_k(w)$$

where $F_i(w)$ are the local node objectives. The first basic algorithm proposed for minimizing this summation (in the original paper) is Federated Averaging (FedAvg)

which involves repeated local training and aggregation of the form:

$$w^{t+1} = \sum_k \frac{n_k}{n} w_k^{t+1}$$

[13] discusses the many extensions of FL to areas like multi-task learning, personalization, ... In this sense, FL and DML may essentially refer to the same general setting and any constraints during training must be clearly laid out.

The most important constraint in FL is privacy. Raw data are not transmitted between nodes or from nodes to a central server. Quantities such as model parameters, model predictions, and pseudo-data are common choices as long as they do not violate any of the later constraints. Another important restriction is on sending copious amounts of data across the network. This is due to things like lack of availability of nodes, network speeds, and device capabilities. While hefty computation is also a concern in distributed learning, it is often preferred over frequent or large communications. Note that the previous concerns may also constrain the abilities to send model parameters in the case of very large models. Modern deep neural networks (like BERT Large) can contain hundreds of millions of parameters, requiring hours to transmit a single time on common network speeds.

Knowledge Distillation (KD) is a common technique when dealing with large models in distributed learning, since only predictions on data need to be shared across the network. [12] first introduced KD for training smaller models from larger ones. [40] introduced this idea to FL. [17] extended this to the heterogeneous case where local neural networks can differ. This adds an additional setting where the usual FedAvg could not work even if heavy communication were allowed.

1.1. Heterogeneity. Heterogeneity is one of the biggest concerns in FL. It can often refer to varying communication abilities of nodes or different local models, etc ... In this dissertation we will mostly use heterogeneity to refer to the feature data itself or class imbalances unless we explicitly say, for example, “device heterogeneity” or “model heterogeneity”. [29] introduced local stochastic gradient descent (localSGD)

and provides analysis as to how local heterogeneity can slow down convergence and diminish accuracy under distributed gradient descent (or FedAvg). Initial ideas used the model parameters to encourage local models to not stray too far from the averaged one ([20] [14], [38]).

Model parameters themselves are not always the most reliable, and certain settings (whether due to large communication or incompatibility) prohibit methods based on sharing and working with model parameters directly. [7] uses KD with assumed device heterogeneity (some devices are available to communicate more than others). [40] trains a smaller student model which is shared among nodes. Often KD papers will utilize public data that each local model can share predictions on or a student model which has fewer parameters that can be shared directly. Synthetic inputs or hidden representation sharing are alternative ideas which do not require a predefined shared dataset or smaller student model.

Robustness to heterogeneity is one of the key challenges in DML. Distributionally Robust Optimization (DRO) is a technique which modifies the objective function to consider “worst case” scenarios when the data lie in compact distributions ([15], [5], [26]). These ideas were extended to FL to improve generalizability and robustness ([6]).

An important way to handle heterogeneity among nodes is to look at the data itself. While raw data cannot be shared, certain summaries, such as the moments, are allowed. We refer to these summaries as representatives. [19] considers mean and score-matching (gradient-matching) representatives for large datasets that can be used to approximate solutions to the full data. Such representative techniques can be cleanly combined with most other model-based techniques to handle heterogeneity even better. More is discussed on this in a section below.

1.2. Personalized Learning. An important area of DML is Personalized Learning. Often in DML, we have nodes which would like to benefit from the large number of data present in the network, but overall would like to learn their own model. [30] is

an early paper which personalizes the FL framework, and [28] considers the multi-task learning setting.

2. Representative Learning

Here we discuss the use of **representatives** in DML. We use the term representatives to refer generally to “compact” information computed or constructed on nodes in DML to facilitate learning. Representatives may be points or more complicated objects. It is important for DML that such points do not coincide with the actual dataset, that is, they are “pseudo” data points. Due to communication constraints (both privacy and communication size), we often require sending such compact summaries between nodes. We first discuss the idea of representatives in approximating the full data solution to distributed optimization problems. Then we consider representatives for more specific purposes in distributed learning. After introducing our form for representatives, we will review some closely related ideas that look to approximate loss functions and gradients.

2.1. Notation. Consider a distributed dataset

$$\mathcal{D} = \cup_{k=1}^K \mathcal{D}_k, \quad \mathcal{D}_k = \{(x_i, y_i)_{i \in I_k}\},$$

consisting of K blocks. We denote by

$$\tilde{\mathcal{D}} = \cup_{k=1}^K \tilde{\mathcal{D}}_k$$

a set of representatives for the entire dataset, constructed on each of the blocks of data $\mathcal{D}_k$. A common form of representative takes the form $(n_k, \tilde{x}_k, \tilde{y}_k)$. This is a tuple containing a representative feature, output, and amount of data being represented constructed on the entire block $\mathcal{D}_k$ containing n_k data. We will also refer to components inside of the tuple more loosely as representatives. While representatives may be constructed from the data alone, they often utilize the loss function and model defined for the given data block to become richer objects.

2.2. Representatives for GLMs. In [19], Li and Yang discuss the use of representatives in approximating the full data solutions to Generalized Linear Models (GLMs). Namely, two important representatives are discussed here, Mean Representative (MR) and Score-matching Representative (SMR).

Given a node k , MR is constructed by

$$\tilde{x}_k = \frac{1}{n} \sum_{i \in I_k} x_i, \quad \tilde{y}_k = \frac{1}{n} \sum_{i \in I_k} y_i.$$

The estimator based on MR is unbiased for linear models, approximating estimator from aggregated dataset. As for non-linear models, such as GLMs, it yields biased estimator due to nonlinearity.

Let

$$s_k(\beta) = \sum_{i \in I_k} r_i \nu(\eta_i) x_i$$

denote the score function contribution from $\mathcal{D}_k$, where $r_i = y_i - G(\eta_i)$, $G(\eta_i) = g^{-1}(\eta_i)$, $\eta_i = x_i^\top \beta = g(\mu_i)$. Denote

$$\tilde{s}_k(\beta) = n_k \tilde{r}_k \nu(\tilde{\eta}_k) \tilde{x}_k$$

the score function contribution from $\tilde{\mathcal{D}}_k$, where $\tilde{r}_k = \tilde{y}_k - G(\tilde{\eta}_k)$ and $\tilde{\eta}_k = \tilde{x}_k^\top \beta$.

SMR is constructed by solving the score-matching equation:

$$s_k(\beta) = \tilde{s}_k(\beta).$$

The score-matching equation proposed in [19] is an underdetermined system. There may be multiple solutions for some $\tilde{\eta}_k$'s. It generates instability under within node heterogeneity.

Zheng et al. [44] further improve performance of SMR by splitting nodes on signs or values of response y_i , linear predictor η_i , or residual r_i . After sub-partitioning, there is a unique solution for $\tilde{\eta}_k$ in most cases, improving stability and accuracy.

Anchored-SMR [18] redesigns a determined score-matching equation, by matching both the covariate and intercept score contributions:

$$\begin{aligned}\sum_{i \in I_k} r_i \nu(\eta_i) x_i &= n_k \tilde{r}_k \nu(\tilde{\eta}_k) \tilde{x}_k, \\ \sum_{i \in I_k} r_i \nu(\eta_i) &= n_k \tilde{r}_k \nu(\tilde{\eta}_k),\end{aligned}$$

where intercept representative is fixed at 1. As a consequence, a unique representative construction in closed form, and representative covariate, $\tilde{x}_k$, stays within local geometric support under sign coherence, improving stability and interpretability.

2.3. Representatives in DML. One important aspect of representatives in DML is their ability to mitigate harmful heterogeneity between nodes. While we often aim to approximate full data solutions, we also aim to keep **client drift** (or the local client’s bias towards optimizing their own dataset) small. For short, we refer to such learning as *Representative Learning* (or *RepL*).

Our definition of representatives is intentionally left vague, and thus the methods for using representatives to learn will depend on the type of information. However, when representatives from nodes directly match the form of data on other nodes, we can use their information by incorporating them as a separate part in the loss function. A simple example of an augmented local loss function would be:

$$\mathcal{L}_k = \sum_{i=1}^{n_k} \ell(x_i; w) + \sum_{h=1}^H \ell(\tilde{x}_h; w)$$

where the first component is the loss on local node data and the second component is the same loss function evaluated on representative data. Our loss functions for future examples of RepL will follow this strategy.

2.4. Coresets. An adjacent idea to our choice of pseudo data representatives are coresets. The goal for machine learning is often to approximate the loss function/optimization problems on the full dataset for any parameter choice. [11] introduces coresets as a way to approximate the full data when computing k-means or k-medians. They prove a close approximation to the full data solution. [8] introduced coresets to a more general machine learning setting while connecting to important approximation theory. [22] used k-means as a coreset, analyzed on more general machine learning problems. [1] consider coresets for large-scale SVMs.

2.5. Data Condensation. We have seen multiple ideas of representatives that try to approximate the full dataset as well as representatives that are constructed to match gradients for a fixed weight parameter. In [43] they consider condensing a large dataset into a smaller, synthetic one which matches the objective gradient for all parameter choices.

While both coreset methods and data condensation methods have very similar goals to general DML ones, the theoretical properties can degrade quickly due to decentralized and heavily heterogeneous nodes. Coreset methods which result in a subset of the original dataset, may not be transmitted across the network in DML. Instead, coresets are used to reduce node computation and reduce noise. Data condensation methods are usually matching a global loss or gradient curve, but would require hefty computation and large amounts of communication between nodes if such synthetic datasets were transmitted.

3. Support Vector Machine

Support Vector Classifiers (SVC) were first introduced in [4] as a binary classification tool built upon Vapnik’s statistical learning theory ([36]) that aimed to provide good generalization to unseen data. This begins as a linear classifier and is later extended via the kernel trick.

3.1. Hard-Margin Formulation. In order for the linear classifier to generalize well, an optimization problem was developed to maximize the margin (or minimum distance to each point). Such a classifier is then as far away from each of the classes while still classifying every point correctly (in the case of linear separability). Maximizing the margin can be shown as equivalent to minimizing $\|w\|$. Adding the condition (with relevant scaling) that all points are classified correctly, we get the optimization problem

$$\begin{aligned} \min_{w,b} \frac{1}{2} \|w\|^2 \\ y_i(w^T x_i + b) \geq 1, \quad \forall i \end{aligned}$$

3.2. Soft-margin Formulation. Since real data are rarely linearly separable, so-called slack variables are introduced: $\xi_i > 0$ for each data point. Thus the optimization problem becomes minimizing $\|w\|$ as well as these slack variables, so that data are classified as strongly as possible. This looks like

$$\begin{aligned} \min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \\ y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

This optimization problem is equivalent to the regularized hinge loss:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i(w^T x_i + b))$$

This will be the most relevant formulation for this dissertation.

3.3. Dual Form of SVM. In order to use the method of Lagrange multipliers, we write the first constraint as

$$1 - \xi_i - y_i(w^T x_i + b) \leq 0$$

and the second constraint as

$$-\xi_i \leq 0$$

Then, using multipliers $\alpha_i \geq 0, \mu_i \geq 0$ for the corresponding constraints, we write the Lagrangian as

$$L(w, b, \xi, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_i \xi_i + \sum_i \alpha_i (1 - \xi_i - y_i(w^T x_i + b)) - \sum_i \mu_i \xi_i$$

In order to minimize, we consider each of the derivatives for the optimization variables w, b, ξ and set them equal to zero:

$$\begin{aligned} \nabla_w L &= w - \sum_i \alpha_i y_i x_i = 0 \\ \implies w &= \sum_i \alpha_i y_i x_i \end{aligned}$$

$$\begin{aligned} \frac{\partial L}{\partial b} &= - \sum_i \alpha_i y_i = 0 \\ \implies \sum_i \alpha_i y_i &= 0 \end{aligned}$$

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0$$

For this last equation we use $\mu_i \geq 0$ to write

$$0 \leq \alpha_i \leq C$$

Finally, using our equation for w we write the dual optimization problem:

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \\ & 0 \leq \alpha_i \leq C, \\ & \sum_i \alpha_i y_i = 0 \end{aligned}$$

3.4. Kernels in SVM. From the dual form above, we can see that the optimization problem depends only on the data through the standard inner product $\langle x_i, x_j \rangle$. Kernel methods replace this inner product with a kernel function $K(x_i, x_j)$ (which we discuss later) so that the optimization problem becomes:

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j) \\ & 0 \leq \alpha_i \leq C, \\ & \sum_i \alpha_i y_i = 0 \end{aligned}$$

It is known that a pair (w^*, α^*) is sufficient for solving the optimization problem if it is a saddle point of the Lagrangian. However, it is also useful to check the necessary conditions. In order for a solution to this nonlinear problem to be optimal, we must satisfy the Karush-Kuhn-Tucker (KKT) conditions. The constraints must be satisfied at a given solution, first order optimality conditions (given above) must be satisfied, and we must also have:

$$\begin{aligned} \mu_i \xi_i &= 0 \\ \alpha_i (1 - \xi_i - y_i (w^T x_i + b)) &= 0 \end{aligned}$$

The second condition here is particularly interesting. Recall that

$$w = \sum_i \alpha_i y_i x_i$$

If $\alpha_i = 0$, then the given data point x_i does not contribute to the solution. If $\alpha_i > 0$, then it does and x_i is called a **support vector**. If a linear SVM (without kernels) is used, then this means

$$f(x) = w^T x + b = \sum_{i \in SV} \alpha_i y_i x_i^T x + b$$

or with kernels,

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b$$

Next, we can discuss some of the theory behind kernels and why they are so useful for SVMs. First, consider the definition

$$K(x_i, x_j) := \langle \psi(x_i), \psi(x_j) \rangle$$

where $\psi(x) : \mathbb{R}^d \rightarrow \mathcal{H}$ into a potentially high dimensional Hilbert Space. Some common choices of kernel functions are:

- Linear kernel:

$$K(x, y) = x \cdot y$$

- RBF kernel:

$$K(x, y) = \exp(-\gamma \|x - y\|^2)$$

- Polynomial kernel:

$$K(x, y) = (x \cdot y + c)^d$$

Each choice of positive, semi-definite kernel function defines a unique Reproducing Kernel Hilbert Space (RKHS) $\mathcal{H}$ with the property

$$f(x) = \langle f, K(x, \cdot) \rangle_{\mathcal{H}}$$

for all $f \in \mathcal{H}$. In this case, our soft-margin optimization problem in section 3.2 could be written as

$$\min_{f \in \mathcal{H}, \lfloor} \frac{1}{2} \|f\|_{\mathcal{H}}^2 + C \sum_i \ell(y_i, f(x_i) + b)$$

where ℓ is hinge loss. While our optimization problem requires searching through an infinite dimensional space $\mathcal{H}$, the **Representer Theorem** tells us that a minimizer for the empirical risk in the Hilbert space has the finite expansion.

$$f(x) = \sum_{i=1}^n c_i K(x_i, x)$$

which matches the form for our optimizer. We note that this result is more general for kernel functions with RKHS regularization and, for SVM, does not require going through the dual form.

3.5. Optimizing SVM. We have listed the optimization problems in the primal and dual form for SVM. Both are suitable forms with their own advantages and disadvantages. We will mention some of the more popular choices here. While SVMs are powerful classifiers with strong nonlinear capabilities, they unfortunately are computationally intense to train. This is especially true when the data are numerous.

We first discuss some methods for solving primal SVM.

- Since the hinge loss is convex but not smooth, the natural choice for optimization is subgradient descent. A subgradient for hinge loss is

$$\begin{aligned} 0, & \quad y_i(w^T x_i + b) > 1, \\ -y_i x_i, & \quad y_i(w^T x_i + b) < 1, \\ \alpha(-y_i x_i), & \quad y_i(w^T x_i + b) = 1, \end{aligned}$$

where $\alpha \in [0, 1]$. Thus we only utilize the data where $y_i(w^T x_i + b) < 1$ holds. Then subgradient descent iterates,

$$w_{t+1} = w_t - \eta_t \left(w - C \sum_i y_i x_i \right)$$

- Coordinate descent methods are also popular and quite simple. They simply fix all of the coordinates i , except one, say j , and optimize with respect to w_j . This works due to the convexity of hinge loss.
- While the kernel trick can be used with the primal form of SVM, it is not a common choice due to being computationally intense.

We can also consider some methods for solving the dual formulation.

- Since the SVM optimization problem is quadratic, convex, and subject to linear inequalities, Interior Point methods were an early choice for solving them. These can be used for either the primal or dual form. The basic idea is to use “barriers” to keep the objective function in the feasible (satisfying constraints) region. This could look like

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \frac{1}{t} \sum_i [\log(y_i(w^\top x_i + b) + \xi_i - 1) + \log(\xi_i)]$$

where the logarithms enforce the optimization constraints, and $\frac{1}{t}$ is decreased towards zero. Newton’s method is repeatedly applied to solve this. This

more general Quadratic Programming (QP) solver is less popular due to the expensive cost when the number of data are large.

- Sequential Minimal Optimization (SMO, [24]) is a more common choice for optimization. It optimizes α_i, α_j from the dual formulation in a pairwise manner via a closed-form solution. While historically these dual variables were chosen randomly, now smarter algorithms will select those violating KKT conditions the most.

Much of the work in optimizing SVMs revolves around reducing computational burden or memory requirements. Storing Kernel matrices is $\mathcal{O}(n^2)$ and computational complexity for the basic solvers can be $\mathcal{O}(n^3)$ or $\mathcal{O}(n^2)$. Both of these are reasonable concerns in distributed learning, but more importantly, methods to mitigate such burdens may be useful considerations for distributed learning algorithms. Below, we list some useful papers for this area.

[25] consider using specially constructed Random Fourier Features (RFF) to train on a lower dimensional feature space and train faster. This idea also reduces the memory requirements of storing a kernel matrix. [39] uses the Nyström approximation to reduce memory requirements by approximating the full data kernel matrix. [16] considers using a small subset of the data to reduce the size of the kernel matrix as well as complexity of the decision boundary. [27] compresses the kernel matrix utilizing the low-rank and block-diagonal structure. They then use scaling to realize low range structure within the blocks.

While computation is not usually the primary concern for distributed learning (even though we wouldn't desire slow, asynchronous nodes), the repercussions of reducing computation are useful in reducing memory requirements on nodes. Additionally, such ideas can, and often do, lead to efficient distributed algorithms.

4. Empirical Risk Minimization

Empirical Risk Minimization (ERM) ([35]) underlies many popular machine learning optimization problems. This is a framework for finding a strong predictive model using supervised learning given that we only have training data and do not know the underlying data distribution.

4.1. Risk. The **risk** (or prediction error) for a given classifier f is defined as the expected value of the loss function where the data come from some unknown distribution

$$R(f) := \mathbb{E}_{(X,Y) \sim P}[\ell(f(X), Y)]$$

Since the underlying distribution is usually unknown, such an expectation cannot be computed.

4.2. Empirical Risk. Instead of the true risk of a prediction f we instead use the empirical risk, defined as

$$(1) \quad R_n(f) := \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i)$$

for point observations, motivated by the law of large numbers.

4.3. Empirical Risk Minimization. In practice, we aim for our classifier to not just be able to predict well on our training data but generalize well to unseen data. Therefore, regularization is commonly used to restrict the complexity of the classifier. Generally, this is written as

$$\min_f R_n(f) + \lambda \Omega(f)$$

where the regularization Ω is often chosen to be the $L1$ or $L2$ norm of the parameter. Different regularizers have different effects on the learned parameter. For example, $L1$

regularization tends to give more sparse solution while $L2$ regularization will generally shrink parameters but not to zero.

Many common optimization problems fit into the framework of ERM including: logistic regression, least squares regression, and SVM. The corresponding loss functions are logistic loss, squared loss, and hinge loss.

4.4. Regularized Distributed Learning Objective. We can extend empirical risk minimization to the distributed learning case. Here, the function to minimize is the sum of local loss functions. From ERM we have,

$$f = \sum_{k=1}^K f_k$$

so that

$$R_n(f) = \sum_{k=1}^K R_{n_k}(f_k),$$

$$\lambda\Omega(f) = \lambda\Omega\left(\sum_{k=1}^K R_{n_k}(f_k)\right)$$

Often, we write the objective more directly in terms of the parameter being learned,

$$\min_{\{w_k\}} \sum_{k=1}^K f_k(w_k; \mathcal{D}_k) + \lambda \cdot \Omega(\{w_k\})$$

Here we use $\mathcal{D}_k$ to denote the dataset belonging to node k . Previously we mentioned that some DML environments aim to learn a single global model, whereas others suppose each node retains their own. Simply speaking, global learning means the distributed objective uses a single model w and personalized learning allows for w_k to differ.

- Global Learning

$$w_k = w,$$

- Personalized Learning

$$w_k \neq w_j$$

- Coordinated Learning

$$\Omega(\{w_k\}) = \sum_{k,j} \alpha_{kj} \|w_k - w_j\|$$

Here we can see that each node is allowed its own model. As an example, if the objective contains K different models (one for each node) then the objective corresponds to minimizing each local loss function separately (possibly depending on the regularization choice). If all w_k are the same, then we are learning a global model which fits every node to match the global data objective. In the last equation, we see a regularization choice that keeps local models close to each other to mitigate client drift. Note that this is just one particular choice.

As an example, the previously cited FedProx algorithm used with hinge loss solves

$$\begin{aligned} \min_{w_k, b_k} \quad & \frac{1}{2} \|w_k\|^2 + C \sum_{i \in \mathcal{D}_k} \max(0, 1 - y_i(w_k^T x_i + b_k)) \\ & + \frac{\mu}{2} (\|w_k - w^t\|) \end{aligned}$$

constraining the drift from the local model w_k to the aggregate global model w , enforcing local model agreement. We can now look at some other methods aimed at solving SVMs.

5. Support Vector Machine

SVMs have been studied and optimized for many decades. They are known to produce quadratic programming problems which are computationally intense when the dataset is large. [24] provides a popular algorithm called Sequential Minimal Optimization for solving SVMs.

Solving SVMs from distributed data nodes poses unique challenges besides computational cost. While the overall datasets are still large, each node usually only holds a small piece of the dataset. Due to the constraints of DML from Chapter 1, raw data may not be transferred, making the usual methods for solving SVM unreliable when applied to DML.

While we focus more on reviewing methods aimed at solving DSVM, some natural choices like subgradient descent may be performed instead of FedAvg as a natural choice since hinge loss is nonsmooth ([21]). Many of the usual distributed algorithms can also be applied to SVM if smoothing techniques are utilized first.

5.1. Support Vector Methods. While raw data (including support vectors) are usually restricted from transferring over the network, it is known that support vectors provide useful information for reconstructing global data solutions from distributed sources. Since support vectors completely define local solutions, they are natural summary information to communicate for efficient training. [10] consider a hierarchical method for training on just support vectors. However, even if raw data exchange were allowed, support vectors can often be a large portion of the dataset which would violate communication size constraints as well.

Different methods involving compressing the amount of data used in training via coresets ([34], [33]) were developed to reduce the computation for SVMs and can be more reasonably extended to DSVMs.

5.2. Consensus-based SVM Using ADMM. [9] considers a DSVM algorithm where each local node solves their own SVM optimization problem subject to the constraint that all nodes' parameters must agree. Only model parameters are needed to be shared. While this paper does not analyze kernel methods, low-rank approximations to the kernel matrix (such as the Nystrom method) can be used so that such methods still work. More precisely, it solves

$$(2) \quad \min_{\{w_k, b_k\}} \sum_{k=1}^K \left[\frac{1}{2} \|w_k\|^2 + C \sum_{i \in \mathcal{D}_k} \max(0, 1 - y_i(w_k^T x_i + b_k)) \right]$$

subject to

$$w_1 = \cdots = w_K, \quad b_1 = \cdots = b_K.$$

by alternating between solving local optimization problems and consensus updates.

Extensions to multi-task transfer learning were done in [41] and utilized more generally (and with smoothness) in [3].

CHAPTER 3

Representative Learning for DSVM

In this chapter, we will discuss our method for learning linear SVM from distributed nodes. Our focus is on handling nodes with highly heterogeneous data, largely with respect to data spread and class imbalances. For this, we utilize representatives to control local drift. We will show two main choices for representatives and indicate other possibilities. First, we show our method for smoothing out the hinge loss in the ERM formulation.

The main contribution of this chapter is to combine representative learning with a lifted SVM objective. The lifted objective replaces each covariate by a local Gaussian cloud, producing a closed-form smooth surrogate of the hinge loss. This enables gradient-based local optimization and allows representatives to carry both location and covariance information. The resulting distributed algorithm exchanges class-wise clouded representatives rather than raw data, gradients, or model parameters.

1. Methodology

1.1. ERM Reformulation and Lifted Hinge Loss. In Chapter 2, we introduced risk minimization and gave the hinge loss formulation for soft-margin SVM. Consider again the risk formula

$$R(f) = \mathbb{E}_{(X,Y) \sim P}[\ell(f(X), Y)]$$

If ℓ is hinge loss, then we have

$$R(f) = \mathbb{E}_{(X,Y) \sim P}[\max(0, 1 - Y(w^\top X + b))].$$

The regularized ERM then has the form

$$\frac{\lambda}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^\top x_i + b))$$

The usual ERM will have us minimize this nonsmooth loss. We propose a method for smoothing out this loss function with the intentions for cleaner and more robust optimization via gradient descent as well as adding robustness to our distributed algorithm (to be discussed later). To do this, we “lift” each of our feature data points x_i to become Gaussians centered at the original data point with covariance Σ_i . To be specific we introduce for each data point X_i as the following random variable,

$$X_i \sim \mathcal{N}(x_i, \Sigma_i)$$

to replace x_i in the ERM expression. Define $M_i = y_i(w^\top X_i + b)$, where $X_i \sim \mathcal{N}(x_i, \Sigma_i)$. Then $M_i \sim \mathcal{N}(\nu_i, s_i^2)$ where $\nu_i = y_i(w^\top x_i + b)$ and $s_i^2 = w^\top \Sigma_i w$. Consider

$$\mathbb{E}[\max(0, 1 - M)] = \int_{-\infty}^1 (1 - m) f_M(m) dm$$

Let $M = \nu + sZ$ where $Z \sim \mathcal{N}(0, 1)$. Consider when $1 - M \geq 0$. Then $M \leq 1$ means

$$Z \leq a$$

where $a = \frac{1-\nu}{s}$. So

$$\mathbb{E}[\max(0, 1 - M)] = \int_{-\infty}^a (1 - \nu - sz) \phi(z) dz$$

This simplifies to give us

$$\ell(w, b) = (1 - \nu) \Phi(a) + s \phi(a)$$

where $s = \sqrt{w^\top \Sigma w}$. The regularized empirical loss formula is then:

$$\frac{\lambda}{2} \|w\|^2 + C \sum_{i=1}^n (1 - \nu_i) \Phi(a_i) + s_i \phi(a_i)$$

While each data point could be equipped with its own unique covariance Σ_i , we give a more reasonable form that can work for entire groups of points.

Let Σ be defined as a diagonal matrix: $\Sigma = \text{diag}(\tilde{\sigma}_1^2, \dots, \tilde{\sigma}_p^2)$ where $\tilde{\sigma}_j := \hat{\sigma}_j \left(\frac{3n}{4}\right)^{-1/5}$. Here $\hat{\sigma}_j$ is the sample variance for feature/dimension j . The factor of $\left(\frac{3n}{4}\right)^{-1/5}$ arises from Silverman’s/Scott’s rule of thumb for density estimation with Gaussians.

Then we let $X_i \sim \mathcal{N}(x_i, \Sigma)$ where Σ is defined as above for the group of points for which x_i belongs. Our loss then becomes

$$\frac{\lambda}{2} \|w\|^2 + C \sum_{i=1}^n (1 - \nu_i) \Phi(a_i) + s \phi(a_i)$$

where $s = \sqrt{w^\top \Sigma w}$. While this loss is the same form as other smoothed hinge loss formulations, those usually have $s = h$ as a fixed smoothing constant from density estimation. We refer to loss above as **lifted hinge loss**.

The purpose of the lifted objective is not to change the target classification problem, but to introduce a differentiable surrogate that approaches the ordinary hinge loss as the covariance scale degenerates to zero. Thus, the empirical comparisons below evaluate whether the Gaussian-lifted hinge loss objective preserves essentially the same predictive behavior as the ordinary nonsmooth one, while improving optimization behavior.

1.2. Loss Properties. Before we develop representatives and describe our distributed algorithm utilizing the lifted hinge loss, we first show some basic properties along with some minor necessary assumptions. In particular, we show that this loss is L -smooth, strongly convex (when regularized), and that we obtain the usual hinge loss when the covariance from the “lifting” process goes to zero in the degenerate case.

Throughout each of the following propositions, let

$$\ell(w, b) = (1 - \nu) \Phi(a) + s \phi(a)$$

where $\nu = y(w^\top x + b)$, $a = \frac{1-\nu}{s}$, $s = \sqrt{w^\top \Sigma w}$. Also, assume $\Sigma \succeq 0$ and $s \geq \epsilon > 0$.

PROPOSITION 3.1 (Lipschitz Continuous Gradient). $\ell(w, b)$ is L -smooth.

PROOF.

$$\begin{aligned}\nabla_w \ell(w, b) &= -y\Phi(a)x + \phi(a) \frac{\Sigma w}{s} \\ \frac{\partial \ell}{\partial b} &= -y\Phi(a)\end{aligned}$$

The Hessian entries are:

$$\begin{aligned}\nabla_w^2 \ell(w, b) &= \frac{\phi(a)}{s} \left(yx + a \frac{\Sigma w}{s} \right) \left(yx + a \frac{\Sigma w}{s} \right)^\top + \phi(a) \left(\frac{\Sigma}{s} - \frac{(\Sigma w)(\Sigma w)^\top}{s^3} \right) \\ \nabla_w \frac{\partial \ell}{\partial b} &= \frac{\phi(a)}{s} \left(xy + a \frac{\Sigma w}{s} \right) \\ \frac{\partial^2 \ell}{\partial b^2} &= \frac{\phi(a)}{s}\end{aligned}$$

Since $\phi(a) \leq \frac{1}{\sqrt{2\pi}}$, $\Sigma \succeq 0$, and s is bounded away from zero, every entry of the Hessian is also bounded. $\square$

PROPOSITION 3.2 ((Strong) Convexity in w). *The lifted hinge loss $\ell(w, b)$ is convex in (w, b) . Further, if we consider the regularized objective*

$$\mathcal{L}(w, b) = \frac{1}{2} \|w\|^2 + \ell(w, b)$$

Then $\mathcal{L}(w, b)$ is strongly convex in w .

PROOF. Recall that lifted SVM is defined as

$$\ell(w, b) := \mathbb{E}_{\xi \sim \mathcal{N}(0, \Sigma)} [\max(0, (1 - y(w^\top (x + \xi) + b)))]$$

Observe that $1 - y(w^\top (x + \xi) + b)$ is convex in (w, b) for every x_i (as an affine function). Then $\max(0, 1 - y(w^\top (x + \xi) + b))$ is the maximum of two convex functions. Thus, our loss is an expectation over convex functions and is convex.

Now the regularization term $\frac{1}{2} \|w\|^2$ is 1-strongly convex with respect to the Euclidean norm.

Therefore $\mathcal{L}(w, b)$ is the sum of a convex function and strongly convex function and is therefore strongly convex. With this particular choice of regularization, $\mathcal{L}(w, b)$ is 1-strong convex in w . $\square$

PROPOSITION 3.3 (Degeneration of Lifted Loss). *Consider the following,*

$$\ell_{\Sigma}(w, b) = \mathbb{E}_{\xi \sim \mathcal{N}(0, \Sigma)} [\max(0, 1 - y(w^{\top}(x + \xi) + b))]$$

Assume w, b and x are fixed. Then

$$\lim_{\Sigma \rightarrow 0} \ell_{\Sigma}(w, b) = \max(0, 1 - y(w^{\top}x + b))$$

PROOF. Now, as $\Sigma \rightarrow 0$, we have $\xi \rightarrow 0$ in probability. Thus

$$x + \xi \rightarrow x$$

in probability as well.

Define

$$h(x) = \max(0, 1 - y(w^{\top}x + b))$$

to be the hinge loss.

Then

$$h(x + \xi) \rightarrow h(x)$$

since h is continuous. Additionally, the hinge loss is Lipschitz continuous in z , that is,

$$|h(z_1) - h(z_2)| \leq \|w\| \|z_1 - z_2\|$$

Thus

$$|h(x + \xi) - h(x)| \leq \|w\| \|\xi\|$$

and therefore

$$\mathbb{E} [|h(x + \xi) - h(x)|] \leq \|w\| \mathbb{E} [\|\xi\|]$$

Since $\xi \sim \mathcal{N}(0, \Sigma)$, we have

$$\mathbb{E} [\|\xi\|^2] = \text{tr}(\Sigma)$$

By Jensen's inequality,

$$\mathbb{E} [\|\xi\|] \leq \sqrt{\text{tr}(\Sigma)}$$

Thus,

$$\mathbb{E} [|h(x + \xi) - h(x)|] \leq \|w\| \sqrt{\text{tr}(\Sigma)}$$

Since $\text{tr}(\Sigma) \rightarrow 0$ as $\Sigma \rightarrow 0$, we have

$$\mathbb{E} [|h(x + \xi) - h(x)|] \rightarrow 0$$

as $\Sigma \rightarrow 0$. Finally we have,

$$\ell_{\Sigma}(w, b) = \mathbb{E} [h(x + \xi)] \rightarrow h(x) = \max[(0, 1 - y(w^{\top} x + b))]$$

□

PROPOSITION 3.4 (Convergence of Minimizers). *Let*

$$\mathcal{L}_{\Sigma}(w, b) = \frac{\lambda}{2} \|w\|^2 + C \sum_{i=1}^n \ell_{\Sigma}^{(i)}(w, b),$$

where

$$\ell_{\Sigma}^{(i)}(w, b) = \mathbb{E}_{\xi_i \sim \mathcal{N}(0, \Sigma)} \left[\max(0, 1 - y_i(w^{\top}(x_i + \xi_i) + b)) \right].$$

Let

$$\mathcal{L}_0(w, b) = \frac{\lambda}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^{\top} x_i + b))$$

denote the regularized SVM objective with the ordinary hinge loss.

If (w_{Σ}, b_{Σ}) minimizes $\mathcal{L}_{\Sigma}$, then as $\Sigma \rightarrow 0$,

$$(w_{\Sigma}, b_{\Sigma}) \rightarrow (w_0, b_0),$$

where (w_0, b_0) minimizes $\mathcal{L}_0$.

Further,

$$\mathcal{L}_{\Sigma}(w_{\Sigma}, b_{\Sigma}) \rightarrow \mathcal{L}_0(w_0, b_0).$$

PROOF. From Proposition 3.3 we have

$$\ell_{\Sigma}^{(i)}(w, b) \rightarrow \max(0, 1 - y_i(w^{\top} x_i + b))$$

for every fixed w, b .

Moreover,

$$\left| \ell_{\Sigma}^{(i)}(w, b) - \max(0, 1 - y_i(w^{\top} x_i + b)) \right| \leq \|w\| \sqrt{\text{tr}(\Sigma)}$$

by the Lipschitz argument used in the proof of Proposition 3.3.

Therefore, $\mathcal{L}_{\Sigma}(w, b) \rightarrow \mathcal{L}_0(w, b)$ uniformly on compact subsets of the parameter space.

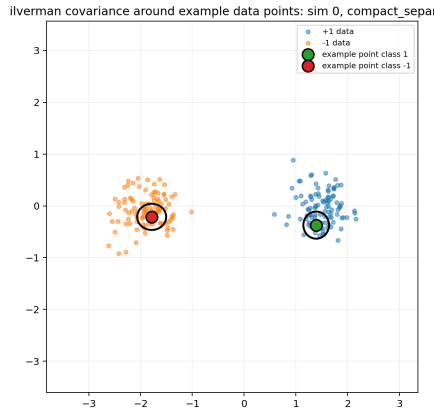
Uniform convergence implies convergence of the minimum values. Since $\mathcal{L}_0$ has a unique minimizer, standard arg min consistency results imply $(w_{\Sigma}, b_{\Sigma}) \rightarrow (w_0, b_0)$.

□

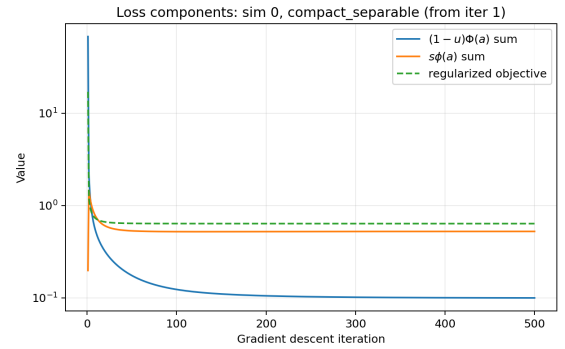
L-smoothness and strong convexity give us confidence in the robustness and efficiency to apply standard gradient descent directly to the lifted objective, whereas the ordinary hinge loss requires nonsmooth optimization methods such as subgradient descent or specialized SVM solvers. The lifted hinge loss preserves the solution of the ordinary SVM problem in the limit $\Sigma \rightarrow 0$, while providing a differentiable objective for optimization when $\Sigma \succ 0$. Our loss being continuous differentiable when $s > 0$ is particularly useful in our construction of representatives in the next section.

1.3. Lifted Loss Simulations. Here we perform some simple simulations on single datasets in two dimensions in order to analyze the loss function during optimization. The purpose of this is to first visualize lifted points with their corresponding Gaussians. This will be important for later when we discuss representative covariance. Secondly, we aim to analyze each of the two components of the lifted loss function in order to motivate possible choices of representatives.

We will just show examples of a few important data configurations. This includes four cases: spread out data that are linearly separable, compact data that are linearly separable, spread out data that overlap, and compact data that overlap.

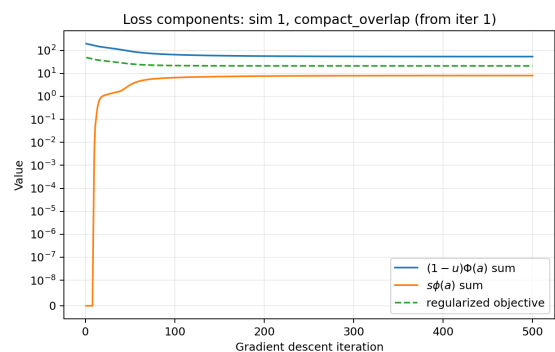
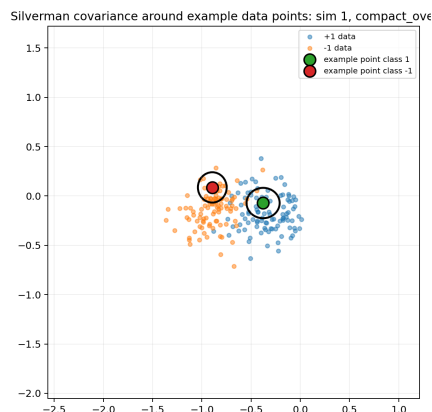


(A) Example point covariance - balance, compact, many data points, separable



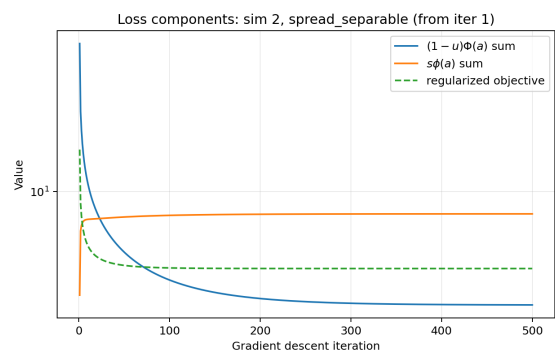
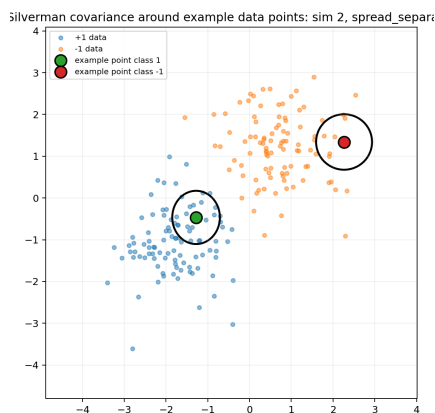
(B) Loss components - balance, compact, many data points, separable

FIGURE 3.1. Plentiful data, Compact, Separable



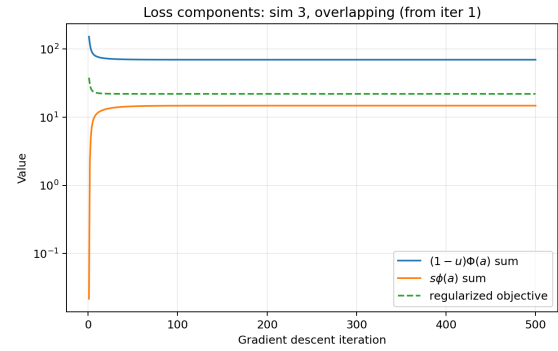
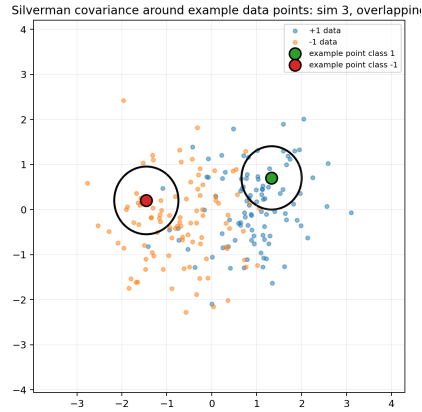
(A) Example point covariance - balance, compact, many data points, not separable (B) Loss components - balance, compact, many data points, not separable

FIGURE 3.2. Plentiful data, Compact, Overlapping



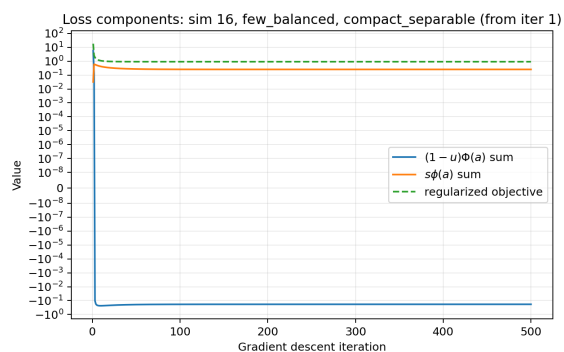
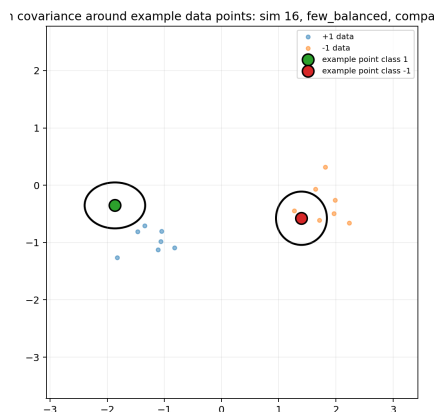
(A) Example point covariance - balance, dispersed, many data points, separable (B) Loss components - balance, dispersed, many data points, separable

FIGURE 3.3. Plentiful data, Dispersed, Separable



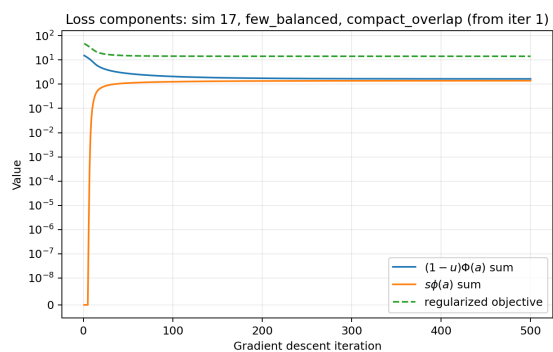
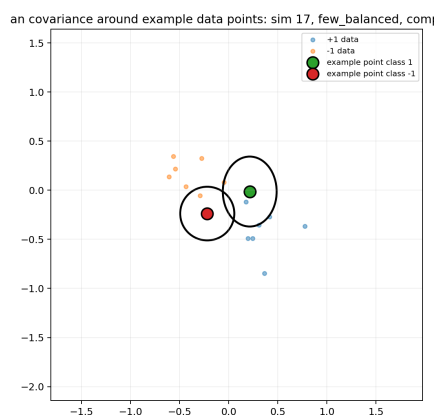
(A) Example point covariance - balance, dispersed, many data points, not separable (B) Loss components - balance, dispersed, many data points, not separable

FIGURE 3.4. Plentiful data, Dispersed, Overlapping



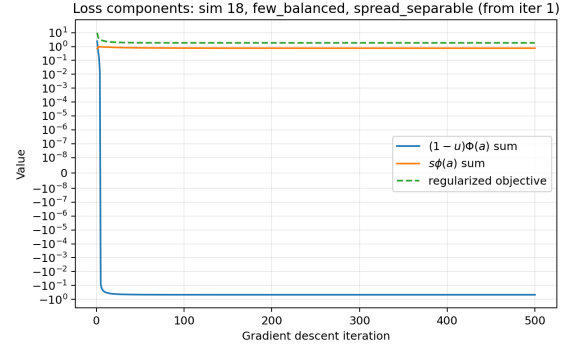
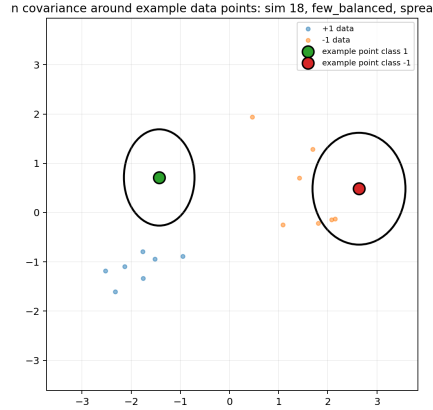
(A) Example point covariance - balance, compact, few data points, separable (B) Loss components - balance, balanced, few data points, separable

FIGURE 3.5. Few data, Compact, Separable



(A) Example point covariance - balance, compact, few data points, overlapping (B) Loss components - balance, compact, few data points, overlapping

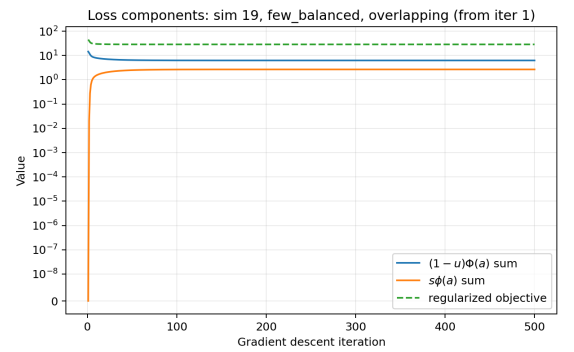
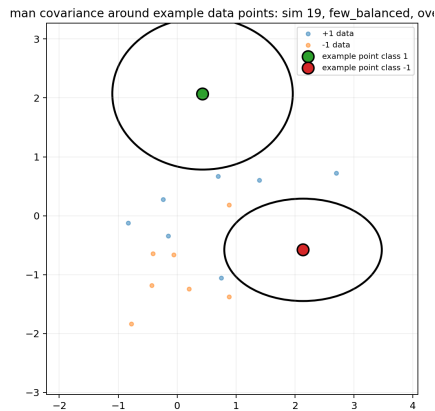
FIGURE 3.6. Few data, Compact, Overlapping



(A) Example point covariance - balance, dispersed, few data points, separable

(B) Loss components - balance, dispersed, few data points, separable

FIGURE 3.7. Few data, Dispersed, Separable

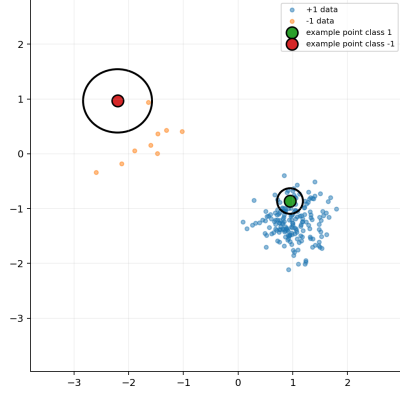


(A) Example point covariance - balance, dispersed, few data points, overlapping

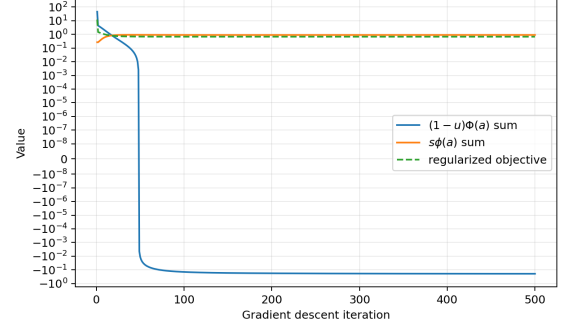
(B) Loss components - balance, dispersed, few data points, overlapping

FIGURE 3.8. Few data, Dispersed, Overlapping

covariance around example data points: sim 56, heavy_imbalance, comp



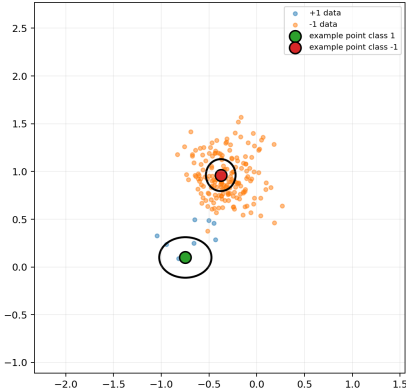
Loss components: sim 56, heavy_imbalance, compact_separable (from iter 1)



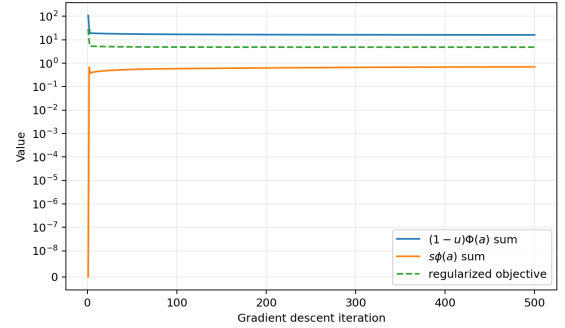
(A) Example point covariance - imbalance, (B) Loss components - imbalance, compact, compact, separable

FIGURE 3.9. Heavy imbalance, Compact, Separable

covariance around example data points: sim 81, heavy_imbalance, cor

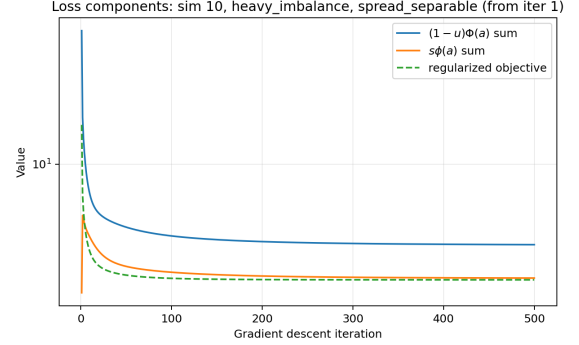
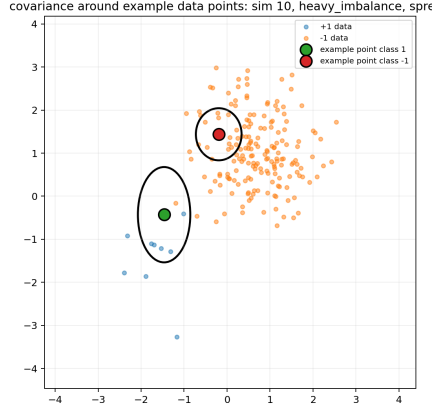


Loss components: sim 81, heavy_imbalance, compact_overlap (from iter 1)



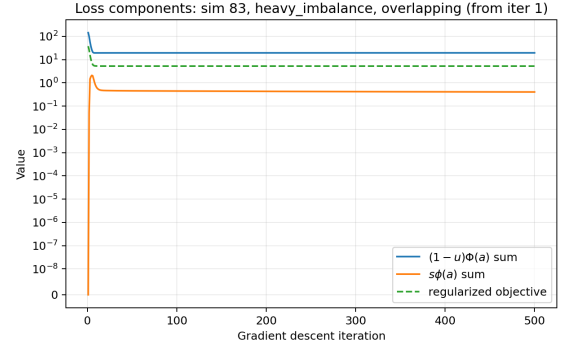
(A) Example point covariance - imbalance, (B) Loss components - imbalance, compact, compact, overlapping

FIGURE 3.10. Heavy imbalance, Compact, Overlapping



(A) Example point covariance - imbalance, (B) Loss components - imbalance, dispersed, dispersed, separable

FIGURE 3.11. Heavy imbalance, Dispersed, Separable



(A) Example point covariance - imbalance, (B) Loss components - imbalance, dispersed, dispersed, overlapping

FIGURE 3.12. Heavy imbalance, Dispersed, Overlapping

Across the configurations in Figures 3.1 – 3.12, the first component $(1 - \nu)\Phi(a)$ generally provides the largest contribution to the lifted loss during optimization, while the covariance-dependent term $s\phi(a)$ provides a smaller correction. This behavior is pretty consistent across changes in class separation, data dispersion, sample size, and class imbalance. However, when the overall loss itself is quite small, we may see this second term dominate.

These simulations motivate representative constructions based on this important first component of the loss, while also illustrating that the covariance term remains

capable of encoding within class spread. The heavily imbalanced examples additionally demonstrate a difficult regime in which the locally optimal classifier may favor one class. Such examples are included as extreme cases of nodes which occur in distributed learning. When the overall distributed scenario is relatively balanced class-wise, the local node failure will be corrected by representatives from other nodes and vice versa.

1.3.1. *Hinge loss vs Lifted loss.*

It is also important to understand the effect of replacing the classical hinge loss by the lifted objective itself. Since the proposed representative methods are built upon the lifted formulation, we first isolate the influence of the loss function from the influence of representative sharing. These experiments therefore compare optimization under the lifted objective and the classical hinge loss while keeping the underlying data fixed. To do this, we plot curves comparing respective objective gaps. That is, we plot

$$\frac{\min_{\theta_s} F(\theta_s) - F^*}{F(\theta_0) - F^*}$$

for the two objectives. While strong methods exist for minimizing hinge loss, we choose computationally comparable optimization methods: gradient descent and subgradient descent. The step-size and decreasing step-size function for subgradient descent was tuned for the optimal choice across all simulations. Thus, the learning rate for subgradient descent was chosen to be

$$\eta_t = \frac{\eta_0}{1+t}$$

with $\eta_0 \in \{0.001, 0.003, 0.01, 0.03, 0.1, 0.3\}$ chosen to be the optimal choice for each simulation. For the lifted loss, we choose use a fixed constant throughout, demonstrating the robustness of a smooth loss function. While the following plots also include a constant learning rate choice for hinge loss, such plots of the “best objective gap so far” will not capture the oscillations near optimal parameters.

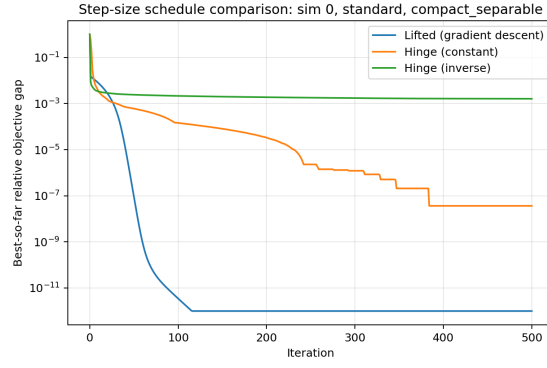


FIGURE 3.13. Plentiful data, Compact, Separable

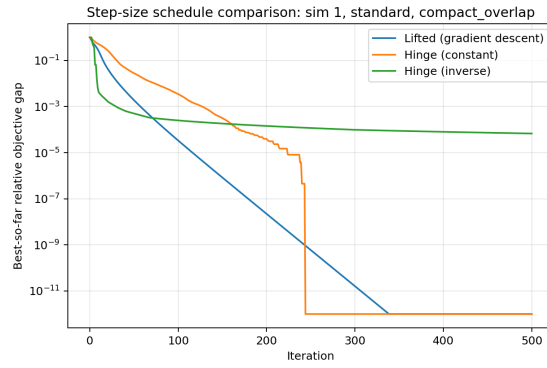


FIGURE 3.14. Plentiful data, Compact, Overlapping

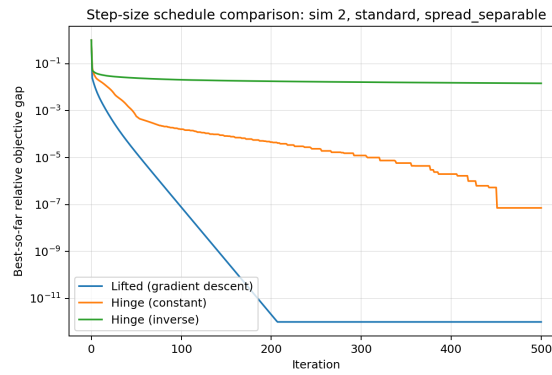


FIGURE 3.15. Plentiful data, Dispersed, Separable

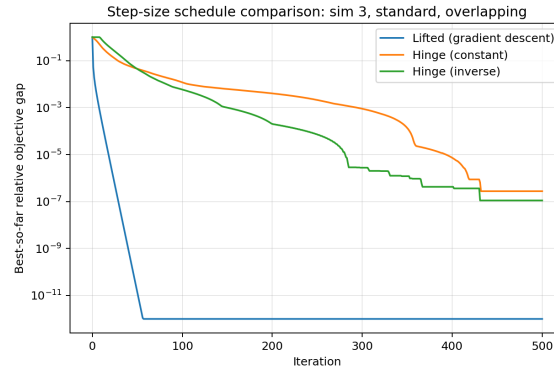


FIGURE 3.16. Plentiful data, Dispersed, Overlapping

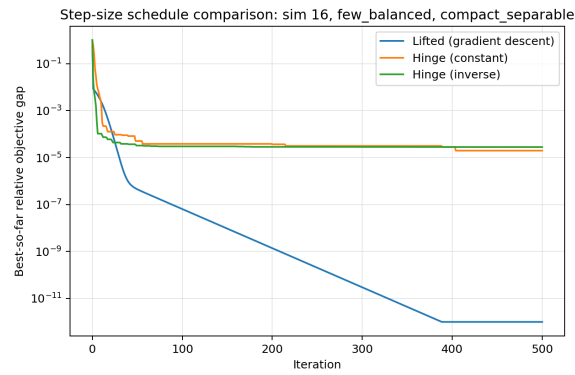


FIGURE 3.17. Few data, Compact, Separable

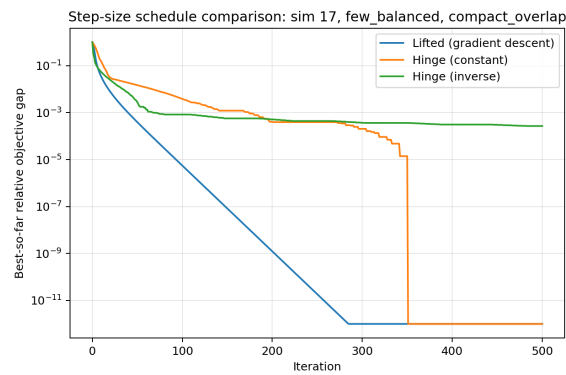


FIGURE 3.18. Few data, Compact, Overlapping

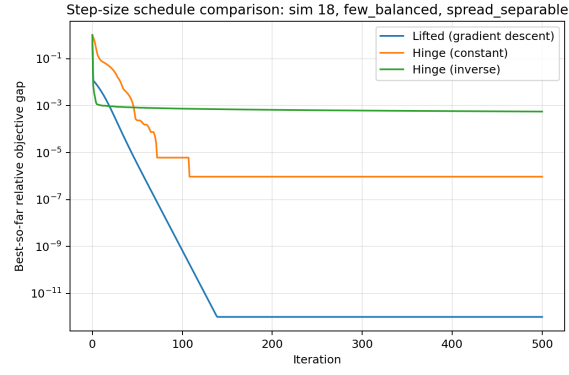


FIGURE 3.19. Few data, Dispersed, Separable

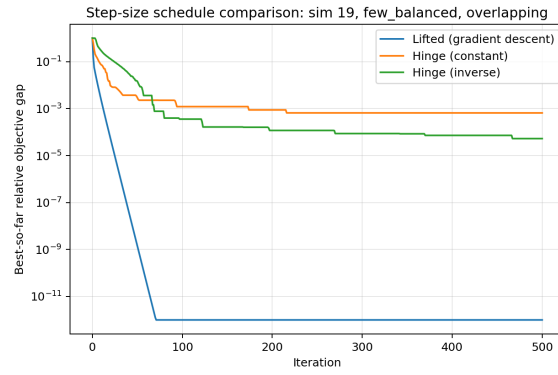


FIGURE 3.20. Few data, Dispersed, Overlapping

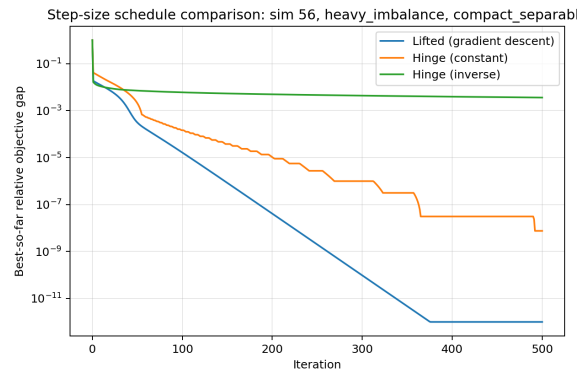


FIGURE 3.21. Heavy imbalance, Compact, Separable

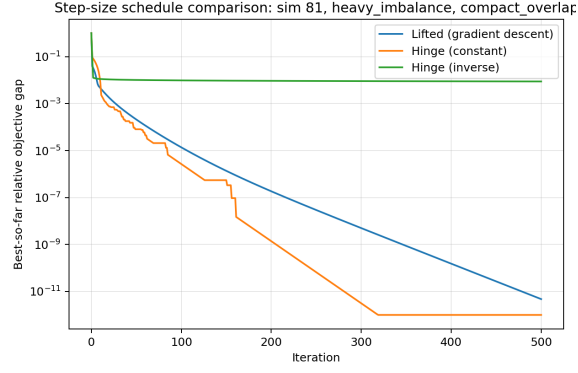


FIGURE 3.22. Heavy imbalance, Compact, Overlapping

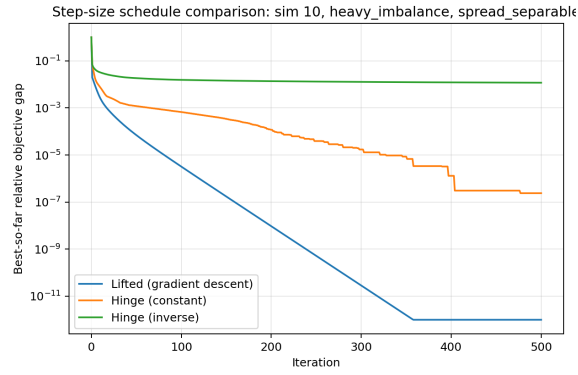


FIGURE 3.23. Heavy imbalance, Dispersed, Separable

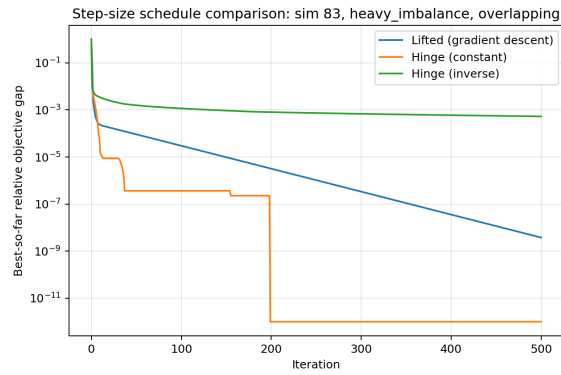


FIGURE 3.24. Heavy imbalance, Dispersed, Overlapping

Below we include a table summarizing the results of the main simulation settings above.

TABLE 3.1. Hinge (constant) vs Lifted: iterations to tolerance and final accuracy

Regime	Method	Iterations to 10^{-3}	Final Accuracy
Standard	Lifted	22.5	0.9353
	Hinge	81.4	0.9356
Few balanced	Lifted	24.9	0.9648
	Hinge	130.1	0.9648
Heavy imbalance	Lifted	46.2	0.9765
	Hinge	81.4	0.9771

Figures 3.13 – 3.24 and Table 3.1 isolate the effect of replacing the ordinary hinge loss by the lifted objective. The two objectives produce essentially the same final classification accuracy in each of the regimes, suggesting that the effect of lifting data points does not harm the strength of the usual SVM classifier. The main difference is instead in optimization behavior. Using the first-order schemes considered here, gradient descent on the smooth lifted objective reaches the objective gap tolerance in substantially fewer iterations than subgradient descent on the hinge-loss objective. Additionally, the subgradient constant step-sizes (Hinge (constant)) were tuned for each simulation, but in practice, a decayed learning rate is required for a stable solution.

This comparison should not be interpreted as claiming that gradient descent on the lifted loss is uniformly faster than all specialized SVM solvers. Instead, it demonstrates the optimization advantage obtained from smoothness when comparable first-order methods are used. This smooth structure is particularly important for the representative constructions developed in the following section, which naturally rely on stable gradient information.

1.4. Representative Constructions. Now that the loss is defined with some basic properties, we will discuss two important representatives we can construct on a given node with this loss function. Consider again

$$\ell(w, b) = (1 - \nu)\Phi(a) + s\phi(a)$$

Motivated by our experiments, we can consider representatives attempting to match the first loss component $(1 - \nu)\Phi(a)$. Note that this depends on ν and Σ . Therefore, a representative will consist of first constructing $\tilde{\nu}$ and $\tilde{\Sigma}$. In order to find $\tilde{x}$ which can be sent to other nodes, we solve

$$(3) \quad \min_{w^\top \tilde{\mathbf{x}} + b = \tilde{\nu}} \|\tilde{x} - \bar{x}\|,$$

which has the analytical solution

$$(4) \quad \tilde{x} = \bar{x} - \frac{w^\top \bar{x} - (\tilde{\nu} - b)}{\|w\|^2} w.$$

For all representative constructions, we compute them per class so that we can reasonably set $y_i = c$ for any data used in the construction and $\tilde{y} = c$ as a reasonable representative label. Then a representative is given by the tuple $(n, \tilde{y}, \tilde{x}, \tilde{\Sigma})$ where n is the number of data being represented, and $\tilde{y} = c$.

Consider a single class set indexed by $\mathcal{I}_{k,c}$ on a given node k , ($c \in \{+1, -1\}$), where $y_i = c$, $i \in \mathcal{I}_{k,c}$. Denote the representative tuple by $(n_{k,c}, \tilde{\mathbf{y}}_{k,c}, \tilde{\mathbf{x}}_{k,c}, \tilde{\Sigma}_{k,c})$. We let $\tilde{\mathbf{y}}_{k,c} = c$ and $n_{k,c}$ be the number of data on node k from class c . We list three possible choices of representatives using the above technique.

1.4.1. *Mean Representative (MR)*. The simplest choice for our representative is to let

$$\tilde{\nu}_{k,c} = n_{k,c}^{-1} \sum_{i \in \mathcal{I}_{k,c}} \nu_i,$$

by just taking the mean of ν_i 's. Recall that ν_i is the margin variable: $\nu_i = y_i(w^\top x_i + b)$.

One can see that this choice is equivalent to simply taking

$$\tilde{\mathbf{x}}_{k,c} = n_{k,c}^{-1} \sum_{i \in \mathcal{I}_{k,c}} \mathbf{x}_i$$

by using the mean projection method given above for a fixed class c . To see this, consider (4) above for $\tilde{\nu}_{k,c} = n_{k,c}^{-1} \sum_{i \in \mathcal{I}_{k,c}} \nu_i$. Fixing $c = 1$ this becomes

$$\begin{aligned}\tilde{x} &= \bar{x} - \frac{w^\top \bar{x} - (n^{-1} \sum_i (\nu_i) - b)}{\|w\|^2} w \\ &= \bar{x} - \frac{w^\top \bar{x} - (n^{-1} \sum_i (w^\top x_i + b) - b)}{\|w\|^2} w\end{aligned}$$

Since $n^{-1} \sum_i w^\top x_i = w^\top \bar{x}$ we have

$$\begin{aligned}\tilde{x} &= \bar{x} - \frac{w^\top \bar{x} - (w^\top \bar{x} + b - b)}{\|w\|^2} w \\ &= \bar{x}\end{aligned}$$

For the covariance matrix, we simply set

$$\tilde{\Sigma}_{k,c} = \text{diag}(\tilde{\sigma}_1^2, \dots, \tilde{\sigma}_p^2) = \Sigma_{k,c}$$

to be the same as the diagonal covariance matrix used for each point in the class. Such a choice of representative does not depend on the parameter being learned. It acts as an approximation to the data rather than the objective function, and often helps with generalizability of the classifier.

1.4.2. *Loss Match.* Another choice is to consider using representative components to match

$$\sum_{i \in \mathcal{I}_{k,c}} (1 - \nu_i) \Phi(a_i) = n_{k,c} (1 - \tilde{\nu}_{k,c}) \Phi(\tilde{a}_{k,c}),$$

where finding $\tilde{\nu}_{k,c}$ and $\tilde{a}_{k,c}$ that satisfy the equation will be the first goal. Here, we use definitions for our representative components that we also have for the data elements:

$$\tilde{a}_{k,c} = \frac{1 - \tilde{\nu}_{k,c}}{\tilde{s}_{k,c}}$$

and $\tilde{s}_{k,c} = \sqrt{w^\top \tilde{\Sigma}_{k,c} w}$. Taking $\tilde{\Sigma}_{k,c}$ as in the previous representative allows us to solve for $\tilde{s}_{k,c}$. Thus again, we only need to find $\tilde{\nu}_{k,c}$. To do this, we simply use Newton's

method to numerically solve the loss matching equation for $\tilde{\nu}_{k,c}$. Once a numerical solution is found, we use the analytical equation (4) to obtain $\tilde{x}_{k,c}$.

Taylor Match. Besides matching this loss term exactly, we could consider approximating with a Taylor series by expanding $\Phi(\frac{1-\nu}{s})$ around $\nu = 1$ which results in needing to match

$$\sum_{i \in \mathcal{I}_{k,c}} a_i \left(\frac{1}{2} + \frac{a_i}{\sqrt{2\pi}} \right) = n_{k,c} \tilde{a}_{k,c} \left(\frac{1}{2} + \frac{\tilde{a}_{k,c}}{\sqrt{2\pi}} \right)$$

instead. However, such an equation results in a quadratic with two possible solutions in general but possibly no real solutions. Once $\tilde{a}_{k,c}$ is chosen, we then directly solve for $\tilde{\nu}_{k,c}$ using the equation

$$\tilde{a}_{k,c} = \frac{1 - \tilde{\nu}_{k,c}}{\tilde{s}_{k,c}}$$

where $\tilde{s}_{k,c} = \sqrt{w^\top \tilde{\Sigma}_{k,c} w}$ with $\tilde{\Sigma}_{k,c} = \Sigma_{k,c}$ being set to the class computed diagonal covariance again.

These representative choices are different from MR due to the dependence on the current model parameter w . It is similar in that it does not attempt to match the optimization problem directly.

1.4.3. *Score-matching Representative (SMR).* The final choice of representative we consider is the so called Score-matching representative. Score here refers to the gradient (we use this term generally despite not having a likelihood function). That is, we solve for a representative matching the local loss gradient at a fixed w for each given class c . We have

$$(5) \quad \sum_{i \in \mathcal{I}_{k,c}} \Phi(a_i) = n_{k,c} \Phi(\tilde{a}_{k,c})$$

for matching the bias derivative equation and

$$(6) \quad n_{k,c}^{-1} \sum_{i \in \mathcal{I}_{k,c}} \left[-\Phi(a_i) c \mathbf{x}_i + \phi(a_i) \frac{\Sigma_{k,c} w}{s_{k,c}} \right] = -\Phi(\tilde{a}_{k,c}) c \tilde{\mathbf{x}}_{k,c} + \phi(\tilde{a}_{k,c}) \frac{\tilde{\Sigma}_{k,c} w}{\tilde{s}_{k,c}}.$$

for matching the w gradient. As before, our goal is to solve for intermediate representative variables with the intention of obtaining $\tilde{x}$ and $\tilde{\Sigma}$ as the final goal.

To solve (5), we have

$$\tilde{a}_{k,c} = \Phi^{-1} \left(n_{k,c}^{-1} \sum_{i \in \mathcal{I}_{k,c}} \Phi(a_i) \right).$$

To solve (6), we first multiply both sides of the equality by $w^\top$ to get a projected matching equation:

$$n_{k,c}^{-1} \sum_{i \in \mathcal{I}_{k,c}} [-\Phi(a_i) c w^\top \mathbf{x}_i + \phi(a_i) s_{k,c}] = -\Phi(\tilde{a}_{k,c}) c w^\top \tilde{\mathbf{x}}_{k,c} + \phi(\tilde{a}_{k,c}) \tilde{s}_{k,c},$$

where we obtained a simplification using $w^\top \Sigma_{k,c} w = s_{k,c}^2$ and $w^\top \tilde{\Sigma}_{k,c} w = \tilde{s}_{k,c}^2$

Then define

$$\mathcal{G}_{k,c} = n_{k,c}^{-1} \sum_{i \in \mathcal{I}_{k,c}} [-\Phi(a_i) c w^\top \mathbf{x}_i + \phi(a_i) s_{k,c}]$$

to be the data portion of the projected equation so that we look to solve,

$$\mathcal{G}_{k,c} = -\Phi(\tilde{a}_{k,c}) c w^\top \tilde{\mathbf{x}}_{k,c} + \phi(\tilde{a}_{k,c}) \tilde{s}_{k,c},$$

Using

$$\tilde{s}_{k,c} = \frac{1 - c(w^\top \tilde{\mathbf{x}}_{k,c} + b)}{\tilde{a}_{k,c}},$$

we have

$$\mathcal{G}_{k,c} = -\Phi(\tilde{a}_{k,c}) c w^\top \tilde{\mathbf{x}}_{k,c} + \phi(\tilde{a}_{k,c}) \frac{1 - c(w^\top \tilde{\mathbf{x}}_{k,c} + b)}{\tilde{a}_{k,c}},$$

which gives us

$$w^\top \tilde{\mathbf{x}}_{k,c} = \left[c \left(\Phi(\tilde{a}_{k,c}) + \frac{\phi(\tilde{a}_{k,c})}{\tilde{a}_{k,c}} \right) \right]^{-1} \left\{ (1 - c) \frac{\phi(\tilde{a}_{k,c})}{\tilde{a}_{k,c}} - \mathcal{G}_{k,c} \right\}.$$

after isolating the projected representative point.

With this, we are able to solve for $\tilde{s}_{k,c}$.

Noting that (6) has p unknowns (for the dimension of our data), we will make an assumption on the representative covariance $\tilde{\Sigma}_{k,c}$ for SMR. Assume $\tilde{\Sigma}_{k,c} = \delta_{k,c} \text{diag}(\tilde{\sigma}_1^2, \dots, \tilde{\sigma}_p^2) = \delta_{k,c} \Sigma_{k,c}$. This differs from previous representatives by allowing a scaling factor of the class computed diagonal covariance. Now, the only unknown for the covariance is $\delta_{k,c}$. This has the solution

$$\delta_{k,c} = (w^\top \Sigma_{k,c} w)^{-1} \tilde{s}_{k,c}^2.$$

where $\tilde{s}_{k,c} = \sqrt{w^\top \tilde{\Sigma}_{k,c} w}$.

Finally, we solve for $\tilde{x}_{k,c}$ in (6)

$$\tilde{\mathbf{x}}_{k,c} = [\Phi(\tilde{a}_{k,c}) c]^{-1} \left\{ \phi(\tilde{a}_{k,c}) \frac{\tilde{\Sigma}_{k,c} w}{\tilde{s}_{k,c}} - n_{k,c}^{-1} \sum_{i \in \mathcal{I}_{k,c}} \left[-\Phi(a_i) c \mathbf{x}_i + \phi(a_i) \frac{\Sigma_{k,c} w}{s_{k,c}} \right] \right\}.$$

with all of the representative parts being solved for.

We then have the full representative tuple $(n_{k,c}, \tilde{y}_{k,c}, \tilde{x}_{k,c}, \tilde{\Sigma}_{k,c})$ where we again set $\tilde{y}_{k,c} = c$ as we construct this class-wise on a given node.

The motivation for such a representative is clear, we are attempting to encode optimization information from each node and therefore match the global optimization problem around a given choice of parameters (w, b) . While sharing gradients is a similar idea, we will see that our distributed algorithms additionally benefit from having a pseudo data point which can perform reasonably when (w, b) have changed.

1.5. Distributed Algorithm. With all of our choices of representatives defined above, we are ready to define a distributed representative learning algorithm (or RepL for short) in order to learn a global lifted svm solution from separated nodes. Each

node will be equipped with their local lifted svm loss along with an additional loss component for representatives. The representative loss will be obtained by using all of the representatives constructed from other nodes at every given outer iteration.

After node k receives representatives $\{(n_{h,c}, \tilde{y}_{h,c}, \tilde{x}_{h,c}, \tilde{\Sigma}_{h,c})\}_{h \neq k, c \in \{-1,1\}}$ from other nodes, it integrates them with its own local data $\{(x_i, y_i)\}_{i \in \mathcal{I}_k}$ to form a unified lifted empirical risk.

For the linear SVM, the lifted objective becomes

$$(7) \quad \mathcal{L}(w, b) = \frac{1}{2} \|w\|^2 + C \left\{ \sum_{c \in \{-1, +1\}} \sum_{i \in \mathcal{I}_{k,c}} [(1 - \nu_i) \Phi(a_i) + s_{k,c} \phi(a_i)] \right. \\ \left. + \sum_{c \in \{-1,1\}} \sum_{h \neq k} n_{h,c} [(1 - \tilde{\nu}_{h,c}) \Phi(\tilde{a}_{h,c}) + \tilde{s}_{h,c} \phi(\tilde{a}_{h,c})] \right\}.$$

where $\tilde{s}_{h,c} = \sqrt{w^\top \tilde{\Sigma}_{h,c} w}$ using the local node's model w . For experiments we will use $C \rightarrow C/n$. This does not affect the optimization problem but allows for an easier comparison with our representative methods to other established methods.

Our representative algorithm proceeds as follows:

(Initialize: (optional) each local node with gradient descent.)

For outer iterations $\mathbf{T} = \mathbf{1}, \dots$ and each node $\mathbf{k} = \mathbf{1}, \dots \mathbf{K}$

- (1) Computes representatives $(n_{k,c}, \tilde{y}_{k,c}, \tilde{x}_{k,c}^{(t)}, \tilde{\Sigma}_{k,c}^{(t)})$ using local data and/or models.
- (2) Communicate representatives to all connected nodes (either directly or through a central server).
- (3) Optimize (7) using gradient descent.

For the experiments below, we consider using varying number of local steps of gradient descent between computing representatives.

2. Experiments

2.1. Representatives for Single Node Training. Before demonstrating our distributed algorithm, we would first like to see how representative diagonal covariance matrices will affect the loss. We aim to check the derived covariance $\tilde{\Sigma}$ for SMR on the same single node data examples from section 1.3. The plots include the point labeled as an “x” as well as a two-standard-deviation ellipse of the covariance matrix. The left plots are from the early stages of learning (iterations range from 1 to 3) while the right is from training conclusion.

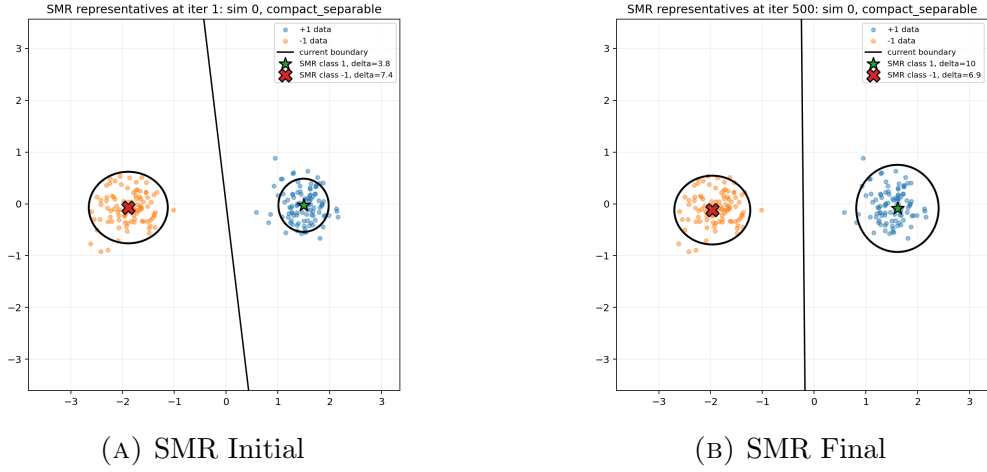


FIGURE 3.25. Plentiful data, Compact, Separable

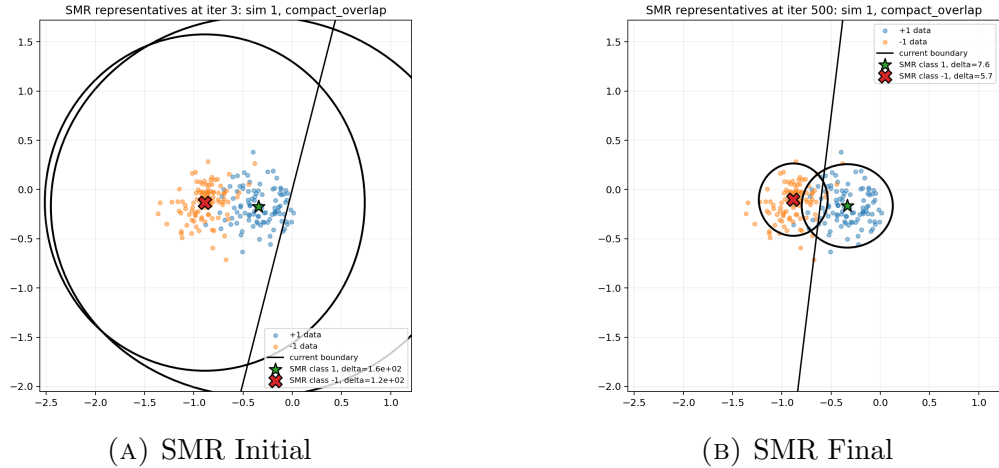
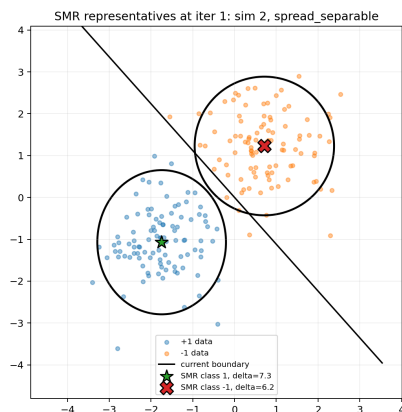
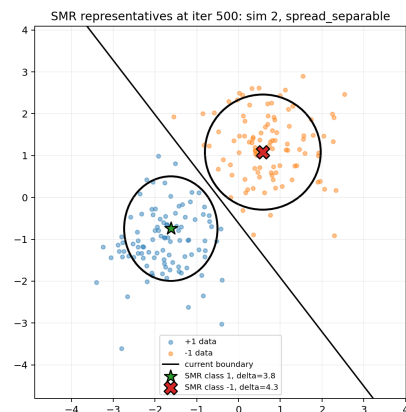


FIGURE 3.26. Plentiful data, Compact, Overlapping

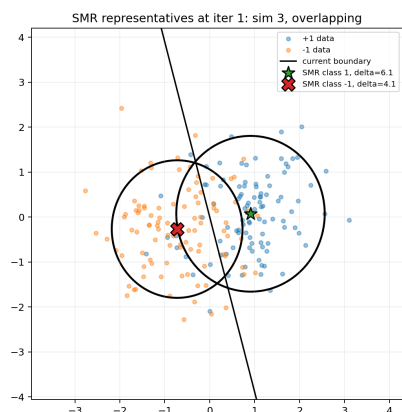


(A) SMR initial

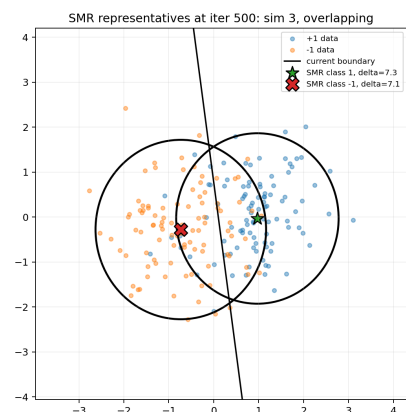


(B) SMR final

FIGURE 3.27. Plentiful data, Dispersed, Separable

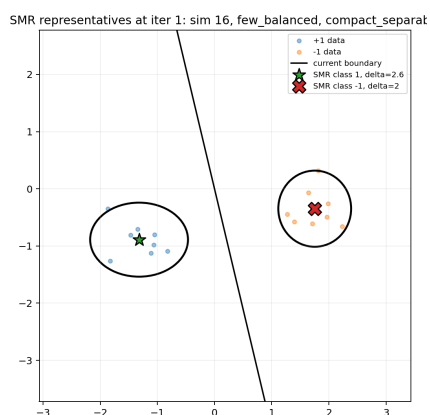


(A) SMR initial

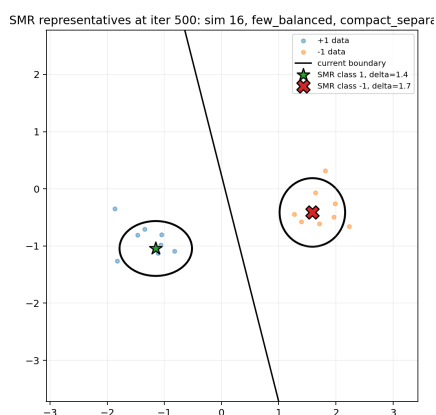


(B) SMR final

FIGURE 3.28. Plentiful data, Dispersed, Overlapping

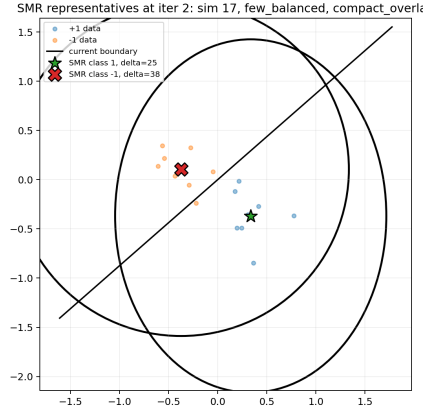


(A) SMR initial

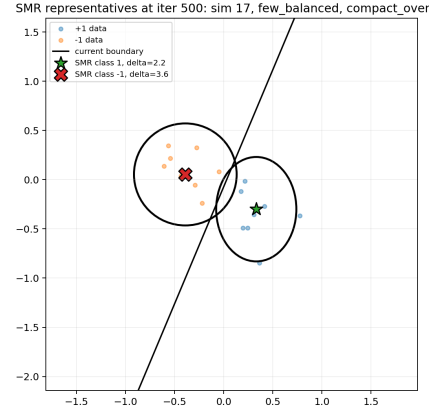


(B) SMR final

FIGURE 3.29. Few data, Compact, Separable

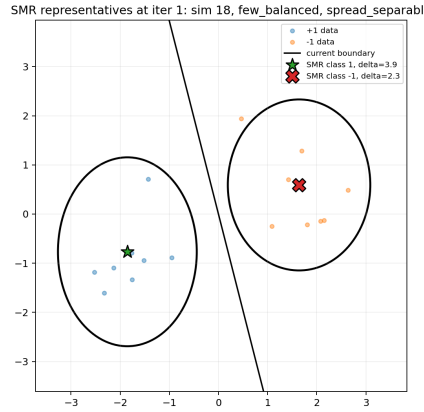


(A) SMR initial

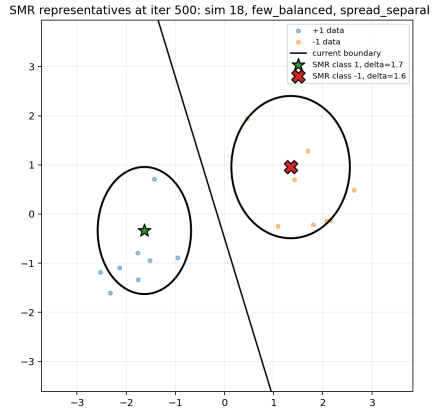


(B) SMR final

FIGURE 3.30. Few data, Compact, Overlapping

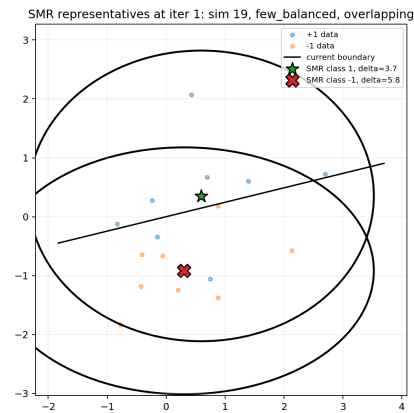


(A) SMR initial

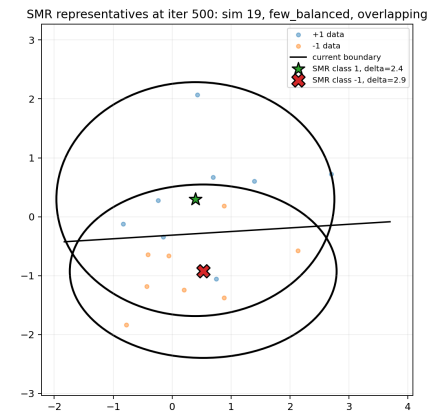


(B) SMR final

FIGURE 3.31. Few data, Dispersed, Separable

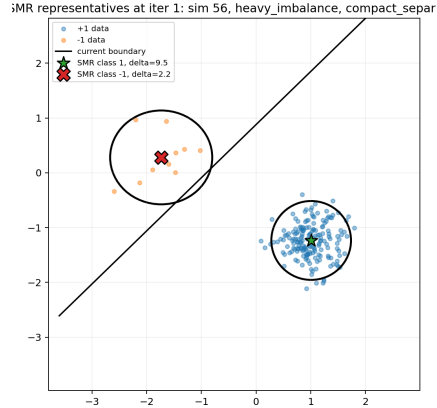


(A) SMR initial

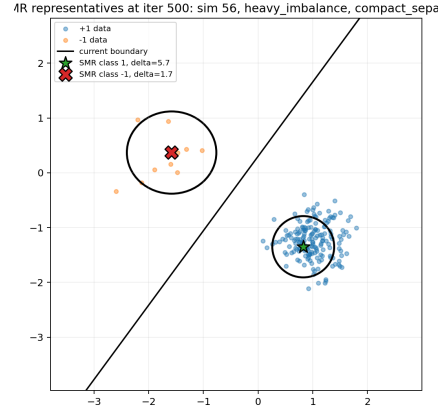


(B) SMR final

FIGURE 3.32. Few data, Dispersed, Overlapping

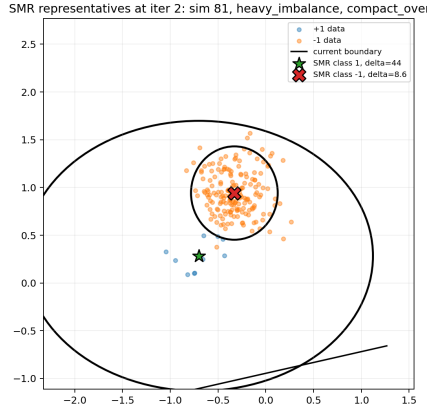


(A) SMR initial

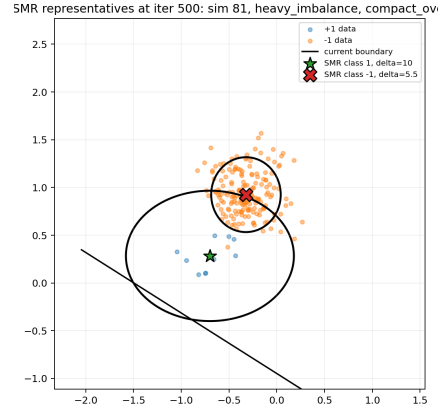


(B) SMR final

FIGURE 3.33. Heavy Imbalance, Compact, Separable

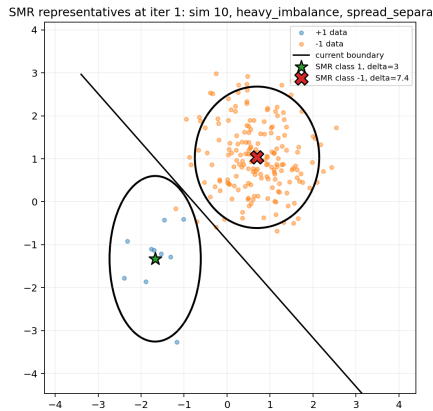


(A) SMR initial

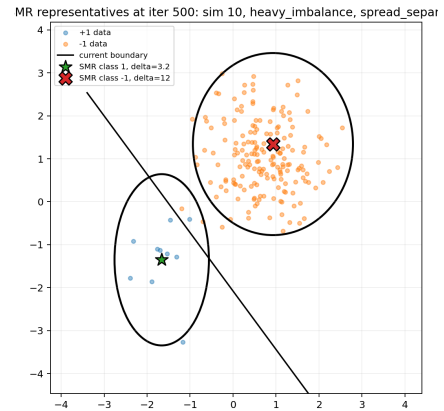


(B) SMR final

FIGURE 3.34. Heavy Imbalance, Compact, Overlapping



(A) SMR initial



(B) SMR final

FIGURE 3.35. Heavy Imbalance, Dispersed, Separable

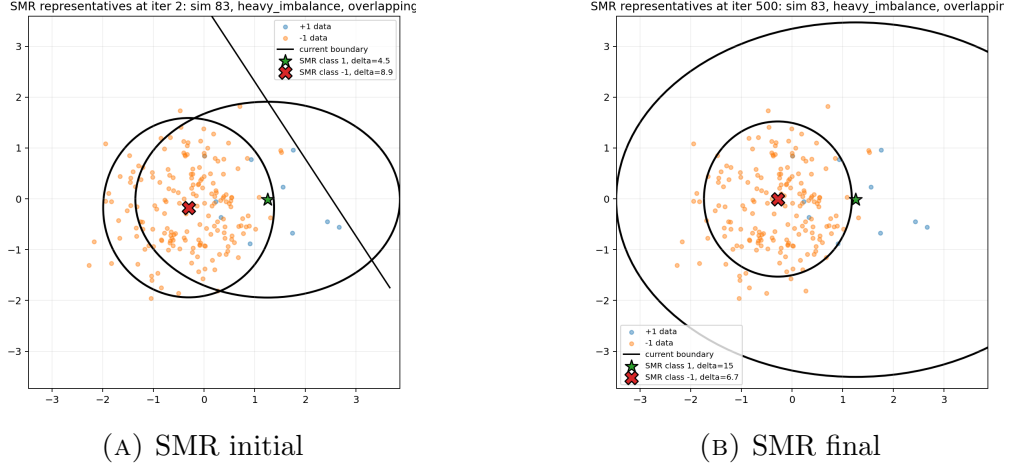


FIGURE 3.36. Heavy Imbalance, Dispersed, Overlapping

Figures 3.25 – 3.36 illustrate how the two components of the SMR, $\tilde{x}$ and $\tilde{\Sigma}$, change during optimization. Across stable configurations, the representative location remains within or near the central region of its class, while the covariance scales for the required gradient/score matching.

The heavily imbalanced examples expose a less stable regime. When the local classifier approaches a nearly single-class solution and $\|w\|$ becomes small, the covariance scaling required by the score-matching equations can become large. These examples motivate the boundedness assumptions introduced later in the theoretical analysis and illustrate a limitation of the unconstrained SMR construction under locally degenerate classification problems.

2.2. Simulations on Synthetic Data. Here we give the main results for our algorithm on simulated mixture Gaussian data. For representatives, we only compare **MR** and **SMR**. While the loss-matching representatives performed somewhat reasonably, MR and SMR were more robust, with stronger performance.

We claim that moving from the nonsmooth hinge loss to our lifted loss has not harmed the usefulness of representatives nor learned solutions. To see this, we add in comparisons for our representatives to their hinge loss counterparts: MR (Hinge) and SMR (Hinge). MR (Hinge) denotes the mean data point in input space. For

SMR (Hinge), we note that hinge loss is not smooth, and therefore the construction attempts to incorporate subgradient information as follows:

For a single observation (x_i, y_i) , the hinge loss is

$$\ell_i(w, b) = \max \{0, 1 - y_i(w^T x_i + b)\},$$

whose subgradient with respect to w is

$$\partial_w \ell_i = \begin{cases} -y_i x_i, & y_i(w^T x_i + b) < 1, \\ 0, & y_i(w^T x_i + b) \geq 1 \end{cases}$$

Similarly,

$$\partial_b \ell_i = \begin{cases} -y_i, & y_i(w^T x_i + b) < 1, \\ 0, & y_i(w^T x_i + b) \geq 1. \end{cases}$$

Thus, only observations with positive hinge loss contribute to the optimization.

Let

$$\mathcal{A} = \{i : y_i(w^T x_i + b) < 1\}$$

denote the active set of hinge loss samples at the current parameter iterate. The representative is then constructed by averaging only these active observations:

$$\tilde{x} = \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} x_i,$$

with one representative computed for each class. This representative therefore summarizes the observations currently contributing to the hinge loss gradient, while ignoring correctly classified samples lying outside the margin.

The other two comparisons are FedAvg and FedProx. These methods involve sending parameters which encode gradient information from local optimization (using the lifted loss) and are therefore important for comparing against lifted SMR. When

gradient descent is used for local updates, FedAvg is equivalent to each node sending the current gradient and then the central server performing an aggregated parameter update. FedProx extends FedAvg to heterogeneous data by adding in a proximal term to penalize local drift from the current aggregate parameter. This is intended to reduce client drift from extended local optimization (e.g. many local gradient descent steps).

We include three choices of optimization that are independent of local gradient (or subgradient) descent steps. Namely, we have “Global Pooled” for the lifted SVM which trains on all distributed data combined into one dataset. We also include “Classical Linear SVM” which optimizes soft margin SVM (as defined in [4]) on the same combined dataset. These two help us frame our ideal distributed learning objectives, as well as the difference induced by smoothing out the loss. Lastly, we have ADMM which minimizes hinge loss on each node subject to the constraint that each node’s parameters are the same ($w_k = w$ for all nodes k). In this case, ADMM solves these optimization problems on each node in every outer iteration, and it is not affected by the local iterations.

More precisely, each node solves a local SVM optimization problem while enforcing agreement among all local models:

$$\min_{\{w_k, b_k\}} \sum_{k=1}^K \left[\frac{1}{2} \|w_k\|^2 + C \sum_{i \in \mathcal{D}_k} \max(0, 1 - y_i(w_k^T x_i + b_k)) \right]$$

subject to

$$w_1 = \dots = w_K, \quad b_1 = \dots = b_K.$$

This comparison is added in to see a strong solution to distributed hinge loss, but the necessity for frequent communication and optimization leaves it just as an ideal goal for hinge based distributed learning methods.

In order to simulate data, we draw from mixture Gaussians representing classes 1 and -1 . We choose Gaussians with relatively high overlap. This keeps the simulations simple with Gaussian-like class distributions and focuses on a class imbalance form of heterogeneity between nodes. We consider two main cases, when these classes are well separated and when they overlap. To obtain a split dataset among three nodes, we use Dirichlet distributions with parameter $\alpha = 0.1$ (drawn on the initial mixture Gaussian data) to generate node data with heavily imbalanced classes. In total, we run 100 simulations on each of low and high overlap settings. The complete simulation configuration, covariance construction, Dirichlet partitioning procedure, initialization strategy, and remaining implementation details are provided in Appendix A.

The tables below consider training and test accuracies across 4 different data dimensions. The test data are generated from the same mixture Gaussians as the training set. Each node learns its own parameter with an associated train and test accuracy. At the end, we average these node accuracies. These accuracies are averaged over 100 simulations, and the standard deviations are reported. For stability in computing variances, we enforce a minimum of 2 data points per class on each node.

Since accuracies are averaged, we include a third table detailing the agreement in parameters for lifted SMR. While under certain settings, FedAvg and FedProx may have parameter agreement, it is generally the case that all other methods do not result in parameters close enough to claim equivalence. Parameter agreement is considered as successful if the maximum euclidean norm distance between nodes is smaller than some threshold (in our case this is set at 10^{-6} but agreement percentages are relatively insensitive to this choice).

For each simulation, we begin by initializing each node with some iterations of local training. Then, we run 25 outer iterations with 40 local gradient descent steps on each node per outer iteration. This is enough local steps to see a decline in parameter based methods like FedAvg and FedProx away from the global pooled solution. MR (Hinge) and MR (Lifted) use representatives which are invariant under

outer iterations, while other methods will construct new representatives to share at each outer iteration.

Our primary interest with simulations is seeing accuracy degrade for FedAvg while lifted SMR remains strong. This is because both methods are aimed at solving the global optimization problem, but FedAvg suffers from client drift due to local optimization steps (or node heterogeneity). Lifted SMR has the same goal but aims to mitigate this drift while remaining close to the global pooled solution.

We begin with a set of simple simulations on classes with very little overlap.

TABLE 3.2. Training accuracy over 100 simulations for low-overlap classes.

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
MR (Hinge)	0.9298 ± 0.0582	0.9341 ± 0.0541	0.9391 ± 0.0558	0.9392 ± 0.0528
SMR (Hinge)	0.8703 ± 0.1392	0.8767 ± 0.1329	0.8935 ± 0.1228	0.8911 ± 0.1124
SMR (Lifted)	0.9537 ± 0.0538	0.9577 ± 0.0462	0.9639 ± 0.0475	0.9624 ± 0.0445
MR (Lifted)	0.9461 ± 0.0540	0.9515 ± 0.0473	0.9580 ± 0.0485	0.9572 ± 0.0455
FedAvg (Lifted)	0.9476 ± 0.0606	0.9544 ± 0.0489	0.9585 ± 0.0567	0.9595 ± 0.0493
FedProx (Lifted)	0.9477 ± 0.0604	0.9544 ± 0.0489	0.9585 ± 0.0566	0.9595 ± 0.0493
ADMM (Hinge)	0.9530 ± 0.0545	0.9571 ± 0.0475	0.9643 ± 0.0476	0.9618 ± 0.0479
Classical Linear	0.9539	0.9579	0.9648	0.9630
SVM (Hinge)	± 0.0540	± 0.0470	± 0.0487	± 0.0456
Global Pooled (Lifted)	0.9534 ± 0.0538	0.9579 ± 0.0461	0.9641 ± 0.0475	0.9625 ± 0.0445

TABLE 3.3. Test accuracy over 100 simulations for low-overlap classes.

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
MR (Hinge)	0.9174 ± 0.0624	0.9297 ± 0.0554	0.9249 ± 0.0623	0.9220 ± 0.0598
SMR (Hinge)	0.8584 ± 0.1550	0.8725 ± 0.1504	0.8768 ± 0.1447	0.8742 ± 0.1324
SMR (Lifted)	0.9295 ± 0.0601	0.9463 ± 0.0481	0.9418 ± 0.0543	0.9391 ± 0.0507
MR (Lifted)	0.9284 ± 0.0573	0.9433 ± 0.0475	0.9410 ± 0.0545	0.9386 ± 0.0506
FedAvg (Lifted)	0.9170 ± 0.0766	0.9359 ± 0.0605	0.9301 ± 0.0715	0.9299 ± 0.0590
FedProx (Lifted)	0.9174 ± 0.0760	0.9355 ± 0.0610	0.9302 ± 0.0716	0.9297 ± 0.0590
ADMM (Hinge)	0.9296 ± 0.0603	0.9452 ± 0.0505	0.9416 ± 0.0548	0.9372 ± 0.0542
Classical Linear	0.9259	0.9437	0.9427	0.9375
SVM (Hinge)	± 0.0635	± 0.0485	± 0.0543	± 0.0511
Global Pooled	0.9290	0.9459	0.9416	0.9381
(Lifted)	± 0.0603	± 0.0485	± 0.0543	± 0.0508

TABLE 3.4. Low-overlap linear SVM percentage of simulations with local model agreement (Medium local/outer iterations)

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
SMR (Lifted)	97%	96%	97%	94%

Tables 3.2–3.4 show that when the underlying classes have low overlap, the lifted representative methods recover performance close to the centralized SVM objectives despite the strongly heterogeneous Dirichlet partition. Lifted SMR is consistently

among the strongest distributed methods in training accuracy and remains close to the centralized lifted solution in test accuracy. In addition, comparing the lifted and hinge versions of SMR shows a substantial improvement from the lifted objective, indicating that the lifted formulation benefits score-matching representatives beyond simply improving optimization. In contrast, the difference between lifted MR and hinge MR is less pronounced, further suggesting that the primary advantage of lifting is realized when representative construction explicitly depends on the objective. Lifted MR is similarly competitive, indicating that class-wise location information is already highly informative in this relatively simple linear setting.

An additional factor influencing these results is the strong class imbalance induced by the Dirichlet partitioning. In the heavy-imbalance settings, some local nodes contain insufficient class information to identify a meaningful separating direction. In these cases, both the ordinary and lifted SVM objectives may favor a nearly single-class classifier locally. The resulting behavior should therefore be interpreted as a stress test of representative construction under local degeneracy rather than as the intended operating regime. For SMR, this degeneracy can inflate the covariance scaling factor because the representative must compensate for a weak or nearly vanishing local direction w . Consequently, the resulting representative may be less stable than in balanced settings, where a well-defined separating direction exists.

Of particular importance for the distributed objective, SMR produces model consensus the vast majority of the time. Thus, the principal evidence in this regime is not that SMR improves predictive accuracy over every alternative, but that it preserves centralized performance while driving local models together. Increased tuning and performance of the global solutions are therefore reflected in lifted SMR solutions, even when a reasonable number of local steps are included between communication rounds. With the choice of 40 local steps, models for the other compared methods did not agree under the same tolerance (although they predict reasonably similarly).

Next we consider the case of high overlap in the classes on each of the nodes. In this setting, it is worth considering different step-size rules to analyze the local client drift for parameter based methods vs representative methods. We consider three different scenarios:

- (1) Medium local/outer iterations (specifically 25 outer iterations with 40 local iterations per outer; the same as the single low overlap setting)
- (2) One outer iteration followed by all local (beginning with a warm local training start)
- (3) Many outer iterations with only one local step between

Each scenario includes three of the same tables as before. The first scenario is the same as considered in the low class overlap case. This is motivated by the need to reduce communication while retaining strong solutions in DML. The last two are more extreme: Most parameter based methods perform poorly in the second case, but very well in the third.

TABLE 3.5. Training accuracy over 100 simulations for high-overlap classes (Medium local/outer iterations).

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
MR (Hinge)	0.8039 ± 0.0925	0.8076 ± 0.0931	0.8267 ± 0.0908	0.8316 ± 0.0899
SMR (Hinge)	0.6946 ± 0.1573	0.6954 ± 0.1553	0.7091 ± 0.1496	0.6975 ± 0.1416
SMR (Lifted)	0.8210 ± 0.0949	0.8240 ± 0.0974	0.8539 ± 0.0972	0.8504 ± 0.0929
MR (Lifted)	0.8194 ± 0.0954	0.8220 ± 0.0974	0.8502 ± 0.0967	0.8501 ± 0.0920
FedAvg (Lifted)	0.7906 ± 0.1235	0.7902 ± 0.1189	0.8242 ± 0.1246	0.8147 ± 0.1272
FedProx (Lifted)	0.7926 ± 0.1207	0.7882 ± 0.1225	0.8235 ± 0.1258	0.8157 ± 0.1261
ADMM (Hinge)	0.8004 ± 0.1180	0.8073 ± 0.1175	0.8410 ± 0.1201	0.8390 ± 0.1073
Classical Linear	0.8179	0.8230	0.8539	0.8500
SVM (Hinge)	± 0.1019	± 0.1018	± 0.1003	± 0.0964
Global Pooled (Lifted)	0.8209 ± 0.0949	0.8239 ± 0.0970	0.8543 ± 0.0969	0.8516 ± 0.0911

TABLE 3.6. Test accuracy over 100 simulations for high-overlap classes (Medium local/outer iterations).

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
MR (Hinge)	0.7599 ± 0.0982	0.7811 ± 0.0920	0.7866 ± 0.0968	0.7834 ± 0.0913
SMR (Hinge)	0.6653 ± 0.1474	0.6838 ± 0.1497	0.6872 ± 0.1544	0.6746 ± 0.1391
SMR (Lifted)	0.7545 ± 0.1049	0.7860 ± 0.0910	0.7995 ± 0.1052	0.7879 ± 0.0965
MR (Lifted)	0.7601 ± 0.1003	0.7873 ± 0.0913	0.7990 ± 0.1039	0.7938 ± 0.0942
FedAvg (Lifted)	0.7345 ± 0.1258	0.7478 ± 0.1221	0.7650 ± 0.1241	0.7417 ± 0.1289
FedProx (Lifted)	0.7356 ± 0.1191	0.7469 ± 0.1251	0.7653 ± 0.1252	0.7427 ± 0.1266
ADMM (Hinge)	0.7437 ± 0.1223	0.7675 ± 0.1124	0.7833 ± 0.1308	0.7819 ± 0.1017
Classical Linear	0.7482	0.7861	0.7981	0.7911
SVM (Hinge)	± 0.1126	± 0.0933	± 0.1044	± 0.0952
Global Pooled	0.7539	0.7868	0.8005	0.7895
(Lifted)	± 0.1044	± 0.0909	± 0.1057	± 0.0950

TABLE 3.7. High-overlap linear SVM percentage of simulations with local model agreement (Medium local/outer iterations).

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
SMR (Lifted)	99%	100%	100%	95%

TABLE 3.8. Training accuracy over 100 simulations for high-overlap classes (One outer iteration).

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
MR (Hinge)	0.8039 ± 0.0925	0.8076 ± 0.0931	0.8267 ± 0.0908	0.8316 ± 0.0899
SMR (Hinge)	0.5881 ± 0.0866	0.5879 ± 0.0970	0.6009 ± 0.1104	0.5942 ± 0.0905
SMR (Lifted)	0.8046 ± 0.1145	0.8065 ± 0.1100	0.8386 ± 0.1155	0.8404 ± 0.1013
MR (Lifted)	0.8194 ± 0.0954	0.8220 ± 0.0974	0.8502 ± 0.0967	0.8501 ± 0.0920
FedAvg (Lifted)	0.7515 ± 0.1300	0.7354 ± 0.1380	0.7735 ± 0.1364	0.7334 ± 0.1495
FedProx (Lifted)	0.7462 ± 0.1365	0.7394 ± 0.1378	0.7773 ± 0.1346	0.7348 ± 0.1508
ADMM (Hinge)	0.7538 ± 0.1284	0.7592 ± 0.1262	0.7986 ± 0.1225	0.7762 ± 0.1328
Classical Linear	0.8179	0.8230	0.8539	0.8500
SVM (Hinge)	± 0.1019	± 0.1018	± 0.1003	± 0.0964
Global Pooled (Lifted)	0.8209 ± 0.0949	0.8239 ± 0.0970	0.8543 ± 0.0969	0.8516 ± 0.0911

TABLE 3.9. Test accuracy over 100 simulations for high-overlap classes (One outer iteration).

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
MR (Hinge)	0.7599 ± 0.0982	0.7811 ± 0.0920	0.7866 ± 0.0968	0.7834 ± 0.0913
SMR (Hinge)	0.5714 ± 0.1126	0.5797 ± 0.1292	0.5904 ± 0.1340	0.5847 ± 0.1143
SMR (Lifted)	0.7399 ± 0.1101	0.7648 ± 0.1096	0.7856 ± 0.1175	0.7809 ± 0.1035
MR (Lifted)	0.7601 ± 0.1003	0.7873 ± 0.0913	0.7990 ± 0.1039	0.7938 ± 0.0942
FedAvg (Lifted)	0.6980 ± 0.1358	0.6828 ± 0.1414	0.7152 ± 0.1400	0.6777 ± 0.1510
FedProx (Lifted)	0.6940 ± 0.1410	0.6859 ± 0.1412	0.7191 ± 0.1380	0.6790 ± 0.1520
ADMM (Hinge)	0.6993 ± 0.1351	0.7058 ± 0.1334	0.7418 ± 0.1323	0.7207 ± 0.1372
Classical Linear	0.7482	0.7861	0.7981	0.7911
SVM (Hinge)	± 0.1126	± 0.0933	± 0.1044	± 0.0952
Global Pooled (Lifted)	0.7539 ± 0.1044	0.7868 ± 0.0909	0.8005 ± 0.1057	0.7895 ± 0.0950

TABLE 3.10. High-overlap linear SVM percentage of simulations with local model agreement (One outer iteration).

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
SMR (Lifted)	0%	0%	0%	0%

TABLE 3.11. Training accuracy over 100 simulations for high-overlap classes (One local per outer).

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
MR (Hinge)	0.8039 ± 0.0925	0.8076 ± 0.0931	0.8267 ± 0.0908	0.8316 ± 0.0899
SMR (Hinge)	0.7066 ± 0.1546	0.6919 ± 0.1497	0.7273 ± 0.1527	0.7007 ± 0.1504
SMR (Lifted)	0.8210 ± 0.0949	0.8240 ± 0.0974	0.8539 ± 0.0972	0.8502 ± 0.0931
MR (Lifted)	0.8194 ± 0.0954	0.8220 ± 0.0974	0.8502 ± 0.0967	0.8502 ± 0.0920
FedAvg (Lifted)	0.8210 ± 0.0949	0.8240 ± 0.0974	0.8539 ± 0.0972	0.8506 ± 0.0927
FedProx (Lifted)	0.8210 ± 0.0949	0.8240 ± 0.0974	0.8539 ± 0.0972	0.8506 ± 0.0927
ADMM (Hinge)	0.8216 ± 0.0947	0.8254 ± 0.0966	0.8546 ± 0.0975	0.8517 ± 0.0917
Classical Linear	0.8179	0.8230	0.8539	0.8500
SVM (Hinge)	± 0.1019	± 0.1018	± 0.1003	± 0.0964
Global Pooled (Lifted)	0.8209 ± 0.0949	0.8239 ± 0.0970	0.8543 ± 0.0969	0.8516 ± 0.0911

TABLE 3.12. Test accuracy over 100 simulations for high-overlap classes (One local per outer).

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
MR (Hinge)	0.7599 ± 0.0982	0.7811 ± 0.0920	0.7866 ± 0.0968	0.7834 ± 0.0913
SMR (Hinge)	0.6719 ± 0.1439	0.6801 ± 0.1458	0.7047 ± 0.1488	0.6833 ± 0.1477
SMR (Lifted)	0.7545 ± 0.1049	0.7860 ± 0.0910	0.7995 ± 0.1052	0.7882 ± 0.0965
MR (Lifted)	0.7601 ± 0.1003	0.7873 ± 0.0913	0.7990 ± 0.1039	0.7938 ± 0.0942
FedAvg (Lifted)	0.7545 ± 0.1053	0.7860 ± 0.0913	0.7995 ± 0.1056	0.7885 ± 0.0966
FedProx (Lifted)	0.7545 ± 0.1053	0.7860 ± 0.0913	0.7995 ± 0.1056	0.7885 ± 0.0966
ADMM (Hinge)	0.7562 ± 0.1048	0.7867 ± 0.0905	0.8007 ± 0.1049	0.7889 ± 0.0954
Classical Linear	0.7482	0.7861	0.7981	0.7911
SVM (Hinge)	± 0.1126	± 0.0933	± 0.1044	± 0.0952
Global Pooled (Lifted)	0.7539 ± 0.1044	0.7868 ± 0.0909	0.8005 ± 0.1057	0.7895 ± 0.0950

TABLE 3.13. High-overlap linear SVM percentage of simulations with local model agreement (One local per outer).

Method	$d = 2$	$d = 3$	$d = 5$	$d = 7$
SMR (Lifted)	100%	100%	100%	96%

We summarize the results for each of the local step/outer iteration regimes below:

- (1) Medium local/outer: Under the medium communication setting, the effect of heterogeneity is more apparent than in the low overlap case. FedAvg and FedProx fall below the centralized objectives as repeated local optimization allows models to drift toward their local datasets. In contrast, both lifted representative methods remain substantially closer to centralized performance. Lifted SMR also achieves local model agreement in 95 – 100% of simulations, indicating that representative exchange is sufficient to maintain a common optimization trajectory despite multiple local updates between communications.
- (2) One outer iteration/communication round: This setting provides the strongest stress test of client drift. After representatives or model information are exchanged only once, the nodes perform extensive local optimization. FedAvg, FedProx, and ADMM deteriorate substantially relative to their centralized counterparts (note again that ADMM optimizes completely for each outer iteration and does not actually use an equivalent local descent step). Lifted MR remains remarkably stable under this regime and becomes the strongest distributed representative method in both training and test accuracy. Because MR depends only on the local data and not on the current model state, its representative remains valid even after substantial local optimization.

Lifted SMR exhibits a different behavior. SMR encodes first-order information about the objective at the parameter value where the representative is constructed. As local optimization proceeds, this information becomes progressively less representative of the local objective. Consequently, although lifted SMR still substantially outperforms FedAvg and FedProx, it loses much of the parameter agreement seen in the other communication regimes.

The model-agreement results reinforce this interpretation. Lifted SMR achieves agreement in 95% – 100% of simulations under moderate communication and 96% – 100% of simulations when communication is frequent, but

drops to 0% agreement when only a single communication round is allowed. Thus, the score-matching representative should be viewed as a periodically refreshed approximation to the local objective rather than a static summary of the local data. These results illustrate an important distinction between MR and SMR: MR summarizes the local data distribution itself, whereas SMR summarizes the local optimization landscape near the model at which it is constructed.

- (3) When communication is frequent and only one local gradient step is taken between exchanges, the differences between the primary distributed methods largely disappear. FedAvg, FedProx, lifted SMR, and ADMM all track the centralized objectives closely, while lifted SMR again achieves parameter agreement in 96% – 100% of simulations. This is consistent with the expected behavior of distributed first-order methods: when local models are allowed little opportunity to drift before information is exchanged, heterogeneity has substantially less influence on the optimization trajectory. Compared with the previous settings, the primary advantage of representative methods is their ability to maintain strong solutions with substantially fewer communication rounds.

Overall, the communication experiments distinguish the roles of MR and SMR. MR is a model independent approximation of the local data and is therefore particularly robust when representatives are communicated only once. SMR, by contrast, is designed to approximate the local objective through first-order matching and performs best when it is periodically refreshed (via outer iterations) as the model evolves. Under moderate communication, lifted SMR maintains both centralized level predictive performance and strong agreement among local models, while parameter based methods exhibit greater degradation as the number of local iterations increases.

The agreement tables above compare node models to one another and therefore measure consistency across distributed nodes. However, agreement alone does not imply

recovery of the centralized optimization solution. We next compare each distributed method directly to the centralized lifted SVM solution in order to determine whether agreement corresponds to recovery of the centralized optimizer. This is only done for the medium local iteration/ medium outer iteration regime.

TABLE 3.14. Low overlap parameter distance ($d = 2$)

Method	$\ w - w_{\text{global}}\ _2$	Cosine Similarity	$ b - b_{\text{global}} $
SMR (Lifted)	0.0444	0.9970	0.0208
SMR (Hinge)	0.4497	0.7857	0.5004
MR (Lifted)	0.6032	0.9929	0.2478
MR (Hinge)	0.7943	0.9698	0.3686

TABLE 3.15. Low overlap parameter distance ($d = 3$)

Method	$\ w - w_{\text{global}}\ _2$	Cosine Similarity	$ b - b_{\text{global}} $
SMR (Lifted)	0.0457	0.9968	0.0221
SMR (Hinge)	0.5059	0.8265	0.5208
MR (Lifted)	0.5946	0.9912	0.2214
MR (Hinge)	0.7335	0.9598	0.3142

TABLE 3.16. Low overlap parameter distance ($d = 5$)

Method	$\ w - w_{\text{global}}\ _2$	Cosine Similarity	$ b - b_{\text{global}} $
SMR (Lifted)	0.0618	0.9912	0.0239
SMR (Hinge)	0.5156	0.8624	0.4638
MR (Lifted)	0.5951	0.9908	0.1995
MR (Hinge)	0.7337	0.9558	0.2876

TABLE 3.17. Low overlap parameter distance ($d = 7$)

Method	$\ w - w_{\text{global}}\ _2$	Cosine Similarity	$ b - b_{\text{global}} $
SMR (Lifted)	0.0651	0.9952	0.0244
SMR (Hinge)	0.5498	0.8649	0.4859
MR (Lifted)	0.5983	0.9901	0.2126
MR (Hinge)	0.7578	0.9551	0.3118

TABLE 3.18. High overlap parameter distance ($d = 2$)

Method	$\ w - w_{\text{global}}\ _2$	Cosine Similarity	$ b - b_{\text{global}} $
SMR (Lifted)	0.0475	0.9958	0.0242
SMR (Hinge)	0.5200	0.7105	0.5544
MR (Lifted)	0.6860	0.9896	0.2904
MR (Hinge)	0.8847	0.9643	0.4139

TABLE 3.19. High overlap parameter distance ($d = 3$)

Method	$\ w - w_{\text{global}}\ _2$	Cosine Similarity	$ b - b_{\text{global}} $
SMR (Lifted)	0.0485	0.9959	0.0258
SMR (Hinge)	0.5906	0.7686	0.5826
MR (Lifted)	0.6658	0.9882	0.2566
MR (Hinge)	0.8094	0.9532	0.3475

TABLE 3.20. High overlap parameter distance ($d = 5$)

Method	$\ w - w_{\text{global}}\ _2$	Cosine Similarity	$ b - b_{\text{global}} $
SMR (Lifted)	0.0699	0.9877	0.0269
SMR (Hinge)	0.5872	0.8240	0.5151
MR (Lifted)	0.6641	0.9890	0.2272
MR (Hinge)	0.8059	0.9491	0.3190

TABLE 3.21. High overlap parameter distance ($d = 7$)

Method	$\ w - w_{\text{global}}\ _2$	Cosine Similarity	$ b - b_{\text{global}} $
SMR (Lifted)	0.0714	0.9944	0.0268
SMR (Hinge)	0.6229	0.8237	0.5438
MR (Lifted)	0.6632	0.9883	0.2456
MR (Hinge)	0.8312	0.9483	0.3525

Predictive accuracy alone does not determine whether a distributed method has recovered the centralized optimization solution, since substantially different models may obtain similar classification accuracy. Tables 3.14 – 3.21 therefore compare the learned parameters directly with the global pooled lifted SVM solution.

These results provide the clearest evidence for the intended role of lifted SMR. Across every dimension and each of the low/high overlap regimes, lifted SMR has by far the smallest Euclidean parameter discrepancy and bias discrepancy from the global lifted solution. Perhaps more importantly, the cosine similarity demonstrates that both the magnitude and direction of the learned lifted SMR models end up close to the centralized one. Lifted MR often obtains similar predictive accuracy, but its parameters remain substantially farther from the centralized minimizer (in addition to local lifted MR models not agreeing as strongly as lifted SMR models do).

This observation helps reconcile the earlier communication experiments. Although lifted MR may occasionally equal or exceed lifted SMR in predictive accuracy, particularly in nearly linear settings or under very infrequent communication, the parameter-distance results show that MR is generally approximating a different solution than the centralized objective. Lifted SMR is the method that most consistently reproduces the centralized optimizer itself.

This distinction is consistent with the constructions of the two representatives. MR approximates the local data whereas SMR is explicitly designed to reproduce

the local first-order information. Strong classification performance from MR does not imply that it is approximating the global objective as closely as SMR.

3. Theoretical Analysis

Here we will give results regarding SMR. We first analyze the discrepancy between full data gradient and the gradient on a set of fixed representatives. We formulate our discrepancy around the heterogeneity of local datasets and show this mismatch improves on the order of the heterogeneity.

LEMMA 3.5 (SMR Covariance Scale Bound). *Consider a collection of observations*

$$\mathcal{D} = \{x_i\}_{i=1}^n, \quad \delta = \max_{1 \leq i, j \leq n} \|x_i - x_j\|,$$

with class label $c \in \{-1, 1\}$. Let Σ be symmetric, positive definite, and let (w_0, b_0) denote the fixed parameter pair at which the representative is constructed.

Define the construction-point quantities

$$\begin{aligned} s_0 &= \sqrt{w_0^\top \Sigma w_0}, \\ \nu_i^0 &= c(w_0^\top x_i + b_0), \\ a_i^0 &= \frac{1 - \nu_i^0}{s_0}. \end{aligned}$$

Let the representative covariance have the proportional form

$$\tilde{\Sigma} = \lambda \Sigma, \quad \lambda > 0,$$

and define

$$\begin{aligned}\tilde{s}_0 &= \sqrt{w_0^\top \tilde{\Sigma} w_0}, \\ \tilde{\nu}_0 &= c(w_0^\top \tilde{x} + b_0), \\ \tilde{a}_0 &= \frac{1 - \tilde{\nu}_0}{\tilde{s}_0}.\end{aligned}$$

Assume that the representative $(\tilde{x}, \tilde{\Sigma})$ is constructed so that the bias and weight score-matching equations hold at (w_0, b_0) :

$$(8) \quad \frac{1}{n} \sum_{i=1}^n \Phi(a_i^0) = \Phi(\tilde{a}_0),$$

$$(9) \quad \frac{1}{n} \sum_{i=1}^n \left[-\Phi(a_i^0) c x_i + \phi(a_i^0) \frac{\Sigma w_0}{s_0} \right] = -\Phi(\tilde{a}_0) c \tilde{x} + \phi(\tilde{a}_0) \frac{\tilde{\Sigma} w_0}{\tilde{s}_0}.$$

Equivalently, projecting (9) onto w_0 gives

$$(10) \quad -\Phi(\tilde{a}_0) c w_0^\top \tilde{x} + \phi(\tilde{a}_0) \tilde{s}_0 = \frac{1}{n} \sum_{i=1}^n \left[-\Phi(a_i^0) c w_0^\top x_i + \phi(a_i^0) s_0 \right].$$

Let

$$U_r = \{(w, b) : \|(w, b) - (w_0, b_0)\| \leq r\}$$

be a compact neighborhood of the construction point. Suppose there exist constants $M_w, M_b, \epsilon_s > 0$ such that, for every $(w, b) \in U_r$,

$$(11) \quad \|w\| \leq M_w, \quad |b| \leq M_b, \quad \sqrt{w^\top \Sigma w} \geq \epsilon_s.$$

Define

$$\begin{aligned}M_x &= \max_{1 \leq i \leq n} \|x_i\|, \\ A_r &= \frac{1 + M_w M_x + M_b}{\epsilon_s}, \\ I_r &= [-A_r, A_r].\end{aligned}$$

Then

$$a_i^0, \bar{a}_0, \tilde{a}_0 \in I_r, \quad \bar{a}_0 = \frac{1}{n} \sum_{i=1}^n a_i^0.$$

Consequently, the constants

$$m_{\phi,r} = \inf_{a \in I_r} \phi(a) > 0,$$

$$h_{\min,r} = \inf_{a \in I_r} h(a) > 0,$$

are well defined, where

$$h(a) = a\Phi(a) + \phi(a).$$

The covariance proportionality constant satisfies

$$(12) \quad \sqrt{\lambda} = \frac{\frac{1}{n} \sum_{i=1}^n h(a_i^0)}{h(\tilde{a}_0)},$$

and

$$(13) \quad \left| \sqrt{\lambda} - 1 \right| \leq \frac{M_w}{h_{\min,r} \epsilon_s} \left(1 + \frac{L_\Phi}{m_{\phi,r}} \right) \delta, \quad L_\Phi = \sup_{a \in \mathbb{R}} \phi(a) = \frac{1}{\sqrt{2\pi}}.$$

Therefore,

$$\left| \sqrt{\lambda} - 1 \right| = \mathcal{O}(\delta).$$

Moreover, for δ sufficiently small,

$$|\lambda - 1| = \mathcal{O}(\delta).$$

PROOF. Since

$$\tilde{\Sigma} = \lambda \Sigma,$$

we have

$$\begin{aligned}
\tilde{s}_0 &= \sqrt{w_0^\top \tilde{\Sigma} w_0} \\
&= \sqrt{\lambda w_0^\top \Sigma w_0} \\
&= \sqrt{\lambda} s_0.
\end{aligned}$$

We first use the score-matching equations at (w_0, b_0) to simplify the projected score-matching equations and obtain a convenient identity for $\tilde{s}_0/s_0$.

From the definitions of a_i^0 and $\tilde{a}_0$,

$$\tilde{\nu}_0 = c(w_0^\top \tilde{x} + b_0) = 1 - \tilde{a}_0 \tilde{s}_0$$

and

$$\nu_i^0 = c(w_0^\top x_i + b_0) = 1 - a_i^0 s_0.$$

Therefore,

$$\begin{aligned}
cw_0^\top \tilde{x} &= 1 - cb_0 - \tilde{a}_0 \tilde{s}_0, \\
cw_0^\top x_i &= 1 - cb_0 - a_i^0 s_0.
\end{aligned}$$

Substituting these expressions into the projected score-matching equation (10) gives

$$\begin{aligned}
& -\Phi(\tilde{a}_0)(1 - cb_0 - \tilde{a}_0 \tilde{s}_0) + \phi(\tilde{a}_0) \tilde{s}_0 \\
&= \frac{1}{n} \sum_{i=1}^n [-\Phi(a_i^0)(1 - cb_0 - a_i^0 s_0) + \phi(a_i^0) s_0].
\end{aligned}$$

Rearranging,

$$\begin{aligned}
& -\Phi(\tilde{a}_0)(1 - cb_0) + [\tilde{a}_0 \Phi(\tilde{a}_0) + \phi(\tilde{a}_0)] \tilde{s}_0 \\
&= -\left(\frac{1}{n} \sum_{i=1}^n \Phi(a_i^0) \right) (1 - cb_0) + s_0 \frac{1}{n} \sum_{i=1}^n [a_i^0 \Phi(a_i^0) + \phi(a_i^0)].
\end{aligned}$$

Applying the bias-matching equation (8), the terms involving $(1 - cb_0)$ cancel. Hence,

$$[\tilde{a}_0\Phi(\tilde{a}_0) + \phi(\tilde{a}_0)]\tilde{s}_0 = s_0 \frac{1}{n} \sum_{i=1}^n [a_i^0\Phi(a_i^0) + \phi(a_i^0)].$$

Using

$$h(a) = a\Phi(a) + \phi(a),$$

this becomes

$$h(\tilde{a}_0)\tilde{s}_0 = s_0 \frac{1}{n} \sum_{i=1}^n h(a_i^0).$$

Dividing by $s_0 h(\tilde{a}_0)$ gives

$$\frac{\tilde{s}_0}{s_0} = \frac{\frac{1}{n} \sum_{i=1}^n h(a_i^0)}{h(\tilde{a}_0)}.$$

Since $\tilde{s}_0/s_0 = \sqrt{\lambda}$, we obtain

$$\sqrt{\lambda} = \frac{\frac{1}{n} \sum_{i=1}^n h(a_i^0)}{h(\tilde{a}_0)}.$$

We next show that the compact interval required in the estimates follows from the local parameter bounds. For every i ,

$$\begin{aligned} |a_i^0| &= \frac{|1 - c(w_0^\top x_i + b_0)|}{s_0} \\ &\leq \frac{1 + \|w_0\| \|x_i\| + |b_0|}{s_0} \\ &\leq \frac{1 + M_w M_x + M_b}{\epsilon_s} \\ &= A_r. \end{aligned}$$

Thus,

$$a_i^0 \in I_r$$

for every i , and consequently

$$\bar{a}_0 = \frac{1}{n} \sum_{i=1}^n a_i^0 \in I_r.$$

Furthermore, by the bias-matching equation,

$$\Phi(\tilde{a}_0) = \frac{1}{n} \sum_{i=1}^n \Phi(a_i^0).$$

Since every $a_i^0 \in [-A_r, A_r]$ and Φ is increasing,

$$\Phi(-A_r) \leq \frac{1}{n} \sum_{i=1}^n \Phi(a_i^0) \leq \Phi(A_r).$$

Therefore,

$$\Phi(-A_r) \leq \Phi(\tilde{a}_0) \leq \Phi(A_r).$$

Since Φ is strictly increasing,

$$\tilde{a}_0 \in [-A_r, A_r] = I_r.$$

It follows that

$$m_{\phi,r} = \inf_{a \in I_r} \phi(a) > 0.$$

Moreover,

$$h'(a) = \Phi(a),$$

and $h(a) > 0$ for every finite a . Hence, by continuity of h on the compact interval I_r ,

$$h_{\min,r} = \inf_{a \in I_r} h(a) > 0.$$

Define

$$\bar{h}_0 = \frac{1}{n} \sum_{i=1}^n h(a_i^0).$$

Then

$$\sqrt{\lambda} - 1 = \frac{\bar{h}_0 - h(\tilde{a}_0)}{h(\tilde{a}_0)}.$$

By the triangle inequality,

$$(14) \quad \left| \sqrt{\lambda} - 1 \right| \leq \frac{|\bar{h}_0 - h(\bar{a}_0)| + |h(\bar{a}_0) - h(\tilde{a}_0)|}{h(\tilde{a}_0)}.$$

Since

$$h'(a) = \Phi(a),$$

and $0 \leq \Phi(a) \leq 1$, the function h is 1-Lipschitz. Therefore,

$$\begin{aligned} |\bar{h}_0 - h(\bar{a}_0)| &= \left| \frac{1}{n} \sum_{i=1}^n h(a_i^0) - h(\bar{a}_0) \right| \\ &\leq \frac{1}{n} \sum_{i=1}^n |h(a_i^0) - h(\bar{a}_0)| \\ &\leq \frac{1}{n} \sum_{i=1}^n |a_i^0 - \bar{a}_0|. \end{aligned}$$

For any i, j ,

$$\begin{aligned} a_i^0 - a_j^0 &= \frac{1 - c(w_0^\top x_i + b_0)}{s_0} - \frac{1 - c(w_0^\top x_j + b_0)}{s_0} \\ &= -\frac{cw_0^\top (x_i - x_j)}{s_0}. \end{aligned}$$

Hence,

$$\begin{aligned} |a_i^0 - a_j^0| &\leq \frac{\|w_0\|}{s_0} \|x_i - x_j\| \\ &\leq \frac{M_w}{\epsilon_s} \delta. \end{aligned}$$

Since

$$a_i^0 - \bar{a}_0 = \frac{1}{n} \sum_{j=1}^n (a_i^0 - a_j^0),$$

it follows that

$$|a_i^0 - \bar{a}_0| \leq \frac{M_w}{\epsilon_s} \delta.$$

Therefore,

$$(15) \quad |\bar{h}_0 - h(\bar{a}_0)| \leq \frac{M_w}{\epsilon_s} \delta.$$

We next bound

$$|\tilde{a}_0 - \bar{a}_0|.$$

By the bias-matching equation,

$$\Phi(\tilde{a}_0) = \frac{1}{n} \sum_{i=1}^n \Phi(a_i^0).$$

Thus,

$$\begin{aligned} |\Phi(\tilde{a}_0) - \Phi(\bar{a}_0)| &= \left| \frac{1}{n} \sum_{i=1}^n \Phi(a_i^0) - \Phi(\bar{a}_0) \right| \\ &\leq \frac{1}{n} \sum_{i=1}^n |\Phi(a_i^0) - \Phi(\bar{a}_0)|. \end{aligned}$$

Since Φ is L_Φ -Lipschitz, where

$$L_\Phi = \sup_{a \in \mathbb{R}} \phi(a) = \frac{1}{\sqrt{2\pi}},$$

we obtain

$$\begin{aligned} |\Phi(\tilde{a}_0) - \Phi(\bar{a}_0)| &\leq L_\Phi \frac{1}{n} \sum_{i=1}^n |a_i^0 - \bar{a}_0| \\ &\leq L_\Phi \frac{M_w}{\epsilon_s} \delta. \end{aligned}$$

Since $\tilde{a}_0, \bar{a}_0 \in I_r$, the mean value theorem gives some $\xi \in I_r$ such that

$$|\Phi(\tilde{a}_0) - \Phi(\bar{a}_0)| = \phi(\xi) |\tilde{a}_0 - \bar{a}_0|.$$

Because $\phi(\xi) \geq m_{\phi,r}$,

$$(16) \quad |\tilde{a}_0 - \bar{a}_0| \leq \frac{L_\Phi}{m_{\phi,r}} \frac{M_w}{\epsilon_s} \delta.$$

Applying the 1-Lipschitz property of h ,

$$(17) \quad |h(\bar{a}_0) - h(\tilde{a}_0)| \leq \frac{L_\Phi}{m_{\phi,r}} \frac{M_w}{\epsilon_s} \delta.$$

Finally, since $\tilde{a}_0 \in I_r$,

$$h(\tilde{a}_0) \geq h_{\min,r}.$$

Substituting (15) and (17) into (14) gives

$$\left| \sqrt{\lambda} - 1 \right| \leq \frac{M_w}{h_{\min,r} \epsilon_s} \left(1 + \frac{L_\Phi}{m_{\phi,r}} \right) \delta.$$

Hence,

$$\left| \sqrt{\lambda} - 1 \right| = \mathcal{O}(\delta).$$

In particular, for δ sufficiently small,

$$\left| \sqrt{\lambda} - 1 \right| \leq 1,$$

so that

$$0 \leq \sqrt{\lambda} \leq 2$$

and therefore

$$\sqrt{\lambda} + 1 \leq 3.$$

Since

$$|\lambda - 1| = \left| \sqrt{\lambda} - 1 \right| \left(\sqrt{\lambda} + 1 \right),$$

we conclude that

$$\begin{aligned} |\lambda - 1| &\leq 3 \left| \sqrt{\lambda} - 1 \right| \\ &\leq 3 \frac{M_w}{h_{\min, r} \epsilon_s} \left(1 + \frac{L_\Phi}{m_{\phi, r}} \right) \delta. \end{aligned}$$

Thus,

$$|\lambda - 1| = \mathcal{O}(\delta).$$

□

It is important to note here that we do not emphasize the asymptotic properties of this relationship. We show that under reasonable circumstances, the proportionality coefficient $\sqrt{\lambda}$ is bounded to first order heterogeneity in the data (determined by class diameter) away from 1. Sending $\delta \rightarrow 0$ directly will violate assumptions without stabilization terms on Σ and s .

For a general empirical covariance matrix we have

$$\left\| \hat{\Sigma} \right\| \leq C\delta^2$$

In simulations, we prefer to use an added ridge term:

$$\Sigma = \hat{\Sigma} + \eta I$$

to ensure stability. Therefore, the above lemma should be interpreted as a local stability result without degeneracy.

With SMR construction bounded for a single group being represented, we look to bound the gradient discrepancy between the full distributed SMR construction ($\tilde{\mathcal{D}}$) and the full data ($\mathcal{D}$).

THEOREM 3.6 (SMR Gradient Discrepancy with Full Data). *For each node k and class c , define*

$$\mathcal{D}_{k,c} = \{x_i\}_{i \in \mathcal{I}_{k,c}},$$

$$\delta_{k,c} := \max_{i,j \in \mathcal{I}_{k,c}} \|x_i - x_j\|,$$

$$\Delta := \max_{k,c} \delta_{k,c}.$$

Assume that the hypotheses of Lemma 3.5 hold for every group (k,c) , with $\Sigma_{k,c}$ symmetric, positive definite and with constants chosen uniformly over all groups (k,c) . In particular, there exists a constant C_λ , which may be chosen over all groups, such that

$$\left| \sqrt{\lambda_{k,c}} - 1 \right| \leq C_\lambda \delta_{k,c}.$$

Let $\bar{x}_{k,c}$ denote the empirical mean of $\mathcal{D}_{k,c}$.

For each group (k,c) , fix a construction point

$$\theta_{0,k} = (w_{0,k}, b_{0,k}),$$

and construct a representative

$$\tilde{\mathcal{D}}_{k,c} = (\tilde{x}_{k,c}, \tilde{\Sigma}_{k,c})$$

satisfying the SMR bias- and weight-matching equations at $\theta_{0,k}$. Assume

$$\tilde{\Sigma}_{k,c} = \lambda_{k,c} \Sigma_{k,c}, \quad \lambda_{k,c} > 0,$$

For some $r > 0$, let

$$U_{k,r} = \{(w, b) : \|(w, b) - (w_{0,k}, b_{0,k})\| \leq r\}.$$

Assume that the common neighborhood

$$U_r := \bigcap_k U_{k,r}$$

is nonempty.

Assume that there exist constants

$$M_w, M_b, M_x, M_\Sigma, \epsilon_s > 0$$

such that, uniformly over all groups and all $(w, b) \in U_{k,r}$,

$$\|w\| \leq M_w,$$

$$|b| \leq M_b,$$

$$\|x_i\| \leq M_x,$$

$$\|\Sigma_{k,c}\| \leq M_\Sigma,$$

$$s_{k,c}(w) = \sqrt{w^\top \Sigma_{k,c} w} \geq \epsilon_s,$$

$$\tilde{s}_{k,c}(w) = \sqrt{w^\top \tilde{\Sigma}_{k,c} w} \geq \epsilon_s.$$

Assume also that the corresponding quantities a_i and $\tilde{a}_{k,c}$ remain in a fixed compact interval I , independent of Δ and of the choice of group, on these neighborhoods.

Define the full-data and frozen-representative objectives by

$$\begin{aligned}\mathcal{L}(w, b) &= \frac{1}{2} \|w\|^2 + C \sum_{k,c} \alpha_{k,c} \ell(\mathcal{D}_{k,c}; w, b), \\ \tilde{\mathcal{L}}(w, b) &= \frac{1}{2} \|w\|^2 + C \sum_{k,c} \alpha_{k,c} \ell(\tilde{\mathcal{D}}_{k,c}; w, b),\end{aligned}$$

where the representatives are held fixed after their construction. Assume that

$$\alpha_{k,c} \geq 0, \quad \sum_{k,c} \alpha_{k,c} = 1.$$

Then there exists a constant $K > 0$, independent of Δ , such that

$$\sup_{(w,b) \in U_r} \left\| \nabla_{(w,b)} \mathcal{L}(w, b) - \nabla_{(w,b)} \tilde{\mathcal{L}}(w, b) \right\| \leq K \Delta.$$

PROOF. Fix a group (k, c) , corresponding to node k and class c , and suppress the group indices. Write

$$\begin{aligned}\delta &= \max_{i,j} \|x_i - x_j\|, \\ \bar{x} &= \frac{1}{n} \sum_{i=1}^n x_i, \\ \tilde{\Sigma} &= \lambda \Sigma, \quad \lambda > 0.\end{aligned}$$

The representative $(\tilde{x}, \tilde{\Sigma})$ is constructed at the fixed point

$$\theta_0 = (w_0, b_0).$$

Throughout the proof, constants denoted by $C_\bullet$ are positive, independent of δ , and may depend only on the uniform constants in the assumptions and on the fixed compact interval I . Their values may change from one estimate to the next when no ambiguity is created.

The proof has two stages. First, the score-matching equations at θ_0 are used to obtain estimates for the fixed representative construction. Second, these estimates are used to compare the full-data and representative gradients at an arbitrary $(w, b) \in U_{k,r}$.

Structural estimates at the construction point.

Define

$$\begin{aligned} s_0 &= \sqrt{w_0^\top \Sigma w_0}, \\ \tilde{s}_0 &= \sqrt{w_0^\top \tilde{\Sigma} w_0}, \\ a_i^0 &= \frac{1 - c(w_0^\top x_i + b_0)}{s_0}, \\ \tilde{a}_0 &= \frac{1 - c(w_0^\top \tilde{x} + b_0)}{\tilde{s}_0}. \end{aligned}$$

Since $\tilde{\Sigma} = \lambda \Sigma$, we have

$$\tilde{s}_0 = \sqrt{w_0^\top (\lambda \Sigma) w_0} = \sqrt{\lambda} s_0,$$

and hence

$$\frac{\tilde{\Sigma} w_0}{\tilde{s}_0} = \sqrt{\lambda} \frac{\Sigma w_0}{s_0}.$$

Using the weight-matching equation and the bias-matching equation at θ_0 , and then subtracting the empirical mean contribution, gives

$$\begin{aligned} \Phi(\tilde{a}_0)(\tilde{x} - \bar{x}) &= \frac{1}{n} \sum_{i=1}^n \Phi(a_i^0)(x_i - \bar{x}) \\ (18) \quad &+ c \left[\phi(\tilde{a}_0) \frac{\tilde{\Sigma} w_0}{\tilde{s}_0} - \left(\frac{1}{n} \sum_{i=1}^n \phi(a_i^0) \right) \frac{\Sigma w_0}{s_0} \right]. \end{aligned}$$

The factor $c \in \{-1, 1\}$ does not affect any subsequent norm bound. We now estimate the two terms on the right-hand side separately.

Because $\bar{x}$ belongs to the convex hull of the points $x_1, \dots, x_n$, for each i ,

$$\begin{aligned}\|x_i - \bar{x}\| &= \left\| \frac{1}{n} \sum_{j=1}^n (x_i - x_j) \right\| \\ &\leq \frac{1}{n} \sum_{j=1}^n \|x_i - x_j\| \leq \delta.\end{aligned}$$

Since $0 \leq \Phi \leq 1$, it follows that

$$(19) \quad \left\| \frac{1}{n} \sum_{i=1}^n \Phi(a_i^0)(x_i - \bar{x}) \right\| \leq \frac{1}{n} \sum_{i=1}^n \Phi(a_i^0) \|x_i - \bar{x}\|$$

$$(20) \quad \leq \frac{1}{n} \sum_{i=1}^n \delta = \delta.$$

By Lemma 3.5,

$$(21) \quad \left| \sqrt{\lambda} - 1 \right| \leq C_\lambda \delta.$$

Inside of this lemma, we bounded:

$$(22) \quad |\tilde{a}_0 - \bar{a}_0| \leq C_a \delta, \quad \bar{a}_0 = \frac{1}{n} \sum_{i=1}^n a_i^0.$$

Additionally, we show that a_i^0 's cluster around $\bar{a}_0$. For any i and j ,

$$\begin{aligned}|a_i^0 - a_j^0| &= \frac{1}{s_0} |cw_0^\top (x_j - x_i)| \\ &\leq \frac{\|w_0\|}{s_0} \|x_i - x_j\| \\ &\leq \frac{M_w}{\epsilon_s} \delta.\end{aligned}$$

Therefore,

$$\begin{aligned}
|a_i^0 - \bar{a}_0| &= \left| \frac{1}{n} \sum_{j=1}^n (a_i^0 - a_j^0) \right| \\
&\leq \frac{1}{n} \sum_{j=1}^n |a_i^0 - a_j^0| \\
&\leq \frac{M_w}{\epsilon_s} \delta.
\end{aligned}$$

Averaging this estimate over i yields

$$(23) \quad \frac{1}{n} \sum_{i=1}^n |a_i^0 - \bar{a}_0| \leq C'_a \delta, \quad C'_a := \frac{M_w}{\epsilon_s}.$$

Because ϕ is continuously differentiable, it is Lipschitz on the compact interval

I . Let

$$L_\phi := \sup_{a \in I} |\phi'(a)| < \infty.$$

Insert and subtract $\phi(\bar{a}_0)$ and use the triangle inequality:

$$\begin{aligned}
&\left| \phi(\tilde{a}_0) - \frac{1}{n} \sum_{i=1}^n \phi(a_i^0) \right| \\
&\leq |\phi(\tilde{a}_0) - \phi(\bar{a}_0)| + \left| \phi(\bar{a}_0) - \frac{1}{n} \sum_{i=1}^n \phi(a_i^0) \right| \\
&= |\phi(\tilde{a}_0) - \phi(\bar{a}_0)| + \left| \frac{1}{n} \sum_{i=1}^n (\phi(\bar{a}_0) - \phi(a_i^0)) \right| \\
&\leq |\phi(\tilde{a}_0) - \phi(\bar{a}_0)| + \frac{1}{n} \sum_{i=1}^n |\phi(a_i^0) - \phi(\bar{a}_0)| \\
&\leq L_\phi |\tilde{a}_0 - \bar{a}_0| + \frac{L_\phi}{n} \sum_{i=1}^n |a_i^0 - \bar{a}_0| \\
&\leq L_\phi (C_a + C'_a) \delta.
\end{aligned}$$

Thus, with

$$C_\phi := L_\phi(C_a + C'_a),$$

we obtain

$$(24) \quad \left| \phi(\tilde{a}_0) - \frac{1}{n} \sum_{i=1}^n \phi(a_i^0) \right| \leq C_\phi \delta.$$

We next incorporate the covariance scaling. Add and subtract $\phi(\tilde{a}_0)$ to write

$$\begin{aligned} & \left| \phi(\tilde{a}_0) \sqrt{\lambda} - \frac{1}{n} \sum_{i=1}^n \phi(a_i^0) \right| \\ &= \left| \phi(\tilde{a}_0)(\sqrt{\lambda} - 1) + \phi(\tilde{a}_0) - \frac{1}{n} \sum_{i=1}^n \phi(a_i^0) \right| \\ &\leq \phi(\tilde{a}_0) \left| \sqrt{\lambda} - 1 \right| + \left| \phi(\tilde{a}_0) - \frac{1}{n} \sum_{i=1}^n \phi(a_i^0) \right|. \end{aligned}$$

Since the standard normal density is uniformly bounded by $M_\phi := \sup_{a \in I} \phi(a) < \infty$, equations (21) and (24) give

$$(25) \quad \left| \phi(\tilde{a}_0) \sqrt{\lambda} - \frac{1}{n} \sum_{i=1}^n \phi(a_i^0) \right| \leq (M_\phi C_\lambda + C_\phi) \delta =: C_{\phi, \lambda} \delta.$$

Moreover,

$$(26) \quad \left\| \frac{\Sigma w_0}{s_0} \right\| \leq \frac{\|\Sigma\| \|w_0\|}{s_0} \leq \frac{M_\Sigma M_w}{\epsilon_s}.$$

Using

$$\frac{\tilde{\Sigma} w_0}{\tilde{s}_0} = \sqrt{\lambda} \frac{\Sigma w_0}{s_0},$$

the covariance term in (18) can be factored as

$$\begin{aligned} \phi(\tilde{a}_0) \frac{\tilde{\Sigma} w_0}{\tilde{s}_0} - \left(\frac{1}{n} \sum_{i=1}^n \phi(a_i^0) \right) \frac{\Sigma w_0}{s_0} \\ = \left[\phi(\tilde{a}_0) \sqrt{\lambda} - \frac{1}{n} \sum_{i=1}^n \phi(a_i^0) \right] \frac{\Sigma w_0}{s_0}. \end{aligned}$$

Consequently, by (25) and (26), its norm is at most

$$C_{\phi, \lambda} \frac{M_{\Sigma} M_w}{\epsilon_s} \delta.$$

Because $\tilde{a}_0 \in I$ and I is compact,

$$\Phi(\tilde{a}_0) \geq m_{\Phi} := \inf_{a \in I} \Phi(a) > 0.$$

Taking norms in (18), applying (20), and using the preceding covariance estimate therefore gives

$$m_{\Phi} \|\tilde{x} - \bar{x}\| \leq \delta + C_{\phi, \lambda} \frac{M_{\Sigma} M_w}{\epsilon_s} \delta.$$

Hence

$$(27) \quad \|\tilde{x} - \bar{x}\| \leq \frac{1}{m_{\Phi}} \left(1 + C_{\phi, \lambda} \frac{M_{\Sigma} M_w}{\epsilon_s} \right) \delta =: C_x \delta.$$

It follows that, for every i ,

$$\begin{aligned} (28) \quad \|x_i - \tilde{x}\| &\leq \|x_i - \bar{x}\| + \|\bar{x} - \tilde{x}\| \\ &\leq (1 + C_x) \delta. \end{aligned}$$

These are structural properties with respect to the fixed (or we may say “frozen”) representatives. This terminology is used to separate from the case of iterative reconstructions at future weights. Thus, they remain valid when the objective is evaluated at parameters different from θ_0 .

Comparison at an arbitrary nearby parameter.

Let

$$\theta = (w, b) \in U_{k,r}$$

be arbitrary, and define

$$s = \sqrt{w^\top \Sigma w},$$

$$\tilde{s} = \sqrt{w^\top \tilde{\Sigma} w},$$

$$\nu_i = c(w^\top x_i + b),$$

$$\tilde{\nu} = c(w^\top \tilde{x} + b),$$

$$a_i = \frac{1 - \nu_i}{s},$$

$$\tilde{a} = \frac{1 - \tilde{\nu}}{\tilde{s}}.$$

Since the representative is frozen after construction, the score-matching equations are not assumed at this θ . Instead, only the structural components are considered to obtain a bound using δ .

$$\text{Since } \tilde{\Sigma} = \lambda \Sigma,$$

$$\tilde{s} = \sqrt{\lambda} s$$

for every w . Therefore,

$$\begin{aligned} |s - \tilde{s}| &= \left| 1 - \sqrt{\lambda} \right| s \\ &= \left| 1 - \sqrt{\lambda} \right| \sqrt{w^\top \Sigma w} \\ &\leq C_\lambda \delta \sqrt{\|\Sigma\|} \|w\| \\ &\leq C_\lambda \sqrt{M_\Sigma} M_w \delta =: C_s \delta. \end{aligned}$$

Here we used $w^\top \Sigma w \leq \|\Sigma\| \|w\|^2$.

Similarly, equation (28) gives

$$\begin{aligned}
(29) \quad |\nu_i - \tilde{\nu}| &= |cw^\top(x_i - \tilde{x})| \\
&\leq \|w\| \|x_i - \tilde{x}\| \\
(30) \quad &\leq M_w(1 + C_x)\delta =: C_{\nu,1}\delta.
\end{aligned}$$

We also require a uniform bound on $1 - \tilde{\nu}$. Since $\|\bar{x}\| \leq n^{-1} \sum_i \|x_i\| \leq M_x$, equation (27) gives

$$\begin{aligned}
\|\tilde{x}\| &\leq \|\tilde{x} - \bar{x}\| + \|\bar{x}\| \\
&\leq C_x\delta + M_x.
\end{aligned}$$

Moreover, the assumed bound $\|x_i\| \leq M_x$ implies $\delta \leq 2M_x$. Thus

$$\|\tilde{x}\| \leq M_x + 2C_xM_x =: M_{\tilde{x}},$$

where $M_{\tilde{x}}$ is independent of δ . It follows that

$$\begin{aligned}
|1 - \tilde{\nu}| &= |1 - c(w^\top \tilde{x} + b)| \\
&\leq 1 + \|w\| \|\tilde{x}\| + |b| \\
(31) \quad &\leq 1 + M_w M_{\tilde{x}} + M_b =: C_\nu.
\end{aligned}$$

We now compare the standardized margins. Add and subtract $(1 - \tilde{\nu})/s$ to obtain

$$\begin{aligned}
|a_i - \tilde{a}| &= \left| \frac{1 - \nu_i}{s} - \frac{1 - \tilde{\nu}}{\tilde{s}} \right| \\
&\leq \left| \frac{1 - \nu_i}{s} - \frac{1 - \tilde{\nu}}{s} \right| + \left| \frac{1 - \tilde{\nu}}{s} - \frac{1 - \tilde{\nu}}{\tilde{s}} \right| \\
&= \frac{|\nu_i - \tilde{\nu}|}{s} + |1 - \tilde{\nu}| \left| \frac{1}{s} - \frac{1}{\tilde{s}} \right| \\
&= \frac{|\nu_i - \tilde{\nu}|}{s} + |1 - \tilde{\nu}| \frac{|s - \tilde{s}|}{s\tilde{s}}.
\end{aligned}$$

Using $s, \tilde{s} \geq \epsilon_s$ together with (29), (30), and (31), we obtain

$$\begin{aligned} |a_i - \tilde{a}| &\leq \frac{C_{\nu,1}}{\epsilon_s} \delta + C_\nu \frac{C_s}{\epsilon_s^2} \delta \\ &=: C_a^* \delta. \end{aligned}$$

Therefore,

$$(32) \quad |a_i - \tilde{a}| \leq C_a^* \delta$$

uniformly over i and $(w, b) \in U_{k,r}$.

Bias-gradient estimate.

The bias derivative of the averaged group loss is

$$\partial_b \ell(\mathcal{D}; w, b) = -\frac{c}{n} \sum_{i=1}^n \Phi(a_i),$$

whereas the representative bias derivative is

$$\partial_b \ell(\tilde{\mathcal{D}}; w, b) = -c \Phi(\tilde{a}).$$

Since $\Phi'(a) = \phi(a)$ and ϕ is globally bounded, Φ is globally Lipschitz. Let

$$L_\Phi := \sup_{a \in \mathbb{R}} \phi(a).$$

Then

$$\begin{aligned}
& \left| \partial_b \ell(\mathcal{D}; w, b) - \partial_b \ell(\tilde{\mathcal{D}}; w, b) \right| \\
&= \left| \frac{c}{n} \sum_{i=1}^n (\Phi(\tilde{a}) - \Phi(a_i)) \right| \\
&\leq \frac{1}{n} \sum_{i=1}^n |\Phi(a_i) - \Phi(\tilde{a})| \\
&\leq \frac{L_\Phi}{n} \sum_{i=1}^n |a_i - \tilde{a}| \\
&\leq L_\Phi C_a^* \delta =: C_b \delta.
\end{aligned}$$

Weight-gradient estimate.

The weight derivative of the averaged full-data group loss is

$$\nabla_w \ell(\mathcal{D}; w, b) = \frac{1}{n} \sum_{i=1}^n \left[-c \Phi(a_i) x_i + \phi(a_i) \frac{\Sigma w}{s} \right],$$

whereas

$$\nabla_w \ell(\tilde{\mathcal{D}}; w, b) = -c \Phi(\tilde{a}) \tilde{x} + \phi(\tilde{a}) \frac{\tilde{\Sigma} w}{\tilde{s}}.$$

We estimate the location and covariance components separately.

For the location component, add and subtract $\Phi(\tilde{a}) x_i$:

$$\Phi(a_i) x_i - \Phi(\tilde{a}) \tilde{x} = (\Phi(a_i) - \Phi(\tilde{a})) x_i + \Phi(\tilde{a}) (x_i - \tilde{x}).$$

Therefore,

$$\begin{aligned}
\|\Phi(a_i) x_i - \Phi(\tilde{a}) \tilde{x}\| &\leq |\Phi(a_i) - \Phi(\tilde{a})| \|x_i\| + \Phi(\tilde{a}) \|x_i - \tilde{x}\| \\
&\leq L_\Phi M_x |a_i - \tilde{a}| + \|x_i - \tilde{x}\| \\
&\leq [L_\Phi M_x C_a^* + (1 + C_x)] \delta \\
&=: C_{\text{loc}} \delta.
\end{aligned}$$

Here we used $0 \leq \Phi \leq 1$ in the second line. Averaging and using the triangle inequality gives

$$\begin{aligned}
(33) \quad & \left\| \frac{1}{n} \sum_{i=1}^n \Phi(a_i) x_i - \Phi(\tilde{a}) \tilde{x} \right\| \\
&= \left\| \frac{1}{n} \sum_{i=1}^n (\Phi(a_i) x_i - \Phi(\tilde{a}) \tilde{x}) \right\| \\
&\leq \frac{1}{n} \sum_{i=1}^n \|\Phi(a_i) x_i - \Phi(\tilde{a}) \tilde{x}\| \\
&\leq C_{\text{loc}} \delta.
\end{aligned}$$

For the covariance component, add and subtract $\phi(\tilde{a}) \Sigma w / s$:

$$\begin{aligned}
& \left(\frac{1}{n} \sum_{i=1}^n \phi(a_i) \right) \frac{\Sigma w}{s} - \phi(\tilde{a}) \frac{\tilde{\Sigma} w}{\tilde{s}} \\
&= \left[\frac{1}{n} \sum_{i=1}^n \phi(a_i) - \phi(\tilde{a}) \right] \frac{\Sigma w}{s} \\
&\quad + \phi(\tilde{a}) \left(\frac{\Sigma w}{s} - \frac{\tilde{\Sigma} w}{\tilde{s}} \right).
\end{aligned}$$

The first scalar factor satisfies, using the same Lipschitz argument as in (24),

$$(34) \quad \left| \frac{1}{n} \sum_{i=1}^n \phi(a_i) - \phi(\tilde{a}) \right| \leq \frac{1}{n} \sum_{i=1}^n |\phi(a_i) - \phi(\tilde{a})|$$

$$(35) \quad \leq \frac{L_\phi}{n} \sum_{i=1}^n |a_i - \tilde{a}|$$

$$(36) \quad \leq L_\phi C_a^* \delta =: C_\phi^* \delta.$$

Also,

$$(37) \quad \left\| \frac{\Sigma w}{s} \right\| \leq \frac{\|\Sigma\| \|w\|}{s} \leq \frac{M_\Sigma M_w}{\epsilon_s} =: C_{\Sigma w}.$$

Finally,

$$\frac{\tilde{\Sigma}w}{\tilde{s}} = \sqrt{\lambda} \frac{\Sigma w}{s},$$

so

$$(38) \quad \left\| \frac{\tilde{\Sigma}w}{\tilde{s}} - \frac{\Sigma w}{s} \right\| = |\sqrt{\lambda} - 1| \left\| \frac{\Sigma w}{s} \right\|$$

$$(39) \quad \leq C_\lambda C_{\Sigma w} \delta =: C_{\text{cov}} \delta.$$

Since $\phi(\tilde{a}) \leq M_\phi$, the covariance-component estimate becomes

$$\begin{aligned} & \left\| \left(\frac{1}{n} \sum_{i=1}^n \phi(a_i) \right) \frac{\Sigma w}{s} - \phi(\tilde{a}) \frac{\tilde{\Sigma}w}{\tilde{s}} \right\| \\ & \leq \left| \frac{1}{n} \sum_{i=1}^n \phi(a_i) - \phi(\tilde{a}) \right| \left\| \frac{\Sigma w}{s} \right\| \\ & \quad + \phi(\tilde{a}) \left\| \frac{\Sigma w}{s} - \frac{\tilde{\Sigma}w}{\tilde{s}} \right\| \\ & \leq C_\phi^* C_{\Sigma w} \delta + M_\phi C_{\text{cov}} \delta \\ & =: C_{\text{scale}} \delta. \end{aligned}$$

Combining the location and covariance estimates and using the triangle inequality gives

$$\begin{aligned} & \left\| \nabla_w \ell(\mathcal{D}; w, b) - \nabla_w \ell(\tilde{\mathcal{D}}; w, b) \right\| \\ & \leq C_{\text{loc}} \delta + C_{\text{scale}} \delta \\ & =: K_w \delta. \end{aligned}$$

Together with (33), and using $\|(u, v)\| \leq \|u\| + |v|$, we obtain

$$\begin{aligned} & \left\| \nabla_{(w,b)} \ell(\mathcal{D}; w, b) - \nabla_{(w,b)} \ell(\tilde{\mathcal{D}}; w, b) \right\| \\ & \leq K_w \delta + C_b \delta =: K_g \delta. \end{aligned}$$

All constants used above are uniform over the groups and over the corresponding neighborhoods. Therefore,

$$(40) \quad \sup_{(w,b) \in U_{k,r}} \left\| \nabla_{(w,b)} \ell(\mathcal{D}; w, b) - \nabla_{(w,b)} \ell(\tilde{\mathcal{D}}; w, b) \right\| \leq K_g \delta.$$

Restoring the group indices, the gradients of the identical regularization terms cancel. Hence, for $(w, b) \in U_r$,

$$\begin{aligned} & \left\| \nabla_{(w,b)} \mathcal{L}(w, b) - \nabla_{(w,b)} \tilde{\mathcal{L}}(w, b) \right\| \\ & = C \left\| \sum_{k,c} \alpha_{k,c} \left[\nabla \ell(\mathcal{D}_{k,c}; w, b) - \nabla \ell(\tilde{\mathcal{D}}_{k,c}; w, b) \right] \right\| \\ & \leq C \sum_{k,c} \alpha_{k,c} \left\| \nabla \ell(\mathcal{D}_{k,c}; w, b) - \nabla \ell(\tilde{\mathcal{D}}_{k,c}; w, b) \right\| \\ & \leq CK_g \sum_{k,c} \alpha_{k,c} \delta_{k,c} \\ & \leq CK_g \Delta \sum_{k,c} \alpha_{k,c} \\ & = CK_g \Delta. \end{aligned}$$

The last equality uses $\sum_{k,c} \alpha_{k,c} = 1$. Thus, setting

$$K := CK_g,$$

and taking the supremum over $(w, b) \in U_r$ proves

$$\sup_{(w,b) \in U_r} \left\| \nabla_{(w,b)} \mathcal{L}(w, b) - \nabla_{(w,b)} \tilde{\mathcal{L}}(w, b) \right\| \leq K \Delta.$$

□

Using this bound, we provide a simple convergence estimate in the case we have a strongly convex loss. Note that this will only be the case for the full parameter θ when the bias b is also penalized. Otherwise, the strong convergence only holds for w .

COROLLARY 3.7 (Inexact gradient descent for iterative SMR). *Let*

$$F : \mathbb{R}^{d+1} \rightarrow \mathbb{R}$$

denote the full-data objective, and let

$$\theta^* = (w^*, b^*)$$

be its minimizer.

At each outer iteration t , representatives are constructed from the current parameter

$$\theta_t = (w_t, b_t),$$

yielding the representative objective

$$\tilde{F}_t.$$

The server initializes the inner optimization by

$$\theta_{t,0} = \theta_t,$$

and performs J gradient descent steps

$$\theta_{t,j+1} = \theta_{t,j} - \gamma \nabla \tilde{F}_t(\theta_{t,j}), \quad j = 0, \dots, J-1.$$

The updated global parameter is then

$$\theta_{t+1} = \theta_{t,J}.$$

Assume that

- (1) F is μ -strongly convex and L -smooth on a convex neighborhood U ;
- (2) every outer and inner iterate of θ ($\theta_t, \theta_{t,j}$) remains in U ;
- (3) for every outer iteration t , the representatives remain fixed during the corresponding inner optimization, and satisfy

$$\sup_{\theta \in U_{t,r}} \left\| \nabla \tilde{F}_t(\theta) - \nabla F(\theta) \right\| \leq K\Delta,$$

where $U_{t,r} \subseteq U$ contains all inner iterates

$$\theta_{t,0}, \dots, \theta_{t,J};$$

- (4) the constants K and Δ are independent of t ;
- (5) the step size satisfies

$$0 < \gamma \leq \frac{1}{L}.$$

Define

$$\rho = 1 - \gamma\mu.$$

Then, after T outer iterations,

$$\|\theta_T - \theta^\star\| \leq \rho^{JT} \|\theta_0 - \theta^\star\| + \frac{K\Delta}{\mu} (1 - \rho^{JT}).$$

Consequently,

$$\|\theta_T - \theta^\star\| \leq \rho^{JT} \|\theta_0 - \theta^\star\| + \frac{K\Delta}{\mu},$$

and therefore

$$\limsup_{T \rightarrow \infty} \|\theta_T - \theta^\star\| \leq \frac{K\Delta}{\mu}.$$

PROOF. For each outer iteration t and inner iteration j , define the gradient discrepancy

$$e_{t,j} = \nabla \tilde{F}_t(\theta_{t,j}) - \nabla F(\theta_{t,j}).$$

The inner update may therefore be written as

$$\theta_{t,j+1} = \theta_{t,j} - \gamma \nabla F(\theta_{t,j}) - \gamma e_{t,j}.$$

Since F is μ -strongly convex and L -smooth, and

$$0 < \gamma \leq \frac{1}{L},$$

the exact gradient descent map satisfies

$$\|\theta_{t,j} - \gamma \nabla F(\theta_{t,j}) - \theta^*\| \leq \rho \|\theta_{t,j} - \theta^*\|.$$

Using the triangle inequality,

$$\begin{aligned} \|\theta_{t,j+1} - \theta^*\| &= \|\theta_{t,j} - \gamma \nabla F(\theta_{t,j}) - \theta^* - \gamma e_{t,j}\| \\ &\leq \rho \|\theta_{t,j} - \theta^*\| + \gamma \|e_{t,j}\|. \end{aligned}$$

By Assumption (3),

$$\|e_{t,j}\| \leq K\Delta,$$

and therefore

$$\|\theta_{t,j+1} - \theta^*\| \leq \rho \|\theta_{t,j} - \theta^*\| + \gamma K\Delta.$$

Applying this recursion over the J inner gradient steps yields

$$\|\theta_{t+1} - \theta^*\| \leq \rho^J \|\theta_t - \theta^*\| + \frac{K\Delta}{\mu} (1 - \rho^J).$$

Applying this recursion over the T outer iterations gives

$$\|\theta_T - \theta^*\| \leq \rho^{JT} \|\theta_0 - \theta^*\| + \frac{K\Delta}{\mu} (1 - \rho^{JT}),$$

□

It's worth noting that when all groups construct SMR from the same parameter, the first inner gradient step following each representative reconstruction is exact for the full dataset and has no gradient discrepancy error. Therefore, we don't suffer from "reconstruction error" at each round / outer iteration.

Another point is that we didn't use this exact matching property of SMR for the usual gradient descent convergence bound. The second term in the bound for the local iterations may be replaced, giving:

$$\|\theta_{t+1} - \theta^*\| \leq \rho^{JT} \|\theta_0 - \theta^*\| + \frac{K\Delta}{\mu} (1 - \rho^{J-1}),$$

losing one factor of ρ . When $J = 1$ we see this term vanishes.

Lastly, this result gives a standard gradient convergence guarantee under joint strong convexity and smoothness with our inexactness due to representative discrepancy. The lifted (and classical) SVM objective regularizes w but not the intercept b , and therefore does not automatically satisfy joint strong convexity in $\theta = (w, b)$. Accordingly, the theorem applies directly when the intercept is fixed, eliminated, or regularized.

CHAPTER 4

Kernel Extension for DSVM

In this chapter we will discuss how to extend our RepL algorithm for linear SVM to kernel SVM. The goal of this chapter is not to introduce a new loss function, but rather to extend the lifted hinge loss framework from Chapter 3 to nonlinear decision functions. The lifted loss is retained, while the linear predictor η is replaced by a kernelized predictor $f(x)$. Consequently, the primary challenge becomes defining an appropriate analogue of the variance term in the RKHS and constructing representatives that can be communicated between nodes.

As we will show, our loss function does not depend solely on inner products in input space and thus there are freedoms on how to choose to do such an extension.

1. Motivation

For kernel SVM, we usually begin by replacing our data x with a transformed version $\psi(x)$ where $\psi \in \mathcal{H}$ lies in a Hilbert space of functions. For the usual SVM optimization problem, the loss only depends on computing inner products $\langle x, y \rangle$ which gets replaced by a kernel $K(x, y)$. However, for the lifted loss, we have an additional term $s = \sqrt{w^T \Sigma w}$ where Σ is computed based on variance in the input space. Denote by Γ the covariance operator in the Reproducing Kernel Hilbert Space (RKHS) $\mathcal{H}$. Then we may let

$$s^2 = \langle w, \Gamma w \rangle$$

act as the corresponding replacement. This covariance term requires more than simply computing kernel evaluations as we don't typically know the transformation $\psi(x)$.

Observe from the linear case, if $X \sim \mathcal{N}(\mu, \Sigma)$ and $f(X) = w^T X + b$, then we have

$$\text{Var}(f(X)) = \langle w, \Sigma w \rangle = s^2$$

Now, consider again $\psi(X) \in \mathcal{H}$. Define

$$\mu_\psi = \mathbb{E}[\psi(X)]$$

then the covariance operator in $\mathcal{H}$ is defined as

$$\Gamma = \mathbb{E}[(\psi(X) - \mu_\psi) \otimes (\psi(X) - \mu_\psi)]$$

or equivalently by,

$$\langle \Gamma u, v \rangle = \mathbb{E}[\langle \psi(X) - \mu_\psi, u \rangle \langle \psi(X) - \mu_\psi, v \rangle]$$

where $\otimes$ denotes the tensor product defined as $(a \otimes b)(u) = \langle b, u \rangle a$. Then for the kernelized predictor

$$f(X) = \langle w, \psi(X) \rangle + b$$

we have

$$\text{Var}(f(X)) = \langle w, \Gamma w \rangle$$

This gives us the setup for introducing a covariance term into our lifted loss for linear SVM. While to obtain the lifted loss function we replaced our data with Gaussian random variables, here our goal is simply to extend the variance term to a workable version in $\mathcal{H}$. In a similar way that we compute local variance for linear SVM, we may aim to approximate this Γ on each local node to share to other nodes. However, this would lead to each node utilizing a basis of local points that could not be understood by other nodes without communicating raw data. This makes sharing representatives

impossible. Therefore, we conclude our motivation section by stating that we need to approximate Γ with a set of points that all nodes can access.

2. Kernel Lifted SVM

Here we derive the form for our kernel lifted SVM beginning with the idea we left off on in the previous section. We begin by defining a shared dictionary of data that serves the initial purpose of restricting us a finite approximation of the covariance operator and the weight parameter w . This shared dictionary will also serve as a common coordinate system for communicating representatives (or other information) between nodes, which alleviates the need to share all data samples.

Let

$$\mathcal{B} = \{\xi_1, \dots, \xi_m\}$$

denote a shared dictionary of points in input space. Let K denote the $m \times m$ matrix of pairwise kernel evaluations on this dictionary defined by

$$[K]_{ij} = k(\xi_i, \xi_j)$$

for some kernel function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$. For convenience, we also define

$$k(x) := \begin{pmatrix} k(\xi_1, x) \\ \vdots \\ k(\xi_m, x) \end{pmatrix} = \begin{pmatrix} \langle \psi(\xi_1), \psi(x) \rangle_{\mathcal{H}} \\ \vdots \\ \langle \psi(\xi_m), \psi(x) \rangle_{\mathcal{H}} \end{pmatrix}$$

to be the vector of kernel evaluations for $x \in \mathcal{X}$ with each of the dictionary elements in $\mathcal{B}$. Taking inspiration from the Representer Theorem, we choose to parametrize w by restricting our solution to the finite span of the dictionary,

$$w = \sum_{j=1}^m \beta_j \psi(\xi_j)$$

This way evaluations involving $\langle w, \psi(x) \rangle$ can be evaluated using kernels, and every node will construct a model using the same basis. Such a classifier will inherently lead to approximating local empirical risk minimizers on each node, excluding the case of the dictionary being the full dataset $\mathcal{D}$.

Next, we can discuss how to approximate the covariance operator with a given dataset. Consider a set $\mathcal{D} = \{x_1, \dots, x_n\}$ and define

$$K_{\mathcal{D}} = [k(x_1) \dots k(x_n)] \in \mathbb{R}^{m \times n}$$

Let

$$H_n = I - \frac{1}{n} \mathbf{1} \mathbf{1}^T$$

be the centering matrix of size n . Then we define

$$C_{\mathcal{D}} = \frac{1}{n} K_{\mathcal{D}} H_n K_{\mathcal{D}}^T$$

to serve as a finite dimensional representation of the empirical covariance operator restricted to a chosen dictionary $\mathcal{B}$.

This matrix takes the place of the empirical covariance matrix Σ in the linear lifted loss. Using this matrix along with the parametrization of w , we can then compute

$$s = \sqrt{\beta^T C_{\mathcal{D}} \beta}$$

which gives us

$$s^2 = \beta^T C_{\mathcal{D}} \beta = \frac{1}{n} (K_{\mathcal{D}}^T \beta)^T H_n (K_{\mathcal{D}}^T \beta)$$

showing that s^2 is the empirical variance of the kernel scores $\beta^T k(x_i)$. This is exactly as we had in the linear case. Thus $C_{\mathcal{D}}$ should be interpreted as a covariance matrix of kernel scores relative to the shared dictionary, rather than as a direct covariance operator in the RKHS.

The regularization term in the lifted loss becomes

$$\frac{1}{2} \|w\|_{\mathcal{H}}^2 = \frac{1}{2} \beta^T K \beta$$

Thus both the variance term and regularization term may be computed through finite dimensional matrix operations.

Now, with

$$\begin{aligned} \nu_i &= y(\beta^T k(x_i) + b), \\ a_i &= \frac{1 - \nu_i}{s} \end{aligned}$$

our regularized kernel loss takes the usual lifted form

$$\mathcal{L}(\beta, b) = \frac{1}{2} \|w\|_{\mathcal{H}}^2 + C_{reg} \sum_{i=1}^n [(1 - \nu_i) \Phi(a_i) + s \phi(a_i)]$$

with w, s redefined accordingly. We write C_{reg} to denote the hyperparameter as opposed to the empirical covariance operator.

With the loss defined, we will look to construct a new MR and SMR for kernel lifted SVM. Naturally, since the data are transformed via

$$x \rightarrow \psi(x) := k(x, \cdot)$$

into functions in $\mathcal{H}$, our representatives will also lie in $\mathcal{H}$. Such representatives can be characterized by elements in $\mathcal{H}$, they generally cannot be represented explicitly or communicated between nodes. We will first discuss the form in the RKHS, and then show how the shared dictionary can resolve this issue.

2.1. Notation and Derivatives. Before constructing representatives, let us define notation and rewrite the loss function with the parametrization of $w =$

$\sum_{j=1}^m \beta_j \psi(\xi_j)$. We then write the derivatives with respect to β . Introduce the notation

$$\Psi\beta = \sum_{j=1}^m \beta_j \psi(\xi_j) = w$$

Then note that

$$\begin{aligned} \langle w, \psi(x_i) \rangle &= \sum_{j=1}^m \beta_j \langle \psi(\xi_j), \psi(x_i) \rangle \\ &= \sum_{j=1}^m \beta_j k(\xi_j, x_i) \\ &= \beta^T k(x_i) \end{aligned}$$

hence clarifying why $\nu_i = y(\beta^T k(x_i) + b)$.

Now, consider

$$\ell(\beta, b) = (1 - \nu)\Phi(a) + s\phi(a)$$

and fix a class c . Then the derivatives with respect to β and b are:

$$\frac{\partial \ell}{\partial b} = -c\Phi(a)$$

and

$$\nabla_{\beta} \ell = -c\Phi(a)k(x) + \phi(a)\frac{C_{\mathcal{D}}\beta}{s}$$

being the same as the linear case with Σ being replaced by $C_{\mathcal{D}}$. With this, we move to the next section to discuss representatives.

3. Representatives for Kernel SVM

Below, we discuss how each of the representatives MR and SMR are computed and communicate between nodes despite being functions in the Hilbert space. For clarity, we will first just consider the dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ rather than a specific

node's dataset. Again, we fix the class $y_i = c$ within a group a data for which we compute representatives.

MR.

As we saw in the linear case, when we used average margin variables as a representative $\tilde{\nu} = \frac{1}{n} \sum_i \nu_i$, this was equivalent to using $\bar{x} = \frac{1}{n} \sum_i x_i$ in input space when the data are fixed from a given class 1 or -1 . The same holds for the kernel lifted loss. Consider on a fixed class $y_i = c$,

$$\begin{aligned} \frac{1}{n} \sum_{i=1}^n \nu_i &= \frac{1}{n} \sum_{i=1}^n c(\langle w, \psi(x_i) \rangle + b) \\ &= c \left(\left\langle w, \frac{1}{n} \sum_{i=1}^n \psi(x_i) \right\rangle + b \right) \end{aligned}$$

where we understand the inner product is computed in $\mathcal{H}$. Then by defining

$$\bar{\psi} = \frac{1}{n} \sum_{i=1}^n \psi(x_i)$$

We have

$$\frac{1}{n} \sum_{i=1}^n \nu_i = c(\langle w, \bar{\psi} \rangle + b)$$

Thus we let $\bar{\psi}$ be the MR. This corresponds to the **empirical kernel mean embedding** so we have $\bar{\psi} \in \mathcal{H}$. To see that we can perform computations with this, define for any $z \in \mathcal{H}$,

$$r(z) := \begin{pmatrix} \langle \psi(\xi_1), z \rangle \\ \vdots \\ \langle \psi(\xi_m), z \rangle \end{pmatrix}$$

Then

$$r(\bar{\psi}) = \frac{1}{n} \sum_{i=1}^n k(x_i)$$

and

$$\langle w, \bar{\psi} \rangle = \beta^T r(\bar{\psi})$$

Score-matching Representative.

Let $\tilde{\psi}$ denote the representative function for SMR which we will derive.

Let

$$\tilde{r} := r(\tilde{\psi}) = \begin{pmatrix} \langle \psi(\xi_1), \tilde{\psi} \rangle \\ \vdots \\ \langle \psi(\xi_m), \tilde{\psi} \rangle \end{pmatrix}$$

Since $w = \Psi\beta$, we have

$$\langle w, \tilde{\psi} \rangle = \beta^T \tilde{r}$$

and the following representative objects

$$\tilde{\nu} = c(\beta^T \tilde{r} + b)$$

$$\tilde{s} = \sqrt{\beta^T \tilde{C} \beta}$$

$$\tilde{a} = \frac{1 - \tilde{\nu}}{\tilde{s}}$$

where $\tilde{C}$ and $\tilde{\psi}$ are the unknowns we aim to solve for. From bias matching we solve for $\tilde{a}$ such that

$$\Phi(\tilde{a}) = \frac{1}{n} \sum_{i=1}^n \Phi(a_i)$$

and so

$$\tilde{a} = \Phi^{-1} \left(\frac{1}{n} \sum_{i=1}^n \Phi(a_i) \right)$$

From the β gradient, we match

$$-c\Phi(\tilde{a})\tilde{r} + \phi(\tilde{a})\frac{\tilde{C}\beta}{\tilde{s}} = \frac{1}{n} \sum_{i=1}^n \left[-c\Phi(a_i)k(x_i) + \phi(a_i)\frac{C_{\mathcal{D}}\beta}{s} \right]$$

As in the linear case, assume

$$\tilde{C} = \rho C_{\mathcal{D}}$$

Then

$$\tilde{s}^2 = \beta^T \tilde{C} \beta = \rho \beta^T C_{\mathcal{D}} \beta = \rho s^2$$

so

$$\tilde{s} = \sqrt{\rho} s$$

and

$$\frac{\tilde{C}\beta}{\tilde{s}} = \frac{\rho C_{\mathcal{D}}\beta}{\sqrt{\rho}s} = \sqrt{\rho} \frac{C_{\mathcal{D}}\beta}{s}$$

Define

$$p := \frac{1}{n} \sum_{i=1}^n \Phi(a_i)k(x_i)$$

and

$$\bar{\phi} := \frac{1}{n} \sum_{i=1}^n \phi(a_i)$$

then the averaged data gradient is

$$-cp + \bar{\phi} \frac{C_{\mathcal{D}}\beta}{s}$$

and the SMR weight matching equation is

$$-c\Phi(\tilde{a})\tilde{r} + \phi(\tilde{a})\sqrt{\rho} \frac{C_{\mathcal{D}}\beta}{s} = -cp + \bar{\phi} \frac{C_{\mathcal{D}}\beta}{s}$$

Solving for $\tilde{r}$ gives,

$$\tilde{r} = \frac{p}{\Phi(\tilde{a})} + \frac{c}{\Phi(\tilde{a})} (\phi(\tilde{a})\sqrt{\rho} - \bar{\phi}) \frac{C_{\mathcal{D}}\beta}{s}$$

(Note that for now, we will not solve explicitly for $\tilde{\psi}$). Next we can solve for ρ . We do this as before by multiplying the SMR weight matching equation by β^T . As before, define

$$\mathcal{G} := \frac{1}{n} \sum_{i=1}^n [-c\Phi(a_i)\beta^T k(x_i) + \phi(a_i)s]$$

to be the projected data gradient expression. The simplification comes from

$$\begin{aligned} \beta^T \frac{C_{\mathcal{D}}\beta}{s} &= s, \\ \beta^T \frac{\tilde{C}\beta}{\tilde{s}} &= \tilde{s} \end{aligned}$$

so that our matching equation becomes

$$\mathcal{G} = -c\Phi(\tilde{a})\beta^T \tilde{r} + \phi(\tilde{a})\tilde{s}$$

We will proceed with a slightly cleaner way for deriving the coefficient ρ .

It follows from our definitions at the beginning that,

$$\tilde{\nu} = 1 - \tilde{a}\tilde{s}$$

Thus

$$\begin{aligned} \mathcal{G} &= -\Phi(\tilde{a})(1 - \tilde{a}\tilde{s} - cb) + \phi(\tilde{a})\tilde{s} \\ &= -\Phi(\tilde{a})(1 - cb) + [\tilde{a}\Phi(\tilde{a}) + \phi(\tilde{a})]\tilde{s} \end{aligned}$$

Define

$$h(a) = a\Phi(a) + \phi(a)$$

Then we have

$$h(\tilde{a})\tilde{s} = \mathcal{G} + \Phi(\tilde{a})(1 - cb)$$

Using the same simplification to the data gradient by noticing that $\nu_i = 1 - a_i s$ gives

$$h(\tilde{a})\tilde{s} = s \frac{1}{n} \sum_{i=1}^n h(a_i)$$

Therefore,

$$\tilde{s} = s \frac{\frac{1}{n} \sum_{i=1}^n h(a_i)}{h(\tilde{a})}$$

hence our proportionality coefficient is

$$\rho = \left[\frac{\frac{1}{n} \sum_{i=1}^n h(a_i)}{h(\tilde{a})} \right]^2$$

using $\tilde{s} = \sqrt{\rho} s$.

Observe that the resulting SMR construction retains the same structure as the linear case, while margins are replaced by kernel scores, covariance matrices are replaced by dictionary-based covariance matrices, and representatives are communicated through dictionary coefficients.

In the next section, we will show how to use the dictionary to turn these representatives into coefficients that may be transferred and understood to other nodes.

3.1. Representatives using Dictionary Span. Above we used a dictionary for restricting the classifier w and defining the matrix $C_{\mathcal{D}}$. For SMR, we only computed the vector of dictionary interactions $\tilde{r}$ as this is enough for interactions with the restricted classifier w . Now, we will choose a realization for the representatives in the dictionary space to match the model. Define

$$\mathcal{H}_{\mathcal{B}} = \{\psi(\xi_1), \dots, \psi(\xi_m)\}$$

For any representative $z \in \mathcal{H}$, we seek

$$\hat{z} = \sum_{j=1}^m \gamma_j \psi(\xi_j) = \Psi \gamma$$

The representative will be understood as its coefficients γ . We call

$$r(\hat{z}) = K \gamma$$

the dictionary interaction vector. Matching this vector requires

$$K \gamma = r(z)$$

Then

$$\gamma = K^{-1} r(z)$$

In practice, we solve the regularized system

$$(K + \lambda_K I) \gamma = r(z)$$

where $\lambda_K > 0$ is small and $\tilde{s} = \sqrt{\beta^T \tilde{C} \beta + \epsilon}$ for small $\epsilon > 0$. It is worth noting that when the kernel is positive definite (e.g. RBF kernel), then K is mathematically (perhaps not numerically) non-singular as long as $\xi_i \neq \xi_j$. The dictionary interaction vectors for MR and SMR as derived above are,

- $r_{MR} = \frac{1}{n} \sum_{i=1}^n k(x_i)$
- $r_{SMR} = \tilde{r} = \frac{p}{\Phi(\tilde{a})} + \frac{c}{\Phi(\tilde{a})} (\phi(\tilde{a})\sqrt{\rho} - \bar{\phi}) \frac{C_D \beta}{s}$

Representatives for kernel SVM when used in distributed learning take the form

$$(\gamma, \tilde{C}, c, n)$$

for the representatives coefficients, representative covariance matrix, class, and number of data. Other nodes use this information in their own loss function by computing

$$\tilde{\nu} = c(\beta^T K \gamma + b)$$

where

$$K \gamma = r(\hat{z})$$

and the lifted term

$$\tilde{s} = \sqrt{\beta^T \tilde{C} \beta}$$

for their own parameter β as each node has parametrized w using the same shared dictionary $\mathcal{B}$. We treat this representative as a weighted observation

$$n [(1 - \tilde{\nu})\Phi(\tilde{a}) + \tilde{s}\phi(\tilde{a})]$$

as in the linear case. Practically speaking, this implementation with the dictionary allows for clean and simple transferring and computation with representatives between nodes.

3.2. Distributed Loss and Algorithm. The algorithm for kernel lifted SVM proceeds exactly as the linear SVM case. The only difference is the form of representatives. Below, we give the distributed loss function used for each node as well as the algorithm outline.

Denote the full dataset distributed across K nodes by

$$\mathcal{D} = \cup_{k=1}^K \mathcal{D}_k$$

where

$$\mathcal{D}_k = \{(x_i, y_i)\}_{i=1}^{n_k}$$

Then each node uses the loss function

$$\mathcal{L}_k(\beta, b) = \frac{1}{2}\beta^T K \beta + C \left(\sum_{c \in \{-1, 1\}} \sum_{i \in \mathcal{I}_{k,c}} \ell_{i,c} + \sum_{c \in \{-1, 1\}} \sum_{h \neq k} n_{h,c} \tilde{\ell}_{h,c} \right)$$

where we abbreviate $\ell_{i,c}$ to be the loss on data sample i from class c and $\tilde{\ell}_{h,c}$ to be the loss on the representative from node h on class c .

The algorithm then follows the steps:

(**Initialize:** each local node with gradient descent)

For outer iterations $T = 1, \dots$ and each node $k = 1, \dots, K$

- (1) Compute representatives $(n_{k,c}, \tilde{y}_{k,c} = c, \tilde{x}_{k,c}, \tilde{\Sigma}_{k,c})$
- (2) Communicate representatives to all connected nodes (or to a central server)
- (3) Optimize for some local iterations using gradient descent on representative augmented loss function

4. Experiments

4.1. Experimental Design. For kernel lifted SVM, we consider the RBF kernel throughout. Instead of using mixture Gaussian synthetic data for simulations, we will use two different public datasets to test our kernel representative methods. The first is the Ionosphere dataset, classifying radar returns as good or bad. The second is the Australian dataset, predicting credit card approval. We consider dividing the original dataset among three nodes based on the Dirichlet parameter α to induce high class imbalance. This is to mimic the linear distributed setup. The main details are summarized in appendix A.

As in the linear SVM case, we differ from the originally proposed local loss functions by choosing the regularization constant $C \rightarrow C/n$ with n being the total amount of data in the experiment. This simply makes for reasonable comparisons with the baseline with parameter averaging schemes like FedAvg when comparing local drift effects.

4.2. Public Dictionary. Our kernel SVM algorithm requires the use of a public dictionary. While tests were done using representatives themselves as the public dictionary (mentioned more in the conclusion 6), we decide on instead using a fraction of the training dataset to be shared between nodes.

The accuracy of our distributed algorithms and global solution can be quite dependent on the size of the dictionary. In order to keep the communication amounts reasonable for our algorithm, we choose this size to be 10% of the training data. The Australian dataset has 690 total data, and the Ionosphere dataset has 351 (the training dataset for which we take the points from is therefore a bit smaller).

4.3. Comparisons. The tables below are similar to the linear SVM results. For each dataset we generate nodes using Dirichlet parameter $\alpha = 0.1$. We include two tables for each dataset describing the training and test accuracies for each method. We compare different levels of local vs outer iterations, demonstrating how parameter based methods like FedAvg deteriorate severely due to client drift and how representative methods are able to reduce this. We also include a similar model agreement table for lifted SMR using RKHS distance (which is explained after the tables). Complete experimental design settings are summarized in A.

One major difference between the kernel methods and linear methods is the shared dictionary. Therefore, the global results contain “Central Full Dictionary” and “Central Shared Dictionary”. The full dictionary is simply RBF kernel training on the full dataset while the shared dictionary utilizes the same shared set of points as used in the distributed methods. This identifies the target of distributed learning, as well as the gap in accuracy due to using the smaller shared dictionary.

We again include ADMM for distributed hinge loss which is dependent only on the number of outer iterations selected. The corresponding constrained optimization problem is solved in each outer iteration.

TABLE 4.1. Ionosphere dataset training accuracy over 100 simulations for varying rounds/local iteration settings.

Method	1/1000	40/25	1000/1
MR (Hinge)	0.8612 ± 0.0318	0.8612 ± 0.0318	0.8612 ± 0.0318
SMR (Hinge)	0.9251 ± 0.0244	0.9109 ± 0.0382	0.7537 ± 0.1644
SMR (Lifted)	0.9294 ± 0.0189	0.9295 ± 0.0190	0.9134 ± 0.0349
MR (Lifted)	0.9275 ± 0.0212	0.9275 ± 0.0212	0.9275 ± 0.0212
FedAvg (Lifted)	0.9294 ± 0.0190	0.8888 ± 0.0773	0.8438 ± 0.1144
FedProx (Lifted)	0.9294 ± 0.0190	0.8880 ± 0.0787	0.8432 ± 0.1138
Local Only (Lifted)	0.7051 ± 0.1090	0.7051 ± 0.1090	0.7051 ± 0.1090
ADMM (Hinge)	0.9297 ± 0.0217	0.9297 ± 0.0219	0.7893 ± 0.1436
Classical Kernel SVM (Hinge)		0.9684 ± 0.0064	
Central Full Dictionary (Lifted)		0.9650 ± 0.0059	
Central Shared Dictionary (Lifted)		0.9324 ± 0.0165	

TABLE 4.2. Ionosphere dataset test accuracy over 100 simulations for varying rounds/local iteration settings.

Method	1/1000	40/25	1000/1
MR (Hinge)	0.8550 ± 0.0541	0.8550 ± 0.0541	0.8550 ± 0.0541
SMR (Hinge)	0.9189 ± 0.0398	0.9032 ± 0.0499	0.7473 ± 0.1640
SMR (Lifted)	0.9276 ± 0.0373	0.9277 ± 0.0376	0.9101 ± 0.0486
MR (Lifted)	0.9233 ± 0.0391	0.9233 ± 0.0391	0.9233 ± 0.0391
FedAvg (Lifted)	0.9275 ± 0.0372	0.8855 ± 0.0810	0.8408 ± 0.1180
FedProx (Lifted)	0.9275 ± 0.0372	0.8847 ± 0.0819	0.8387 ± 0.1168
Local Only (Lifted)	0.7006 ± 0.1069	0.7006 ± 0.1069	0.7006 ± 0.1069
ADMM (Hinge)	0.9200 ± 0.0392	0.9204 ± 0.0401	0.7843 ± 0.1454
Classical Kernel SVM (Hinge)		0.9402 ± 0.0325	
Central Full Dictionary (Lifted)		0.9383 ± 0.0324	
Central Shared Dictionary (Lifted)		0.9309 ± 0.0342	

TABLE 4.3. Ionosphere kernel SVM percentage of local model agreement (tolerance 10^{-2}).

Method	1/1000	40/25	1000/1
SMR (Lifted)	90%	84%	0%

TABLE 4.4. Australian dataset training accuracy over 100 simulations for varying rounds/local iteration settings.

Method	1/1000	40/25	1000/1
MR (Hinge)	0.8586 ± 0.0077	0.8586 ± 0.0077	0.8586 ± 0.0077
SMR (Hinge)	0.8620 ± 0.0087	0.8433 ± 0.0363	0.7196 ± 0.1662
SMR (Lifted)	0.8743 ± 0.0077	0.8743 ± 0.0077	0.8691 ± 0.0080
MR (Lifted)	0.8731 ± 0.0073	0.8731 ± 0.0073	0.8731 ± 0.0073
FedAvg (Lifted)	0.8742 ± 0.0077	0.8226 ± 0.0831	0.7964 ± 0.1128
FedProx (Lifted)	0.8742 ± 0.0077	0.8221 ± 0.0859	0.8024 ± 0.1045
ADMM (Hinge)	0.8655 ± 0.0076	0.8664 ± 0.0080	0.7321 ± 0.1524
Classical Kernel SVM (Hinge)		0.9067 ± 0.0071	
Central Full Dictionary (Lifted)		0.8994 ± 0.0077	
Central Shared Dictionary (Lifted)		0.8754 ± 0.0079	

TABLE 4.5. Australian dataset test accuracy over 100 simulations for varying rounds/local iteration settings.

Method	1/1000	40/25	1000/1
MR (Hinge)	0.8547 ± 0.0300	0.8547 ± 0.0300	0.8547 ± 0.0300
SMR (Hinge)	0.8572 ± 0.0287	0.8424 ± 0.0442	0.7178 ± 0.1686
SMR (Lifted)	0.8658 ± 0.0313	0.8657 ± 0.0312	0.8605 ± 0.0319
MR (Lifted)	0.8657 ± 0.0322	0.8657 ± 0.0322	0.8657 ± 0.0322
FedAvg (Lifted)	0.8658 ± 0.0314	0.8158 ± 0.0876	0.7917 ± 0.1137
FedProx (Lifted)	0.8658 ± 0.0314	0.8151 ± 0.0893	0.7964 ± 0.1066
ADMM (Hinge)	0.8618 ± 0.0330	0.8617 ± 0.0323	0.7264 ± 0.1532
Classical Kernel SVM (Hinge)		0.8586 ± 0.0310	
Central Full Dictionary (Lifted)		0.8668 ± 0.0308	
Central Shared Dictionary (Lifted)		0.8671 ± 0.0308	

TABLE 4.6. Australian kernel SVM percentage of local model agreement (tolerance 10^{-2}).

Method	1/1000	40/25	1000/1
SMR (Lifted)	93%	93%	0%

The kernel experiments follow the behavior observed in the linear setting on two real datasets. Under frequent or moderate representative updates, lifted MR and lifted SMR remain close to the centralized shared dictionary model on both training and test accuracy. In contrast, FedAvg and FedProx become increasingly sensitive to the number of local steps taken between communications. The comparison between hinge-based and lifted representatives provides additional evidence for the lifted ERM framework. While both MR and SMR benefit from lifting, the improvement is particularly dramatic for SMR. Because score-matching representatives rely directly on objective derivatives, they appear especially sensitive to the smoothness provided

by the lifted objective. This effect is most apparent in the low communication regime, where hinge-based SMR deteriorates substantially faster than its lifted counterpart.

Lifted SMR also produces local parameter agreement in the large majority of simulations, at least when representatives are refreshed during training, obtaining 90 – 93% agreement under the frequent communication setting and 84 – 93% under the intermediate setting. As expected, agreement is lost when we use a single outer iteration followed by only local.

Lifted MR is particularly competitive in these experiments and often matches lifted SMR in predictive accuracy. However, predictive accuracy alone does not imply recovery of the centralized optimization solution. The following RKHS parameter comparisons distinguish these two notions. Although MR often achieves similar classification performance, lifted SMR consistently produces classifiers that are substantially closer to the centralized shared-dictionary solution in RKHS distance while also maintaining stronger directional agreement.

The agreement tables above compare node models to one another. We next compare each distributed method directly to the centralized shared dictionary solution in order to determine whether agreement corresponds to recovery of the centralized optimizer.

TABLE 4.7. Ionosphere - RKHS model distance to centralized

Method	1/1000	25/40	1000/1
MR (Hinge)	2.0588 ± 0.2334	2.0588 ± 0.2334	2.0588 ± 0.2334
MR (Lifted)	0.7246 ± 0.1988	0.7246 ± 0.1988	0.7246 ± 0.1988
SMR (Hinge)	0.7684 ± 0.2062	0.9086 ± 0.2593	2.0234 ± 0.7213
SMR (Lifted)	0.5961 ± 0.2039	0.5960 ± 0.2038	0.6972 ± 0.2386

TABLE 4.8. Ionosphere - RKHS cosine similarity to centralized

Method	1/1000	25/40	1000/1
MR (Hinge)	0.8865 ± 0.0342	0.8865 ± 0.0342	0.8865 ± 0.0342
MR (Lifted)	0.9795 ± 0.0177	0.9795 ± 0.0177	0.9795 ± 0.0177
SMR (Hinge)	0.9785 ± 0.0179	0.9729 ± 0.0226	0.9037 ± 0.0569
SMR (Lifted)	0.9842 ± 0.0141	0.9842 ± 0.0141	0.9793 ± 0.0163

TABLE 4.9. Australian - RKHS model distance to centralized

Method	1/1000	25/40	1000/1
MR (Hinge)	1.9901 ± 0.1043	1.9901 ± 0.1043	1.9901 ± 0.1043
MR (Lifted)	0.5755 ± 0.0422	0.5755 ± 0.0422	0.5755 ± 0.0422
SMR (Hinge)	0.6300 ± 0.1177	0.8669 ± 0.1938	1.8893 ± 0.6440
SMR (Lifted)	0.2967 ± 0.0667	0.2970 ± 0.0666	0.4642 ± 0.1331

TABLE 4.10. Australian - RKHS cosine similarity to centralized

Method	1/1000	25/40	1000/1
MR (Hinge)	0.9616 ± 0.0053	0.9616 ± 0.0053	0.9616 ± 0.0053
MR (Lifted)	0.9904 ± 0.0025	0.9904 ± 0.0025	0.9904 ± 0.0025
SMR (Hinge)	0.9883 ± 0.0033	0.9739 ± 0.0148	0.8827 ± 0.1281
SMR (Lifted)	0.9953 ± 0.0022	0.9953 ± 0.0022	0.9887 ± 0.0068

Tables 4.7–4.10 reveal a similar story to linear SVM. This observation helps reconcile the predictive-accuracy tables above. MR occasionally matches or exceeds SMR in classification accuracy, especially when communication is very limited, but SMR remains the method that most faithfully reproduces the centralized optimization solution. Although lifted MR and lifted SMR obtain very similar predictive performance, lifted SMR is consistently closer to the centralized shared dictionary solution in the RKHS. On both datasets, SMR yields the smallest RKHS norm discrepancy and

the largest cosine similarity under the regimes in which representatives are periodically updated.

As we explained in the linear case, this meets our expectations as MR approximates the class data while SMR contains first-order gradient information depending on the model.

The RKHS inner product between the distributed models and centralized model is

$$\begin{aligned}\langle w, w_g \rangle_{\mathcal{H}} &= \langle \Psi_D \beta, \Psi_D \beta_g \rangle_{\mathcal{H}} \\ &= \beta^T \Psi_D^T \Psi_D \beta_g \\ &= \beta^T K_D \beta_g.\end{aligned}$$

due to the parametrization of w .

The RKHS distance between the distributed and centralized models is

$$\begin{aligned}\|w - w_g\|_{\mathcal{H}} &= \sqrt{\langle w - w_g, w - w_g \rangle_{\mathcal{H}}} \\ &= \sqrt{(\beta - \beta_g)^T K_D (\beta - \beta_g)}.\end{aligned}$$

Similarly, the cosine similarity induced by the RKHS inner product is

$$\begin{aligned}\text{Cos}_{\mathcal{H}}(w, w_g) &= \frac{\langle w, w_g \rangle_{\mathcal{H}}}{\|w\|_{\mathcal{H}} \|w_g\|_{\mathcal{H}}} \\ &= \frac{\beta^T K_D \beta_g}{\sqrt{\beta^T K_D \beta} \sqrt{\beta_g^T K_D \beta_g}}.\end{aligned}$$

These quantities provide natural kernel analogues of Euclidean distance and cosine similarity for linear models. The RKHS distance measures the absolute discrepancy between the learned classifiers, while the RKHS cosine similarity measures the alignment of the corresponding parameter vectors in the Hilbert space, with values closer to one indicating greater agreement with the centralized solution.

For each simulation seed, both quantities were computed by comparing every node model with the corresponding centralized model. The reported values first average these comparisons across the three nodes before averaging over all simulation seeds.

5. Experiments with RFFs

Here we briefly introduce a different idea for kernel SVM which approximates the kernel function with a linear one. Due to the simplification, this method is popular in distributed learning and introduces an important concept in learning with kernels. Our shared dictionary method above approximates solutions via a restriction to the shared subspace. The method with Random Fourier Features (RFFs) aims at approximating the kernel matrix itself. Our representative methods for linear lifted SVM naturally apply to RFFs. Unlike the shared-dictionary approach, RFFs embed the problem into a common finite-dimensional Euclidean space.

5.1. Background. RFFs work due to Bochner’s Theorem which applies to any continuous, positive definite, shift-invariant kernel function $k(x, y) = \psi(x - y)$. Namely, we have

$$k(x, y) = \mathbb{E}_{\omega \sim p} \left[e^{i\omega^T(x-y)} \right] = \int_{\mathbb{R}^d} e^{i\omega^T(x-y)} p(\omega) d\omega$$

for a non-negative probability measure $p(\omega)$. For the RBF kernel we have

$$k(x, y) = \exp -\gamma \|x - y\|^2$$

and the associated probability measure is

$$p(\omega) = \mathcal{N}(0, 2\gamma I)$$

We then sample

$$\omega_1, \dots, \omega_D \sim \mathcal{N}(0, 2\gamma I)$$

and define the features

$$z(x) = \frac{1}{\sqrt{D}} \begin{bmatrix} \cos(\omega_1^T x) \\ \sin(\omega_1^T x) \\ \vdots \\ \cos(\omega_D^T x) \\ \sin(\omega_D^T x) \end{bmatrix}$$

so that

$$z(x)^T z(y) \approx k(x, y)$$

Our decision function is thus of the form

$$f(x) = w^T z(x) + b$$

where $z(x) \in \mathbb{R}^{2D}$. All nodes share this same feature space and thus the same parameter dimension for w . For the simulations below, we refer back to our section on linear lifted SVM.

5.2. Experiments. We provide similar tables as the previous simulation section using again three nodes selected utilizing the Dirichlet distribution with $\alpha = 0.1$. For the shared feature space among nodes, we draw from a single Gaussian 50 times. Each Gaussian draw results in two effective features due to the cosine and sine terms existing in $z(x)$ above. The results are averaged over 100 seeds of experiments where we again allow each node a warm start with local training iterations.

TABLE 4.11. RFF Ionosphere train accuracy with $\alpha = 0.1$

Method	1/1000	40/25	1000/1
MR (Hinge)	0.8718 ± 0.0279	0.8645 ± 0.0264	0.8532 ± 0.0293
SMR (Hinge)	0.9420 ± 0.0092	0.8902 ± 0.1044	0.6602 ± 0.1493
SMR (Lifted)	0.9408 ± 0.0126	0.9408 ± 0.0126	0.9300 ± 0.0210
MR (Lifted)	0.9438 ± 0.0105	0.9438 ± 0.0105	0.9438 ± 0.0105
FedAvg (Lifted)	0.9410 ± 0.0126	0.8539 ± 0.1014	0.8179 ± 0.1246
ADMM (Hinge)	0.9422 ± 0.0095	0.8807 ± 0.0659	0.7390 ± 0.1058
Classical RFF SVM (Hinge)		0.9741 ± 0.0070	
Global Pooled RFF (Lifted)		0.9410 ± 0.0126	

TABLE 4.12. RFF Ionosphere test accuracy with $\alpha = 0.1$

Method	1/1000	40/25	1000/1
MR (Hinge)	0.8584 ± 0.0383	0.8513 ± 0.0405	0.8409 ± 0.0402
SMR (Hinge)	0.9201 ± 0.0335	0.8725 ± 0.1008	0.6543 ± 0.1451
SMR (Lifted)	0.9121 ± 0.0349	0.9121 ± 0.0349	0.8999 ± 0.0395
MR (Lifted)	0.9189 ± 0.0343	0.9189 ± 0.0343	0.9189 ± 0.0343
FedAvg (Lifted)	0.9125 ± 0.0350	0.8353 ± 0.0988	0.8025 ± 0.1203
ADMM (Hinge)	0.9208 ± 0.0350	0.8600 ± 0.0735	0.7325 ± 0.1069
Classical RFF SVM (Hinge)		0.9296 ± 0.0324	
Global Pooled RFF (Lifted)		0.9125 ± 0.0350	

TABLE 4.13. RFF Australian train accuracy with $\alpha = 0.1$

Method	1/1000	40/25	1000/1
MR (Hinge)	0.8385 ± 0.0141	0.8363 ± 0.0153	0.8017 ± 0.0340
SMR (Hinge)	0.8371 ± 0.0583	0.7090 ± 0.1401	0.5854 ± 0.1182
SMR (Lifted)	0.8616 ± 0.0124	0.8616 ± 0.0124	0.8087 ± 0.0577
MR (Lifted)	0.8595 ± 0.0119	0.8595 ± 0.0119	0.8595 ± 0.0119
FedAvg (Lifted)	0.8616 ± 0.0122	0.7394 ± 0.1184	0.7031 ± 0.1434
ADMM (Hinge)	0.8594 ± 0.0117	0.7240 ± 0.1280	0.6456 ± 0.1163
Classical RFF SVM (Hinge)		0.8957 ± 0.0111	
Global Pooled RFF (Lifted)		0.8616 ± 0.0122	

Tables 4.11–4.14 show that the lifted representative learning behavior is not specific to the shared dictionary RKHS formulation. After approximating the RBF kernel with an RFF feature map, the linear representative methods can be applied directly in the resulting finite dimensional feature space.

TABLE 4.14. RFF Australian test accuracy with $\alpha = 0.1$

Method	1/1000	40/25	1000/1
MR (Hinge)	0.8157 ± 0.0341	0.8153 ± 0.0343	0.7875 ± 0.0438
SMR (Hinge)	0.8058 ± 0.0637	0.6884 ± 0.1271	0.5781 ± 0.1114
SMR (Lifted)	0.8276 ± 0.0369	0.8275 ± 0.0369	0.7738 ± 0.0583
MR (Lifted)	0.8267 ± 0.0363	0.8267 ± 0.0363	0.8267 ± 0.0363
FedAvg (Lifted)	0.8277 ± 0.0371	0.7176 ± 0.1042	0.6862 ± 0.1246
ADMM (Hinge)	0.8292 ± 0.0389	0.7024 ± 0.1180	0.6402 ± 0.1086
Classical RFF SVM (Hinge)		0.8267 ± 0.0388	
Global Pooled RFF (Lifted)		0.8277 ± 0.0371	

Lifted MR and lifted SMR remain highly competitive with the pooled RFF objective under frequent and intermediate representative updates on both datasets. In contrast to the shared-dictionary kernel formulation, the performance differences between MR and SMR are generally much smaller under RFFs, with both representatives closely tracking the centralized lifted solution.

The same communication-setting effects observed previously are present in the RFF experiments. When communication becomes infrequent, FedAvg and ADMM deteriorate substantially as local models drift toward node-specific solutions. Lifted SMR also experiences some degradation when only a single communication round is available, but remains considerably more stable than both FedAvg and the hinge-based SMR construction. Under frequent and intermediate communication, all lifted representative methods closely match the centralized lifted RFF solution.

The comparison between hinge-based and lifted representatives provides additional evidence for the lifted ERM framework. Across both datasets, the lifted versions of the representative methods are consistently more stable than their hinge-based counterparts. The effect is particularly pronounced for SMR, whose construction depends explicitly on gradient information. The smoothing introduced by the lifted objective therefore appears to improve both optimization behavior and the quality of the resulting representatives.

CHAPTER 5

Representative Based Collaborative Learning

1. Motivation

Preceding chapters showed how to utilize representatives for optimizing DSVMs. The role of the representatives was to encapsulate information of node data and/or gradients in order for local nodes to approximate a global solution. For completeness, here we will discuss a different use of representatives: to serve as a public shared dataset. Specifically, we will discuss using this shared dataset to facilitate KD as a means for personalized optimization.

In Chapter 2, we discussed personalized learning in distributed settings. Personalized learning often looks to utilize other nodes to strengthen and generalize their own model while remaining strong on local data. In this chapter, we will focus on a subset of personalized learning where nodes look to generalize their tasks, or model prediction goals, on distributions of data that the node doesn't have access to.

While we will discuss relatively small linear models, KD techniques are commonly used with large neural networks, and thus we restrict the sharing of model parameters between nodes. This also agrees with the regime of the previous lifted SVM algorithms.

2. Collaborative Framework

As with DSVM, the framework for collaborative learning is different from a usual centralized one. Here we assume each node is training locally but may receive help from connected nodes. Additionally, our goal is not to train one centralized model, but rather a personalized model for each node depending on their “prediction

task goals” which we discuss later. Since each node has a personalized task, we will show how help from certain nodes benefits more than others.

Suppose we have datasets for each node

$$\mathcal{D}_1, \dots, \mathcal{D}_K$$

Specifically, we consider local logistic regression models on each, therefore the datasets on each node take the form

$$\mathcal{D}_k = \{(x_i, y_i) : x_i \in \mathcal{X}_k, y_i \in \{0, 1\}\}$$

where $\mathcal{X}_k \subset \mathbb{R}^d$ is the feature space for node k . Define by

$$P_k(x, y)$$

the joint probability distribution for node k ’s data $\mathcal{D}_k$.

The logistic loss on data $\mathcal{D}_k$ is defined as

$$\mathcal{L}(\mathcal{D}_k) = -\frac{1}{n_k} \sum_{i \in \mathcal{I}_k} \log p(y_i | x_i; \theta_k)$$

where $\mathcal{I}_k$ denotes the index set for $\mathcal{D}_k$ and θ_k is the model from node k . $p(y|x; \theta)$ denotes the prediction function which is modeled by a sigmoid activation, so that

$$p(y|x; \theta) = \frac{1}{1 + e^{-\theta^T x}}$$

Often, nodes aim to predict well on another distribution $Q_k(z, y)$ which we refer to as the **prediction task distribution**. In the case we consider, this will happen when the features of the training data differ from the features of the task data. Such prediction task distributions motivate our necessity for help from select nodes to improve the local models when the number of choices of helpers is limited.

One way for a local node to get help is to align its model more closely with other nodes. To accomplish this, we propose using Knowledge Distillation (KD) and

incorporating this into the loss function. KD is implemented by sharing predictions on a common dataset and adding a penalty for the differences.

Utilizing representatives from each of the nodes, we can define a representative shared public dataset

$$\tilde{\mathcal{D}} = \cup_{k=1}^K \tilde{\mathcal{D}}_k$$

where

$$\tilde{\mathcal{D}}_k = \{(\tilde{x}_h, \tilde{y}_h)\}_{h=1}^{r_k}$$

denotes the representatives/pseudo data points from node k . For KD, we will need to share predictions (or sometimes scores) on these representative features, $p_k(\tilde{x}) = f_k(\tilde{x}; \theta_k)$. As we are using logistic models on each node, the predictions lie between 0 and 1 for every model and thus define a Bernoulli distribution on each data point.

To align local models with another node, we look to penalize the difference in these probability distributions defined by predicting on the representative data. The natural choice is to use Kullback-Leibler (KL) Divergence. For logistic regression, each local node uses Cross Entropy (CE) loss on their own data. For our optimization purposes, the CE loss and KL loss are equivalent here, therefore we use the letter $\mathcal{L}$ to denote both the local CE loss, as well as the between node KL loss.

Thus we define the local objective for node k at time t as

$$\mathcal{L}_k^{(t)} = (1 - \alpha)\mathcal{L}(\mathcal{D}_k) + \alpha \sum_{j \neq k} \lambda_{j,k}^{(t)} \mathcal{L}(p_j^{(t-1)}, p_k^{(t-1)})$$

Here the KL loss is not symmetric. We specifically choose the first argument as p_j to view these predictions as the “true” ones for penalization.

$$\mathcal{L}(p_j, p_k) = \frac{1}{n} \sum_{h=1}^K n_h D_{KL}(p_j^{(h)} || p_k^{(h)})$$

We use the notation $p_j := p_j(\tilde{\mathcal{D}}_k)$, to denote the predictions on each point from the shared dataset. $\lambda_{j,k}^{(t)}$ represent adaptive distillation weights. These are designed to help decide which nodes should be selected in helping with the local node’s task(s). In the next section, we will discuss how to compute these.

3. Energy Coefficients

In the previous section, we defined our loss function. The adaptive distillation weights $\lambda_{j,k}^{(t)}$ will determine the contribution of node j ’s model to node k ’s. This weight will be related to how much node j can help node k with their prediction task distribution.

We first discuss a distance defined between random variables known as the **energy coefficient** H , defined as

$$H = \frac{2\mathbb{E} \|X - Y\| - \mathbb{E} \|X - X'\| - \mathbb{E} \|Y - Y'\|}{2 \|X - Y\|}$$

where X' and Y' are independent copies of X and Y respectively. Computing this distance empirically requires hefty computation. Therefore, we use Taylor approximations to compute the energy distance between each feature i on the datasets X and Y . Denote this distance by H_i . Additionally, we combine these individual feature distances using a weighted average to compute

$$H = \sum_{i=1}^d w_i H_i$$

where w_i incorporates the importance of feature i and $\sum_i w_i = 1$. The exact approximations and formulas are left to the appendix (B). Since we work specifically with binary class datasets in this work, we compute two energy distances between each node: $H^{(0)}$ for class 0 and $H^{(1)}$ for class 1. Then the final distillation weight formula is defined as

$$\lambda_{j,k} = \sum_{i=1}^{\tau_k} \hat{H}_k^{(i)} (1 - \hat{H}_j^{(i)})$$

where i indexes the number of datasets we compute the distance for. Here i will be 0 and 1 to represent the two classes on each node.

4. Algorithm

Below we give a flowchart as well as a detailed description of the synchronous distributed algorithm.

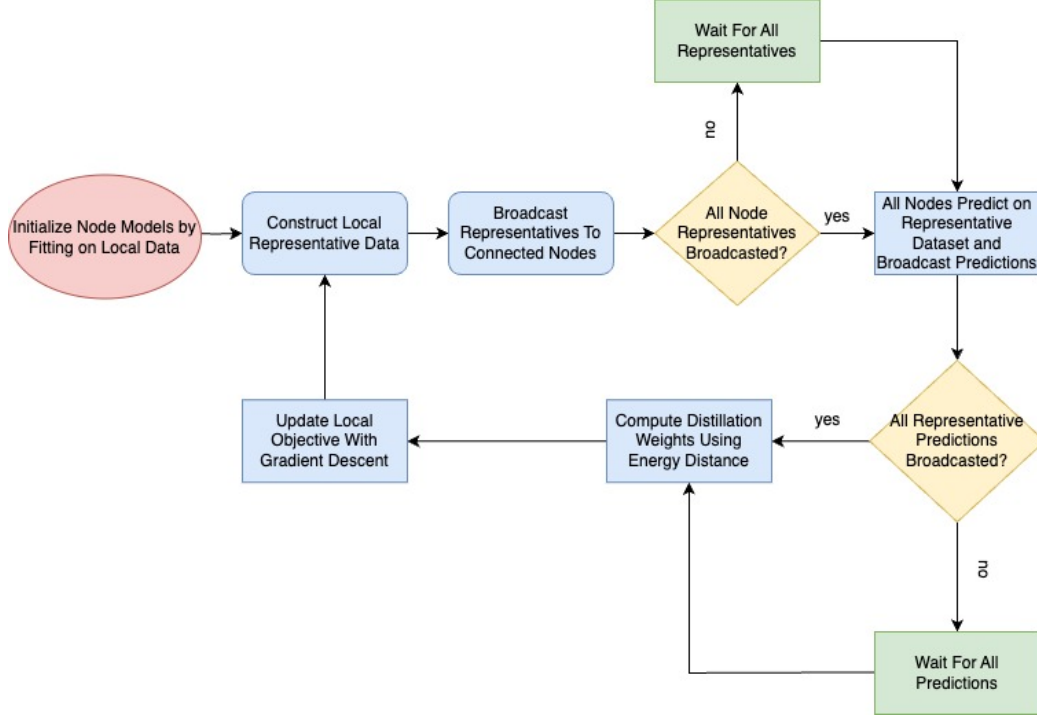


FIGURE 5.1. Flowchart of Collaborative Transfer Learning framework.

Algorithm 1 Collaborative Transfer Learning (CTL)

```
1: Initialization:
2: for each node  $k = 1, \dots, K$  do
3:   Fit local model  $f(\cdot; \theta_k^{(0)})$  on  $\mathcal{D}_k$ 
4:   Construct representative dataset  $\tilde{\mathcal{D}}_k^{(0)}$ 
5:   Broadcast  $\tilde{\mathcal{D}}_k^{(0)}$  to connected nodes
6: end for
7: [Synchronize]: Wait until all  $\tilde{\mathcal{D}}_j^{(0)}$  are received
8: for each node  $k = 1, \dots, K$  do
9:   Compute predictions  $p_k^{(0)} = f(\tilde{\mathbf{x}}_h^{(0)}; \theta_k^{(0)})$ 
10:  Broadcast  $p_k^{(0)}$  to connected nodes
11: end for
12: [Synchronize]: Wait until all  $p_j^{(0)}$  are received
13: for  $t = 1, \dots, T$  do
14:   for each node  $k = 1, \dots, K$  do
15:     Update distillation weights  $\lambda_{j,k}^{(t)}$  using Energy Coefficient  $H$  by
16:     Update local objective function  $\mathcal{L}_k^{(t)}$ 
17:     Perform gradient descent to update  $\theta_k^{(t)}$ 
18:     Construct updated representative dataset  $\tilde{\mathcal{D}}_k^{(t)}$ 
19:     Broadcast  $\tilde{\mathcal{D}}_k^{(t)}$  to connected nodes
20:   end for
21:   [Synchronize]: Wait until all  $\tilde{\mathcal{D}}_j^{(t)}$  are received
22:   for each node  $k = 1, \dots, K$  do
23:     for each updated  $\tilde{\mathcal{D}}_j^{(t)}$  do
24:       Compute updated predictions  $p_k^{(t)} = f(\tilde{\mathbf{x}}_h^{(t)}; \theta_k^{(t)})$ 
25:     end for
26:     Broadcast  $p_k^{(t)}$  to connected nodes
27:   end for
28: end for
```

5. Simulation Results

For the following simulations, we refer to the host node as the current local node which is training, and guest node as a helper for the host node's prediction task(s).

We evaluated five strategies for assigning guest nodes to each host node:

- (1) *All*: All other nodes are used as guest nodes.
- (2) *Best*: Select the two nodes with the lowest Energy Coefficients H relative to each prediction task.

- (3) *Random*: Select two random guest nodes.
- (4) *Worst*: Select the two nodes with the highest H relative to each prediction task.
- (5) *Local*: No guest nodes are used (local-only training).

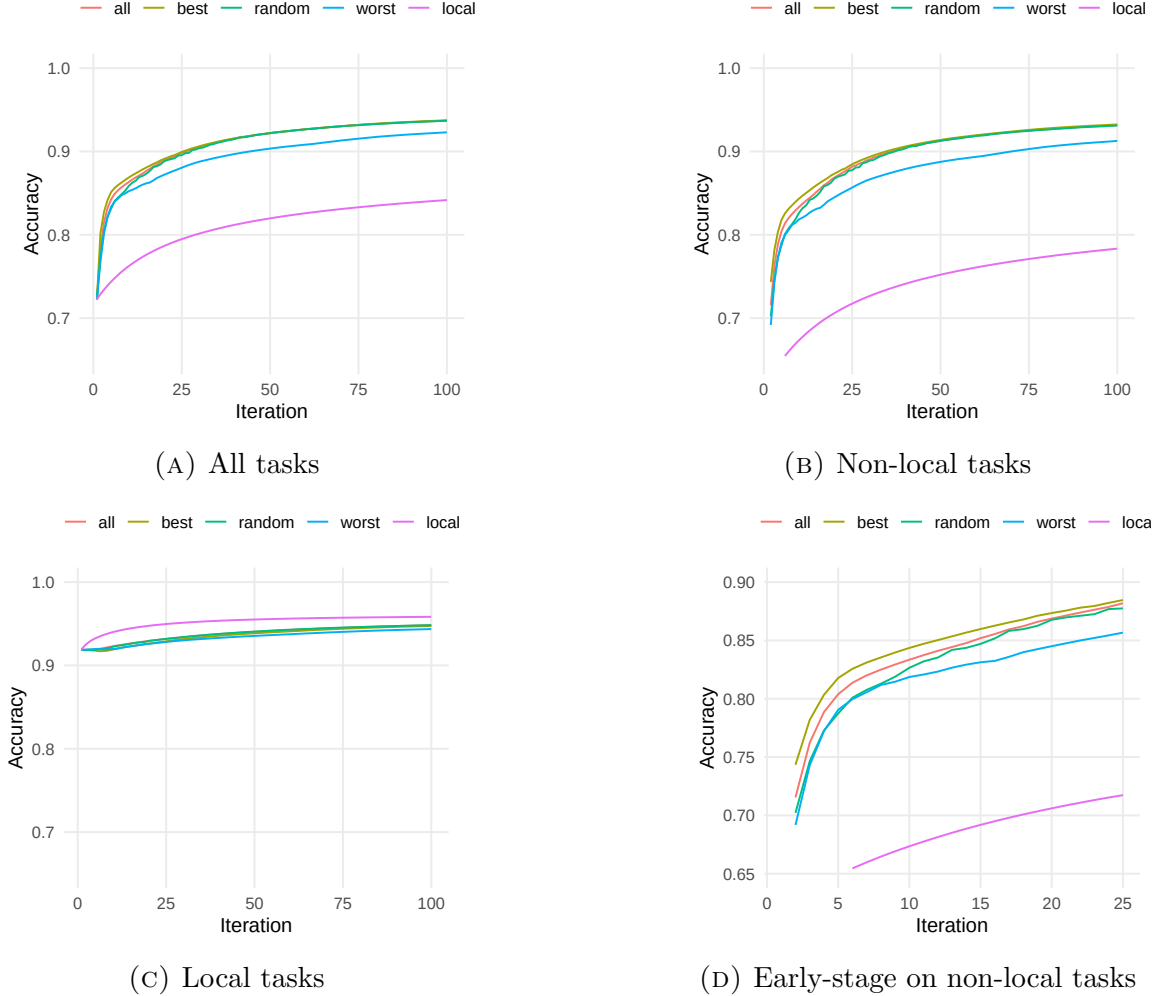


FIGURE 5.2. Performance of guest-node selection strategies over 100 simulations. (a) Mean accuracy across all prediction tasks. (b) Mean accuracy for tasks from distributions different from local training data. (c) Mean accuracy for tasks aligned with the local training data distribution. (d) A zoomed-in view of early-stage accuracy on non-local tasks (first 25 iterations), highlighting the initial adaptation dynamics of different strategies.

Figure 5.2 demonstrates that collaboration is most beneficial when the prediction task differs from the host node’s local training distribution. For local tasks,

local-only training already performs strongly and collaboration provides little additional benefit. In contrast, on non-local tasks the local-only model remains substantially below every collaborative strategy, showing that information from other nodes is necessary for the host model to generalize to distributions it has not observed locally.

The energy coefficient based “best” selection strategy is particularly effective during the early stages of adaptation, where it consistently outperforms selection of the least important guest nodes or even all of the nodes. As training continues, the collaborative strategies become closer, although selecting the “worst” guests remains generally inferior. Thus, the primary value of the Energy Coefficient is not merely to enable collaboration, but to identify informative collaborators when the number of available guest nodes is restricted and adaptation to a non-local task is important.

These simulations support the use of representative data for knowledge distillation while also showing that the usefulness of a guest node depends on its relationship to the host’s prediction task. Using representatives as a shared dataset differs substantially from their use in Chapters 3 and 4, where they were used to help approximate objectives by condensing data or matching first-order information. In CTL, MR can be used to facilitate distillation without the need of a previously defined shared dataset while being very simple.

CHAPTER 6

Conclusion and Future Work

1. Conclusion

We develop the lifted SVM framework from ERM as a smooth surrogate for the usual hinge loss. This form allows for not only a more robust loss function in distributed learning, but strong representatives capable of handling heterogeneity and client drift under extensive local learning. This in turn motivates a framework with reduced communication overhead. Lifted MR remains a simple construction which produces strong experimental results and lifted SMR demonstrates the ability to track the global model.

While kernels introduce a more complicated setting in DML, representative methods retain their strong properties. Beyond SVM, representatives prove versatility as pseudo-data points in other DML regimes involving knowledge distillation.

2. Further Experiments

An interesting experiment to run in distributed learning is the case of a single communication round between nodes. Here, we consider this case prior to any full-scale learning and after a warm local start. However, when the loss is properly configured to handle heavy class imbalances, it is also interesting to consider the case where each local node trains to completion prior to this single communication round. Based on Figures 3.25 – 3.32, we would expect lifted SMR to perform strongly here. We may expect lifted MR to be the best choice here, but alternatives could provide a useful comparison.

3. Representative Kernel Dictionary

For kernel lifted SVM, we assumed the use of a public dictionary shared between nodes. For experiments, this was simply a random percentage of the original dataset, and in practice, this may be some publicly available data online. An interesting choice is using representatives to act as the public dictionary. This is in part motivated by the use of representatives for KD in [2].

For example, SMR constructs a representative with components $(\tilde{x}, \tilde{\Sigma})$. We showed in the Chapter 4 experiments that, after local node training is completed locally, this representative tends to give a point near the center with covariance that approximates the convex hull of the data when considering two standard deviations of the Gaussian. This may suggest a useful way to construct a public dictionary composed of representatives from each node.

A simpler way to make a public dictionary using representatives is to choose MR. If only one representative per class per node is chosen, such a small public dictionary will struggle for learning complicated models on difficult datasets. In that case, it may be more appropriate to choose multiple representatives per class, potentially using clustering strategies.

4. SVM and Deep Learning

Besides the fact that SVM is a powerful algorithm for classification (and regression), one of the primary motivations for studying them lies in the applications to more modern deep learning frameworks. [31] demonstrates how linear SVM can be used as an activation in deep learning. [32] found an equivalence between learning linear SVM and the attention optimization problems in transformers.

5. Representatives For Neural Networks

In [43], they considered approximating the loss gradient using a smaller, synthetic dataset. However, this does not easily apply to DML with the usual restrictions.

It is interesting to consider representatives (similar to those in this dissertation or perhaps totally different) in the case of neural networks. Such representatives could be constructed in the embedding space or for learned feature spaces.

While gradient-matching (score-matching) is complicated for the entire neural network, matching the gradients for given layers may prove to be a promising idea. Eventually, representatives may be constructed to match the gradients constructed in backpropagation for important layers.

APPENDIX A

LINEAR AND KERNEL SVM EXPERIMENTAL DETAILS

1. Linear SVM Simulation Details

1.1. Simulation Parameters. This table summarizes the parameters used throughout the synthetic linear SVM experiments:

TABLE A.1. Default simulation parameters used for the synthetic linear SVM experiments.

Parameter	Value
Number of nodes	3
Number of simulations	100
Dimensions tested	$\{2, 3, 5, 7\}$
Lifted Regularization parameter	$C = 75 \ (\rightarrow C/n)$
Hinge Regularization parameter	$C = 75 \ (\rightarrow C/n)$
Learning rate	0.005
Initial local epochs	20
Local epochs per communication round	40
Communication rounds	25
Dirichlet parameter	$\alpha = 0.1$
Global positive class proportion	0.5
Minimum node samples per class	2
Gaussian mean selection range	$[0.5, 2.0]$
Covariance diagonal range	$[0.05, 0.8]$

1.2. Synthetic Data Generation. Each synthetic dataset consists of two classes generated from class-conditional Gaussian mixture distributions. Each synthetic dataset consists of 600 total observations, equally divided between the two classes prior to distributed partitioning. For each class, observations are sampled from a mixture of three Gaussian components. The geometric relationship between the two classes is controlled by a class-separation parameter that determines the degree of overlap between the positive and negative classes. Let m denote one-half of the

distance between the two base class centers. Larger values of m therefore correspond to increased separation between the classes. In the high-overlap simulations,

$$m \sim \text{Unif}(0.25, 1.00),$$

while in the low-overlap (separable) simulations,

$$m \sim \text{Unif}(1.00, 2.50).$$

To produce more realistic class distributions, the center of each Gaussian component is obtained by randomly perturbing the corresponding class center. Specifically, if μ_c denotes the center of class c , then the j th mixture component has mean

$$\mu_{c,j} = \mu_c + r_{c,j} z_{c,j},$$

where

$$r_{c,j} \sim \text{Unif}(0, 0.75),$$

and $z_{c,j}$ is a randomly generated unit vector. The mixture weights are sampled according to

$$(\pi_{c,1}, \pi_{c,2}, \pi_{c,3}) \sim \text{Dirichlet}(1, 1, 1),$$

yielding the class-conditional distribution

$$X \mid Y = c \sim \sum_{j=1}^3 \pi_{c,j} \mathcal{N}(\mu_{c,j}, \Sigma_c).$$

1.3. Covariance Selection. Recall that the lifted loss permits assigning a covariance matrix to each input point. This covariance matrix $\Sigma_{k,c}$ is selected randomly from a predefined covariance bank. The bank contains dispersion levels $[0.25, 0.50, 1.00, 1.50, 2.00]$. At each dispersion level, three covariance structures are included: isotropic, mildly anisotropic, and strongly anisotropic. Each covariance is

constructed in the form

$$\Sigma = s^2 A,$$

where s is the corresponding dispersion level and A is normalized so that

$$\frac{\text{tr}(A)}{d} = 1.$$

Thus,

$$\sqrt{\frac{\text{tr}(\Sigma)}{d}} = s,$$

and the parameter s can be interpreted as the typical within-component standard-deviation scale.

For the isotropic case,

$$A = I_d.$$

The mildly anisotropic matrices assign twice the variance to the first coordinate before normalization, whereas the strongly anisotropic matrices assign 25 times the variance.

The covariance matrices for the positive and negative classes are selected independently, so the two classes on a given node are not required to have equal covariance structures.

1.4. Dirichlet Data Partitioning. After generating the complete dataset, observations were distributed among the three nodes using independent Dirichlet-multinomial allocations for each class with parameter

$$\alpha = 0.1.$$

The pooled dataset was generated with an equal proportion of positive and negative samples prior to partitioning. The Dirichlet allocation therefore preserved

global class balance while producing highly heterogeneous local datasets. Such a sampling is redrawn to comply with the minimum class data of 2 on each node.

1.5. Covariance Estimation. The lifted loss function allows assigning to each point a “covariance” matrix. These are constructed in the simulations as follows: for each class, we construct the empirical diagonal covariance matrix for each class, estimated from the local observations belonging to that class. Motivated by density estimation, the empirical standard deviations were multiplied by Silverman’s bandwidth factor

$$h = \left(\frac{3n}{4} \right)^{-1/5},$$

before constructing the diagonal covariance matrix

$$\Sigma = \text{diag} \left((h\hat{\sigma}_1)^2, \dots, (h\hat{\sigma}_d)^2 \right).$$

When insufficient observations were available to estimate the covariance reliably, a small positive definite diagonal covariance matrix was substituted to maintain numerical stability during optimization.

1.6. Initialization. All distributed methods were initialized identically to ensure a fair comparison. Each node first performed 20 local optimization epochs using only its private data before any communication occurred. All methods proceeded according to their respective communication and representative-update strategies. While this isn’t strictly necessary for any of the methods, it results in cleaner optimization.

The model parameters were initialized at

$$w^{(0)} = 0, \quad b^{(0)} = 0,$$

prior to the warm-start optimization.

2. Kernel SVM Experimental Details

2.1. Experimental Configuration. Table A.2 summarizes the principal parameters used throughout the kernel SVM experiments unless otherwise stated.

TABLE A.2. Kernel SVM experimental configuration.

Parameter	Value
Datasets	Ionosphere, Australian Credit Approval
Number of simulations	100
Number of computational nodes	3
Training/test split	85% / 15%
Dirichlet parameter	$\alpha = 0.1$
Minimum samples per node	4
Minimum samples per class per node	2
Public dictionary fraction	0.10
Kernel	Gaussian RBF
Kernel bandwidth multiplier	$\frac{1}{d}$
Distributed methods regularization	$C = 100 \rightarrow (C/n)$
Classical SVM regularization	$C = 1$
Warm-start iterations	50
Warm-start learning rate	0.001
Hinge initial step size	0.1
Hinge step schedule	inverse square-root

2.2. Datasets. Experiments were performed using the Ionosphere and Australian Credit Approval datasets from the UCI Machine Learning Repository. Both datasets represent binary classification problems.

The Ionosphere dataset consists of radar returns classified as either “good” or “bad” with 351 data samples. The training dataset is 298 points and test 53. The shared dictionary is size 30 taken to be 10% of the training dataset.

The Australian Credit Approval dataset contains financial attributes used to predict whether a credit application is accepted or rejected with 690 samples. Training and testing were sizes 586 and 104 respectively with shared dictionary of size 59.

2.3. Data Preprocessing and Public Dictionary. Each dataset was randomly partitioned into training and testing sets using an 85%/15% split. Feature standardization was performed by fitting a StandardScaler using only the training data. The resulting transformation was subsequently applied to the private training data, the shared public dictionary, and the held-out test set. Consequently, no information from the testing data was used during preprocessing.

Following standardization, a shared public dictionary was selected exclusively from the training data using stratified random sampling. The public dictionary consisted of 10% of the training data for both datasets. The remaining training observations were reserved for distributed optimization and are subsequently partitioned among the nodes using a Dirichlet distribution.

2.4. Distributed Data Partitioning. After removing the public dictionary, the remaining private training data were partitioned among three computational nodes using independent Dirichlet allocations for each class with parameter

$$\alpha = 0.1.$$

The partitioning procedure was repeated until every node contained at least four observations in total and at least two observations from each class. This produced highly heterogeneous local datasets while ensuring that every node possessed sufficient data for local optimization and representative construction.

2.5. Kernel and Optimization Parameters. All representative methods employed the Gaussian radial basis function (RBF) kernel

$$K(x, y) = \exp(-\gamma \|x - y\|^2),$$

where γ was selected, after tuning, to be $\gamma = \frac{1}{d}$ where d is the dimension of the corresponding dataset.

The distributed methods used a regularization parameter of

$$C = 100,$$

divided by the total number of data n (as this constant regularizes local data as well as the added representatives), while the centralized classical support vector machine baseline used

$$C = 1.$$

Prior to distributed optimization, each node performed a warm start phase consisting of 50 gradient steps with a learning rate of 0.001. Optimization of the hinge loss methods employed an initial step size of 0.1 together with an inverse square-root step-size schedule.

APPENDIX B

COLLABORATIVE TRANSFER LEARNING DETAILS

1. CTL

Formula to Approximate Energy Distance. Below we show how we chose to approximate the energy distance between corresponding dataset features X_i and Y_i .

$$\mathbb{E} [\|X_i - Y_i\|] \approx \sqrt{\nu_{X_i Y_i}} \left(1 - \frac{C_{4X_i Y_i} + 4C_{3X_i Y_i} \delta_{\mu_i} + 2\nu_{X_i Y_i}^2 - 2\delta_{\mu_i}^4}{8\nu_{X_i Y_i}^2} \right),$$

where $\nu_{X_i Y_i} = \sigma_{X_i}^2 + \sigma_{Y_i}^2 + \delta_{\mu_i}^2$, $C_{4X_i Y_i} = \sigma_{X_i}^4 \gamma_{4X_i} + \sigma_{Y_i}^4 \gamma_{4Y_i}$, and $C_{3X_i Y_i} = \sigma_{X_i}^3 \gamma_{3X_i} - \sigma_{Y_i}^3 \gamma_{3Y_i}$, $\delta_{\mu_i} = \mu_{X_i} - \mu_{Y_i}$

We gave the following formula for the final energy computation between two node datasets:

$$H = \sum_{i=1}^d w_i H_i.$$

w_i refers to i th component of the linear model parameter w corresponding to the feature i in the dataset. Here,

$$w_i = \frac{|q_i - q_0|}{\sum_{j=1}^d |q_j - q_0|}$$

where $q_0, q_1, \dots, q_d$ are current predictions for pseudo-observations $z_0, z_1, \dots, z_d \in \mathbb{R}^d$ constructed by setting the i th component to 1 and others to 0. This weight quantifies the relative contribution of feature i to the model's predictions.

REFERENCES

1. Cenk Baykal and Daniela L Rus, *On coresets for support vector machines*, (2020).
2. Joseph Casey, Qianjiao Chen, Mengchen Fan, Baocheng Geng, Roman Shterenberg, Zhong Chen, and Keren Li, *Distributed collaborative learning with representative knowledge sharing*, *Mathematics* **13** (2025), no. 6, 1004.
3. Canyi Chen, Nan Qiao, and Liping Zhu, *Efficient distributed learning over decentralized networks with convoluted support vector machine*, *Journal of the American Statistical Association* (2025), 1–13.
4. Corinna Cortes and Vladimir Vapnik, *Support-vector networks*, *Machine learning* **20** (1995), no. 3, 273–297.
5. Erick Delage and Yinyu Ye, *Distributionally robust optimization under moment uncertainty with application to data-driven problems*, *Operations research* **58** (2010), no. 3, 595–612.
6. Yuyang Deng, Mohammad Mahdi Kamani, and Mehrdad Mahdavi, *Distributionally robust federated averaging*, *Advances in neural information processing systems* **33** (2020), 15111–15122.
7. Enmao Diao, Jie Ding, and Vahid Tarokh, *Heterofl: Computation and communication efficient federated learning for heterogeneous clients*, *arXiv preprint arXiv:2010.01264* (2020).
8. Dan Feldman and Michael Langberg, *A unified framework for approximating and clustering data*, *Proceedings of the forty-third annual ACM symposium on Theory of computing*, 2011, pp. 569–578.
9. Pedro A Forero, Alfonso Cano, and Georgios B Giannakis, *Consensus-based distributed linear support vector machines*, *Proceedings of the 9th ACM/IEEE international conference on information processing in sensor networks*, 2010, pp. 35–46.
10. Hans Graf, Eric Cosatto, Leon Bottou, Igor Dourdanovic, and Vladimir Vapnik, *Parallel support vector machines: The cascade svm*, *Advances in neural information processing systems* **17** (2004).

11. Sarel Har-Peled and Soham Mazumdar, *On coresets for k -means and k -median clustering*, Proceedings of the thirty-sixth annual ACM symposium on Theory of computing, 2004, pp. 291–300.
12. Geoffrey Hinton, Oriol Vinyals, and Jeff Dean, *Distilling the knowledge in a neural network*, arXiv preprint arXiv:1503.02531 (2015).
13. Peter Kairouz and H Brendan McMahan, *Advances and open problems in federated learning*, Foundations and trends in machine learning **14** (2021), no. 1-2, 1–210.
14. Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh, *Scaffold: Stochastic controlled averaging for federated learning*, International conference on machine learning, PMLR, 2020, pp. 5132–5143.
15. Daniel Kuhn, Peyman Mohajerin Esfahani, Viet Anh Nguyen, and Soroosh Shafieezadeh-Abadeh, *Wasserstein distributionally robust optimization: Theory and applications in machine learning*, Operations research & management science in the age of analytics, Informs, 2019, pp. 130–166.
16. Yuh-Jye Lee and Olvi L Mangasarian, *Rsvm: Reduced support vector machines*, Proceedings of the 2001 SIAM international conference on data mining, SIAM, 2001, pp. 1–17.
17. Daliang Li and Junpu Wang, *Fedmd: Heterogenous federated learning via model distillation*, arXiv preprint arXiv:1910.03581 (2019).
18. Keren Li, *Representative learning for distributed learning with heterogeneity and asynchrony*, Journal of Computational and Graphical Statistics **0** (2026), no. 0, 1–24.
19. Keren Li and Jie Yang, *Score-matching representative approach for big data analysis with generalized linear models*, Electronic Journal of Statistics **16** (2022), no. 1, 592–635.
20. Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith, *Federated optimization in heterogeneous networks*, Proceedings of Machine learning and systems **2** (2020), 429–450.
21. Ilan Lobel and Asuman Ozdaglar, *Distributed subgradient methods for convex optimization over random networks*, IEEE Transactions on Automatic Control **56** (2010), no. 6, 1291–1306.
22. Hanlin Lu, Ming-Ju Li, Ting He, Shiqiang Wang, Vijaykrishnan Narayanan, and Kevin S Chan, *Robust coreset construction for distributed machine learning*, IEEE Journal on Selected Areas in Communications **38** (2020), no. 10, 2400–2417.

23. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas, *Communication-efficient learning of deep networks from decentralized data*, Artificial intelligence and statistics, Pmlr, 2017, pp. 1273–1282.
24. John Platt, *Sequential minimal optimization: A fast algorithm for training support vector machines*, (1998).
25. Ali Rahimi and Benjamin Recht, *Random features for large-scale kernel machines*, Advances in neural information processing systems **20** (2007).
26. Shiori Sagawa, Pang Wei Koh, Tatsunori B Hashimoto, and Percy Liang, *Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization*, arXiv preprint arXiv:1911.08731 (2019).
27. Si Si, Cho-Jui Hsieh, and Inderjit S Dhillon, *Memory efficient kernel approximation*, Journal of Machine Learning Research **18** (2017), no. 20, 1–32.
28. Virginia Smith, Chao-Kai Chiang, Maziar Sanjabi, and Ameet S Talwalkar, *Federated multi-task learning*, Advances in neural information processing systems **30** (2017).
29. Sebastian U Stich, *Local sgd converges fast and communicates little*, arXiv preprint arXiv:1805.09767 (2018).
30. Alysa Ziyang Tan, Han Yu, Lizhen Cui, and Qiang Yang, *Towards personalized federated learning*, IEEE transactions on neural networks and learning systems **34** (2022), no. 12, 9587–9603.
31. Yichuan Tang, *Deep learning using linear support vector machines*, arXiv preprint arXiv:1306.0239 (2013).
32. Davoud Ataee Tarzanagh, Yingcong Li, Christos Thrampoulidis, and Samet Oymak, *Transformers as support vector machines*, arXiv preprint arXiv:2308.16898 (2023).
33. Ivor W Tsang, Andras Kocsor, and James T Kwok, *Simpler core vector machines with enclosing balls*, Proceedings of the 24th international conference on Machine learning, 2007, pp. 911–918.
34. Ivor W Tsang, James T Kwok, Nello Cristianini, et al., *Core vector machines: Fast svm training on very large data sets.*, Journal of machine Learning research **6** (2005), no. 4.
35. Vladimir Vapnik, *The nature of statistical learning theory*, Springer science & business media, 2013.

36. Vladimir N Vapnik, *An overview of statistical learning theory*, IEEE transactions on neural networks **10** (1999), no. 5, 988–999.
37. Boxiang Wang, Le Zhou, Yuwen Gu, and Hui Zou, *Density-convoluted support vector machines for high-dimensional classification*, IEEE transactions on Information Theory **69** (2022), no. 4, 2523–2536.
38. Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H Vincent Poor, *Tackling the objective inconsistency problem in heterogeneous federated optimization*, Advances in neural information processing systems **33** (2020), 7611–7623.
39. Christopher Williams and Matthias Seeger, *Using the nyström method to speed up kernel machines*, Advances in neural information processing systems **13** (2000).
40. Chuhan Wu, Fangzhao Wu, Lingjuan Lyu, Yongfeng Huang, and Xing Xie, *Communication-efficient federated learning via knowledge distillation*, Nature communications **13** (2022), no. 1, 2032.
41. Rui Zhang and Quanyan Zhu, *Consensus-based transfer linear support vector machines for decentralized multi-task multi-agent learning*, 2018 52nd Annual Conference on Information Sciences and Systems (CISS), IEEE, 2018, pp. 1–6.
42. Ruiliang Zhang and James Kwok, *Asynchronous distributed admm for consensus optimization*, International conference on machine learning, PMLR, 2014, pp. 1701–1709.
43. Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen, *Dataset condensation with gradient matching*, arXiv preprint arXiv:2006.05929 (2020).
44. Duo Zheng, Keren Li, and Jie Yang, *Response-aided score-matching representative approaches for big data analysis and model selection under generalized linear models*, Algorithms **17** (2024), no. 10.